



16th Lesson

Stream Processing:

- ~~Apache Storm~~
- Spark Structured Streaming

Spark Structured Streaming



- Structured Streaming provides
 - fast, scalable,
 - fault-tolerant,
 - end-to-end exactly-once
 - stream processing without the user having to reason about streaming.

Spark Structured Streaming – Micro-batch



- <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- «Internally, by default, Structured Streaming queries are processed using a *micro-batch processing* engine, which processes data streams as a series of small batch jobs thereby achieving end-to-end latencies as low as **100 milliseconds** and exactly-once fault-tolerance guarantees. However, since Spark 2.3, we have introduced a new low-latency processing mode called **Continuous Processing**, which can achieve end-to-end latencies as low as **1 millisecond** with at-least-once guarantees. Without changing the Dataset/DataFrame operations in your queries, you will be able to choose the mode based on your application requirements.»

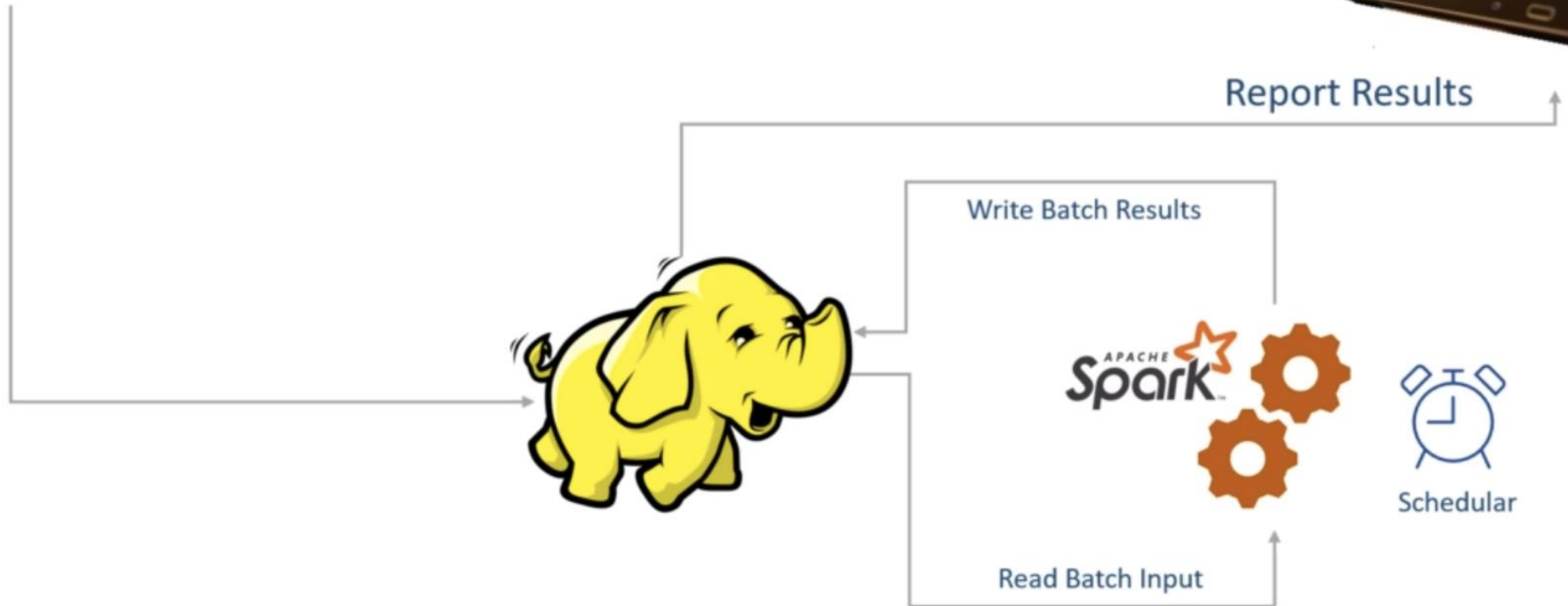
Advantages of Structured Streaming



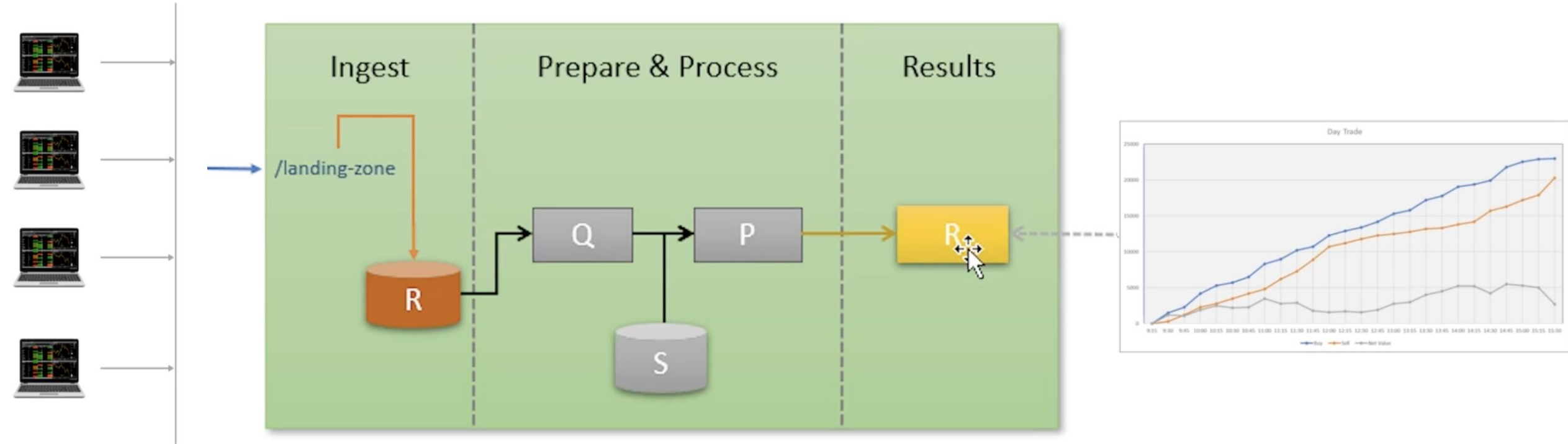
- Streaming code looks a lot like the equivalent of non-streaming code.
- Structured data allows spark to represent data more efficiently.
- SQL-style queries allow for query optimization opportunities and even better performance.
- More compatibility with other spark components like MLlib

<https://github.com/LearningJournal/Spark-Streaming-In-Python>

Spark Structured Streaming Framework



Data Engineering Platform



- **Ingest:**

- La fase di ingestion dei dati prevede la raccolta dei dati provenienti da diverse fonti (ad es., sistemi di trading).
- I dati vengono archiviati in una *landing zone* e memorizzati in un repository (R).
- Fase di pre-elaborazione e pulizia.

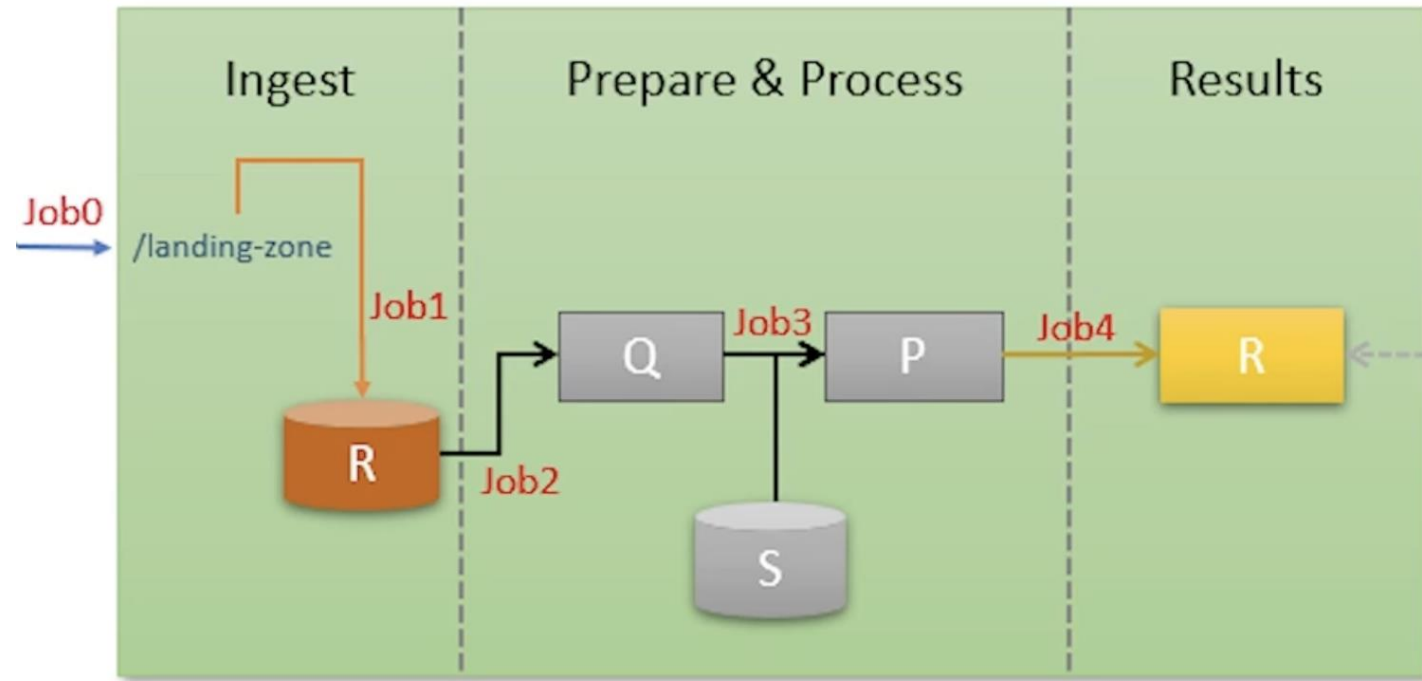
- **Prepare & Process:**

- I dati vengono preparati (Q) e processati (P).
- Memorizzazione temporanea dello stato (S).

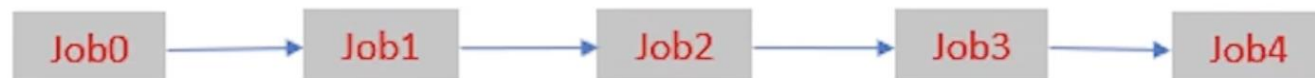
- **Results:**

- I risultati vengono scritti nel sink finale (R).
- Visualizzazione tramite applicazioni di consumo, come un grafico di riepilogo delle operazioni commerciali.

Data Engineering Platform



Data Pipeline (Workflow)



- Pipeline suddivisa in vari job sequenziali:
 - **Job 0:** Estrazione dati da sistema sorgente.
 - **Job 1:** Ingestione dati nella *landing zone*.
 - **Job 2:** Preparazione dei dati (pulizia e trasformazione).
 - **Job 3:** Elaborazione dati (aggregazioni, calcoli).
 - **Job 4:** Memorizzazione del risultato finale e visualizzazione.
- Ogni job rappresenta una fase del flusso di dati dall'ingestione fino alla presentazione finale.

Frequenza di Esecuzione dei Job



- La frequenza dei job dipende dal tipo di elaborazione richiesta:
 - **Batch:**
 - Frequenza mensile, settimanale, giornaliera o ogni ora.
 - **Near Real-Time:**
 - Frequenza ogni 15 minuti, ogni minuto, o ogni secondo.
 - **Real-Time:**
 - Elaborazione continua o basata su millisecondi.
 - **Gestione della Backpressure:**
 - Occorre tener conto del tempo di avvio, tempo di elaborazione e pulizia, bilanciando la pressione sul sistema.

Gestione dei Record in Ritardo



New records till 10.00 AM

Time	Sale	Purchase
9.32	1200	0
9.33	900	0
9.35	0	800
9.45	1300	0
9.52	0	2200
	3400	3000

Iteration 1 @ 10.00

Aggregate calculated at 10.00 AM

Time	Total Sales	Total Purchase
10.00	3400	3000

 ✓

New records till 10.30 AM

Time	Sale	Purchase
9.59	400	0
10.03	1300	0
10.05	0	800
10.02	0	600
10.15	900	0
10.18	0	1500
10.26	500	0
	2700	2900

Iteration 2 @ 10:30

Aggregate calculated at 10.30 AM

Time	Total Sales	Total Purchase
10.00	3400	3000
10.30	2700	2900

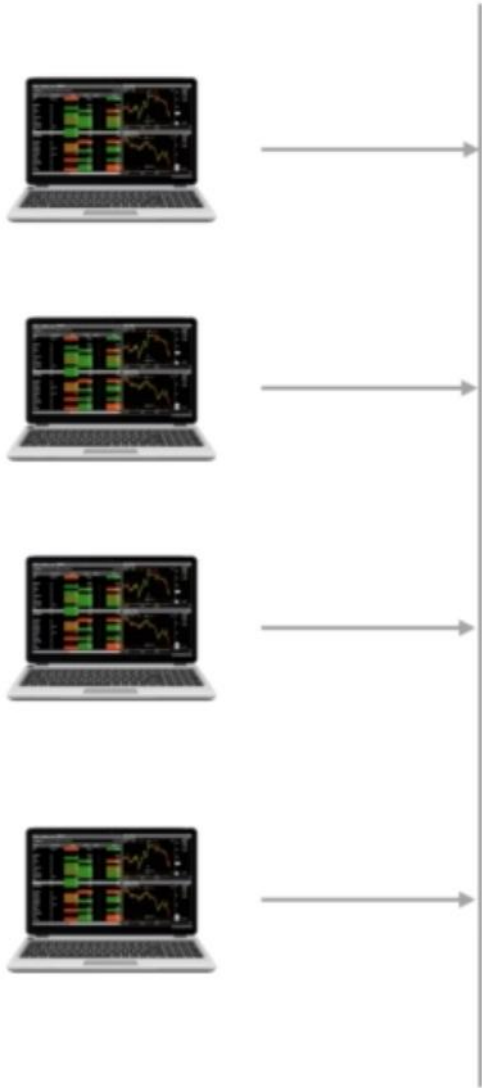
 ✗ ✓

Punti Chiave per Spark Structured Streaming



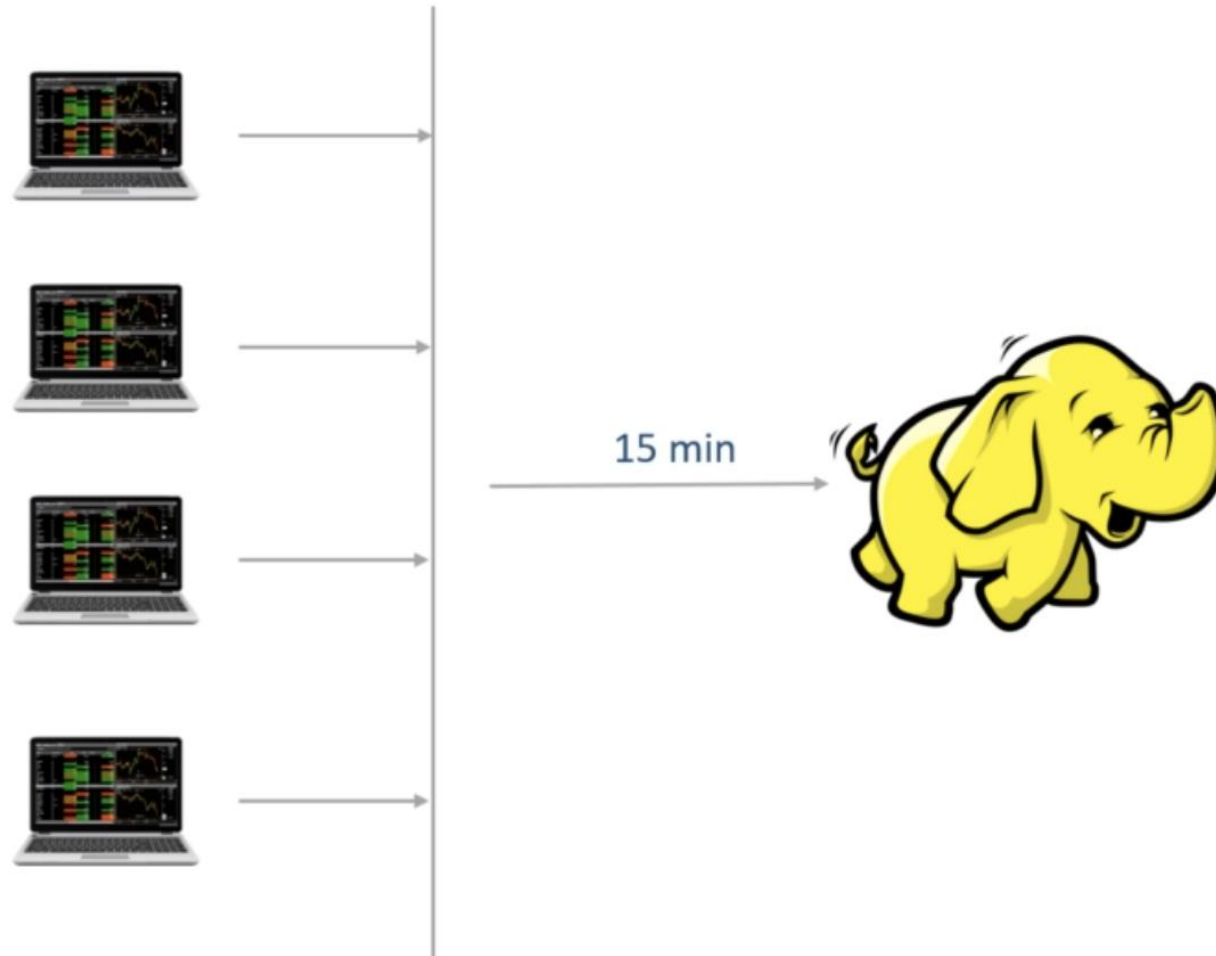
- **Backpressure:**
 - Gestione della pressione del flusso di dati per evitare sovraccarichi.
- **Workflow Continuo:**
 - Pianificazione dei job per un ascolto continuo e un'elaborazione costante dei dati.
- **Checkpoint:**
 - Creazione e gestione dei checkpoint per garantire elaborazione incrementale e tolleranza ai guasti.
- **Fault Tolerance:**
 - Meccanismi di tolleranza ai guasti tramite checkpoint transazionali.
- **State Management:**
 - Gestione dello stato aggregato e aggiornamento dello stato per i dati in arrivo in ritardo.

Spark Structured Streaming Framework

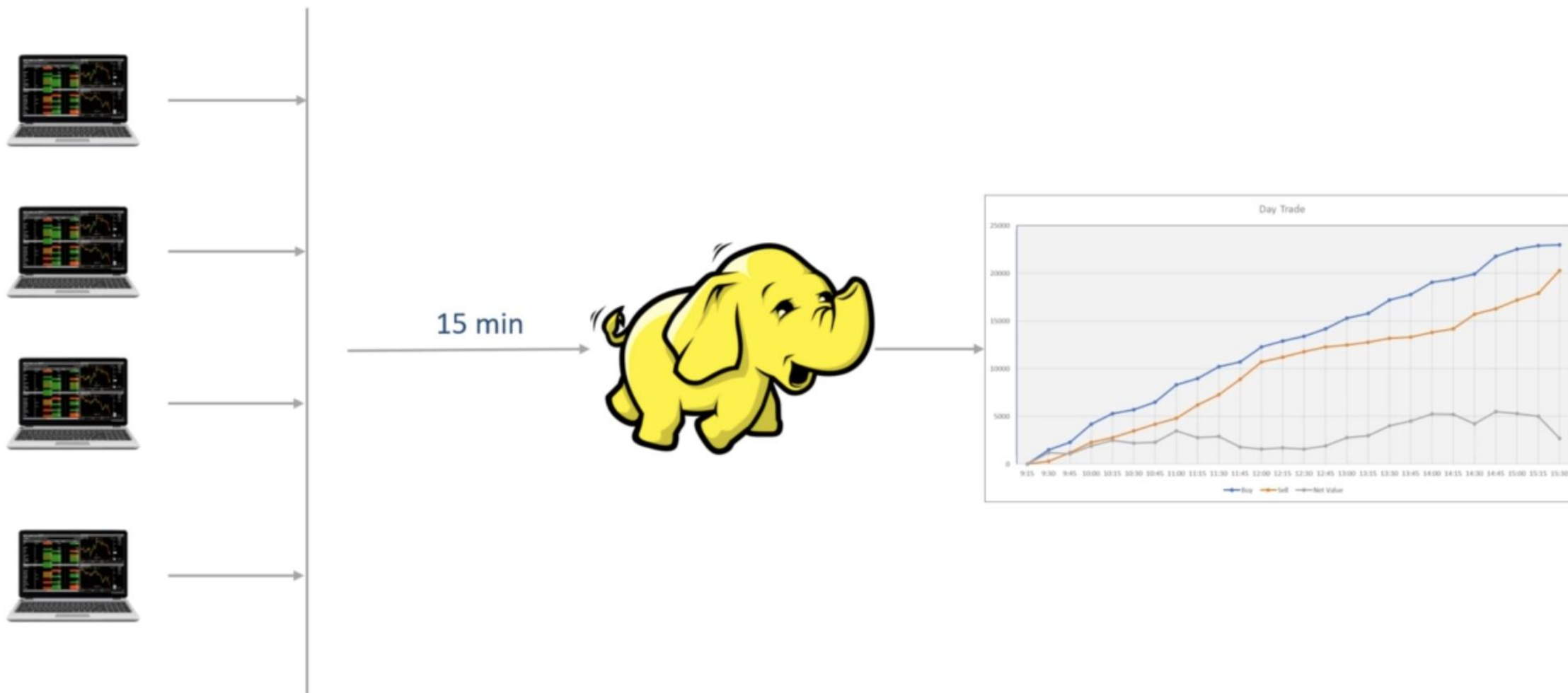


- | | |
|---------------------------|---|
| 1. Exchange Code | - Stock exchange code such as NSE, BSE, etc. |
| 2. Stock Item Code | - Stock Item code such as ASIAN PAINT, RELIANCE, etc. |
| 3. Trade Type | - Transaction type such as Buy or Sell |
| 4. Trade Quantity | - Number of unites traded |
| 5. Trade Price | - Price per unit of trade |

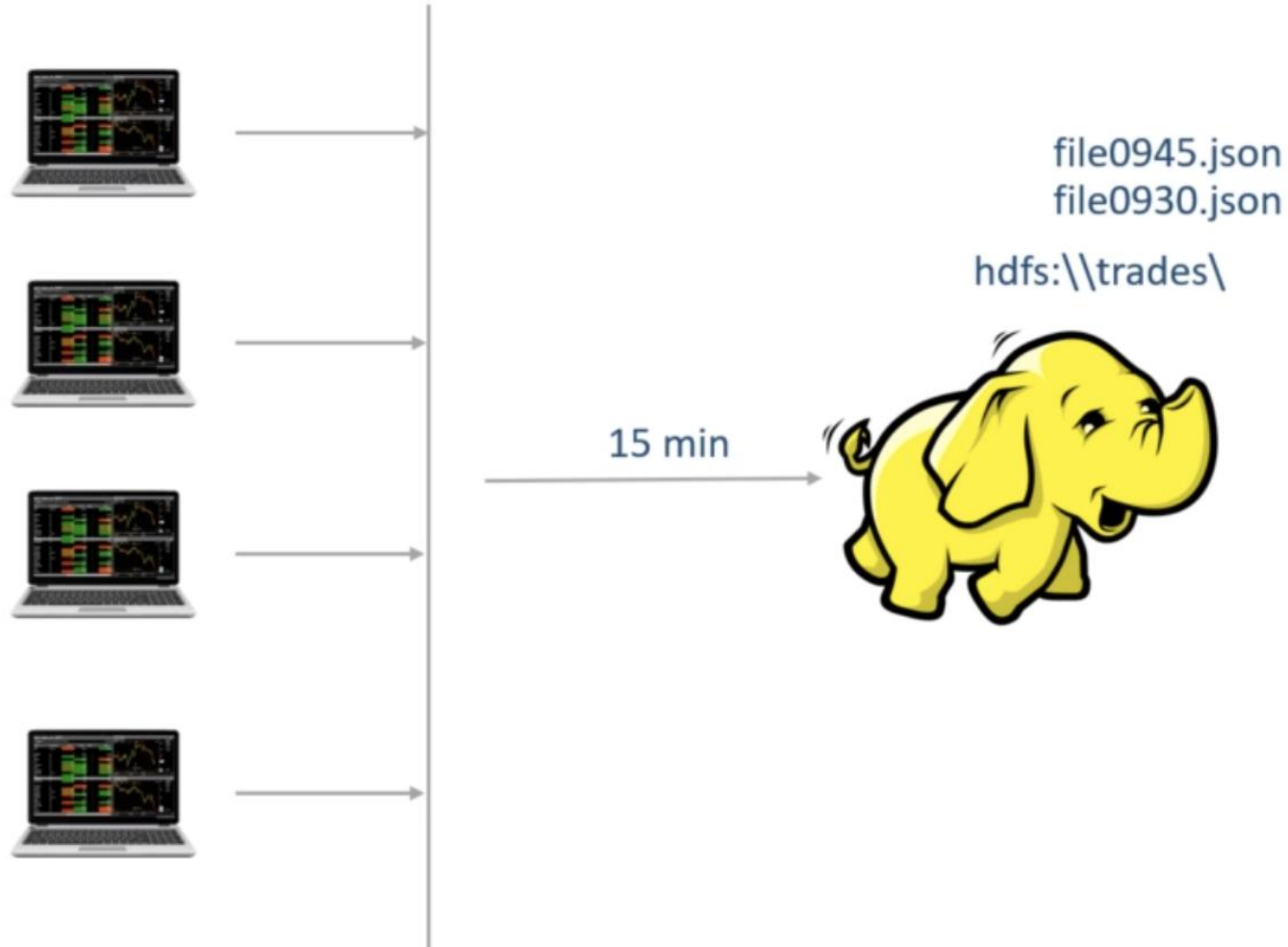
Spark Structured Streaming Framework



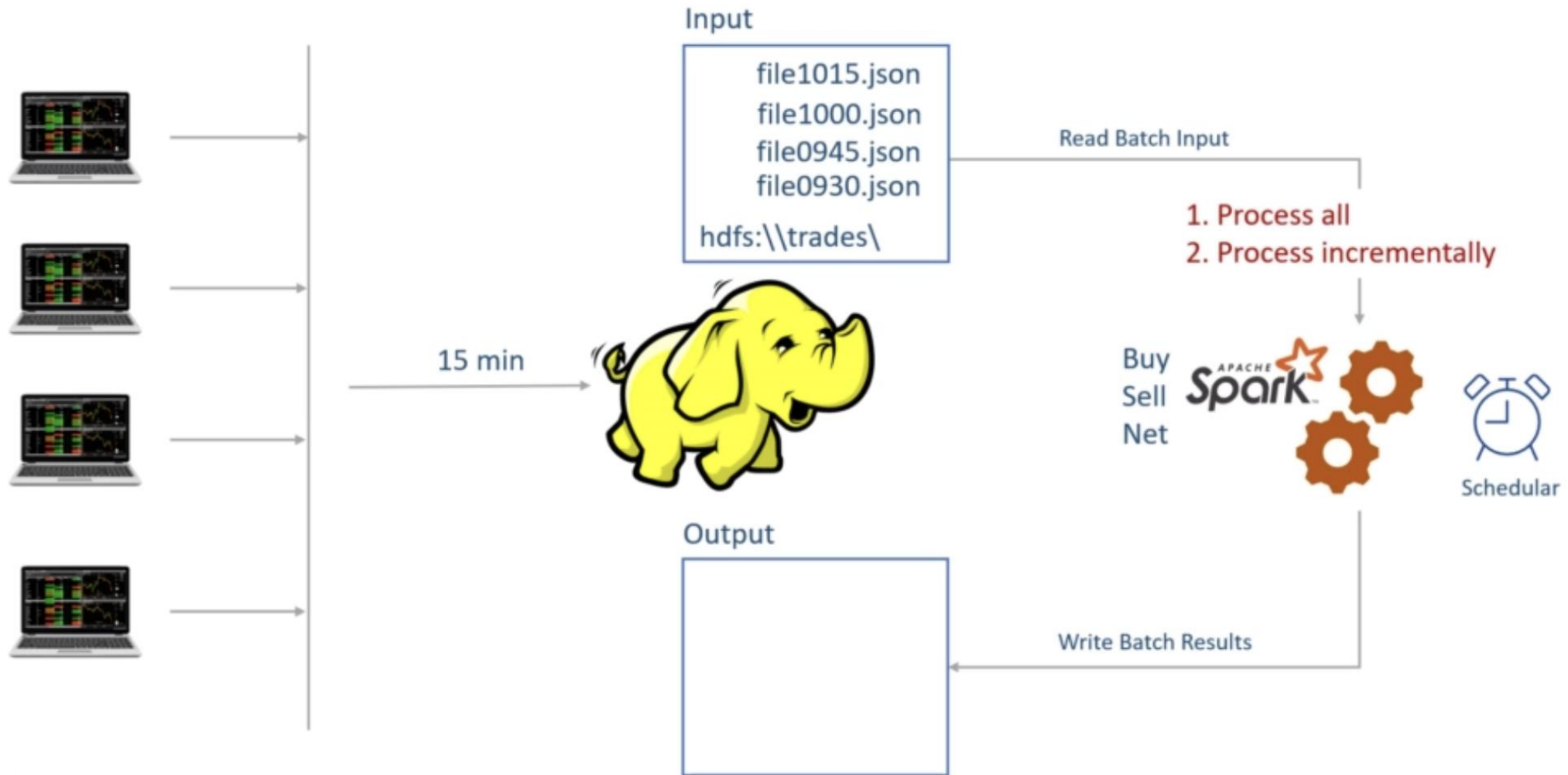
Spark Structured Streaming Framework

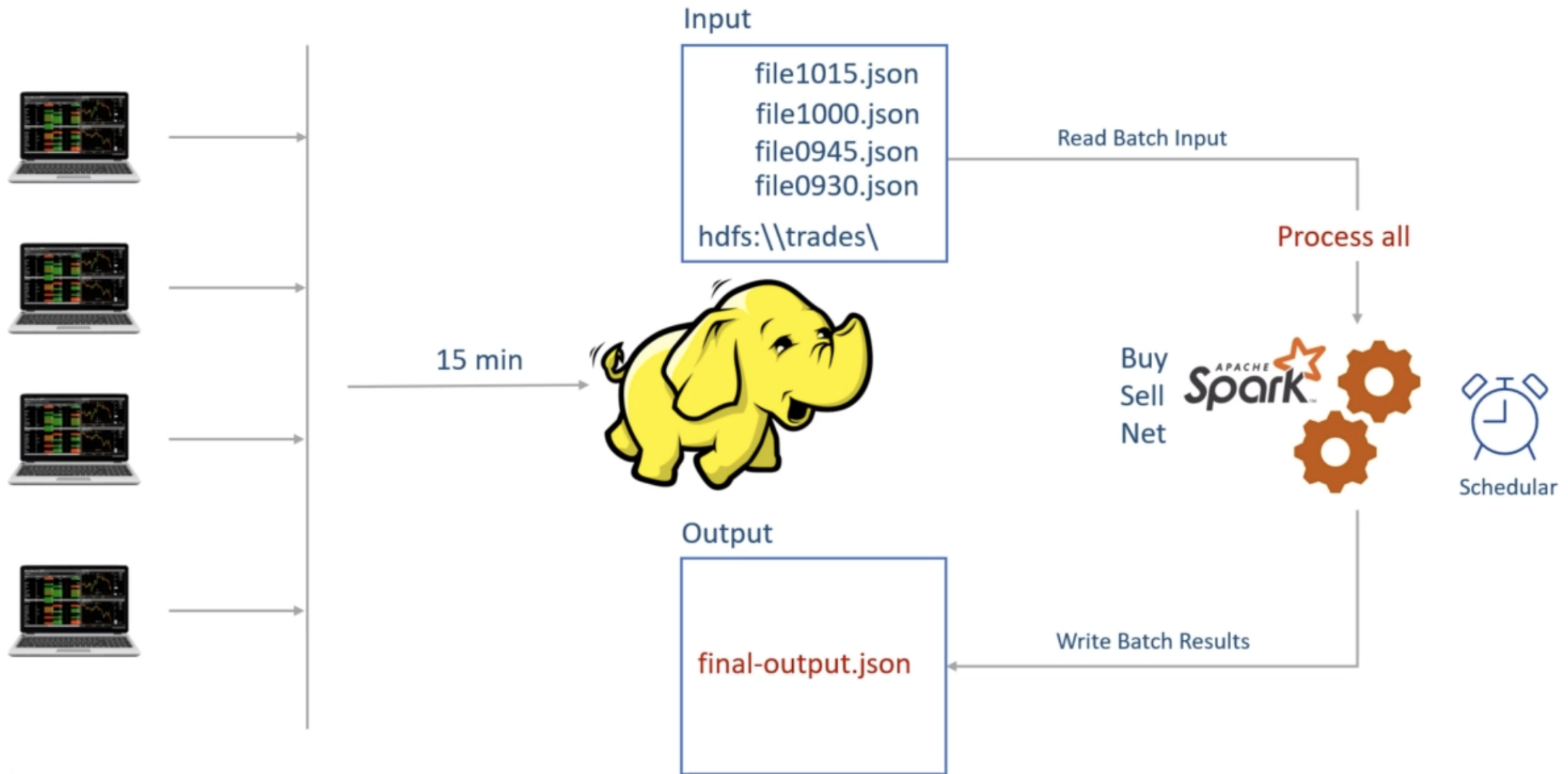


Spark Structured Streaming Framework

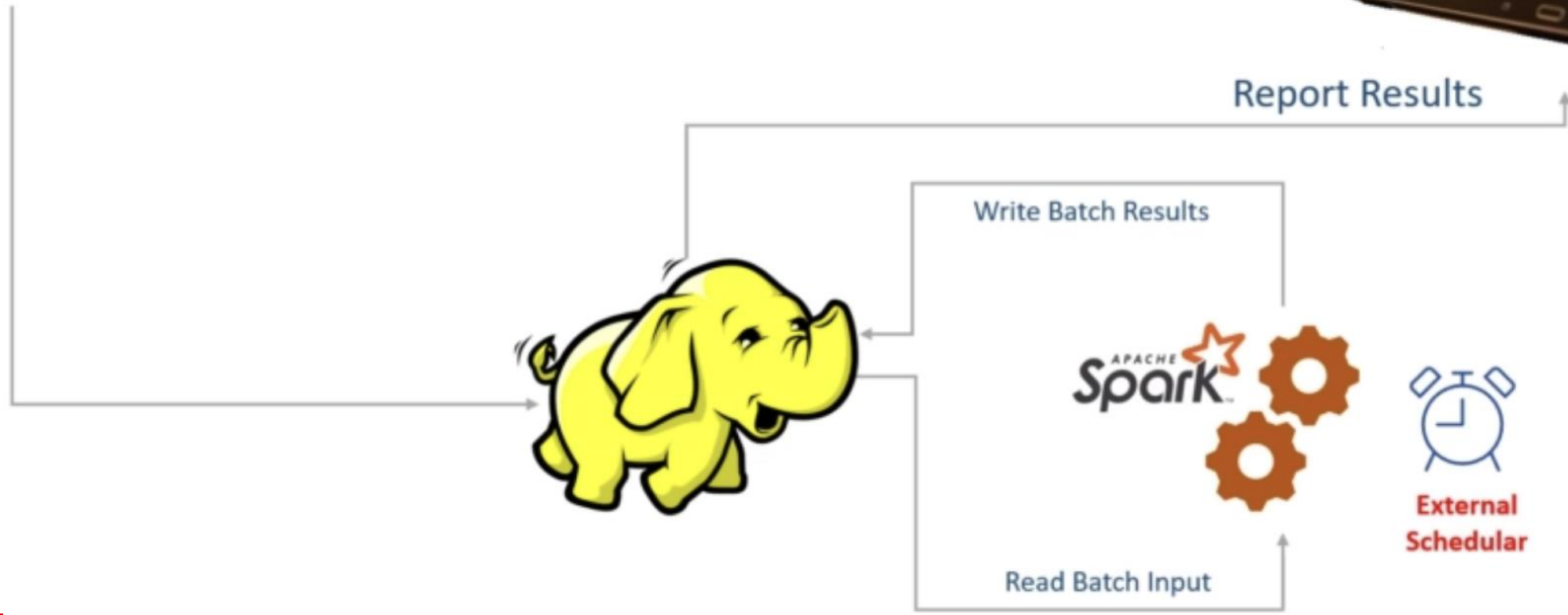


Spark Structured Streaming Framework

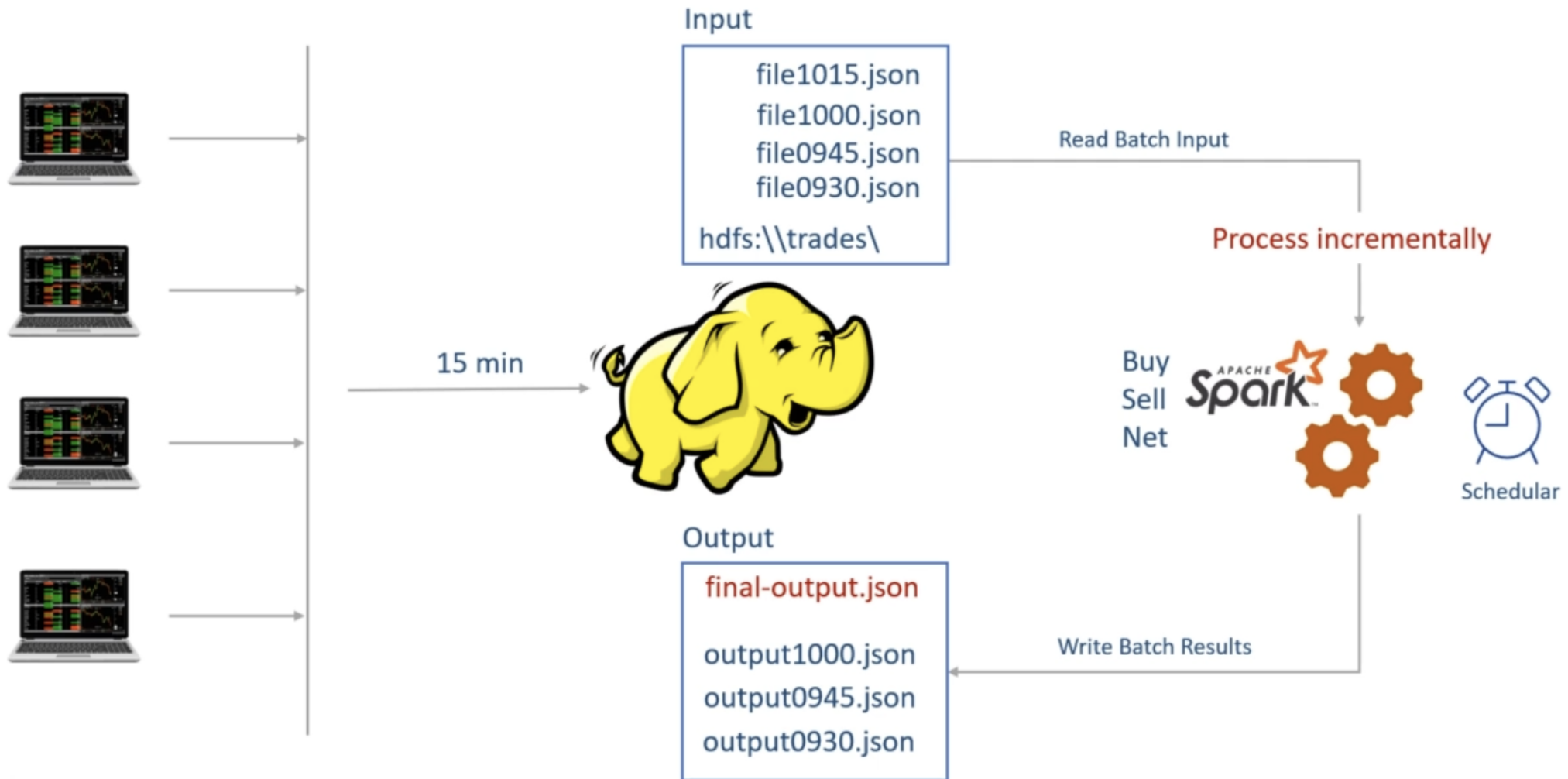




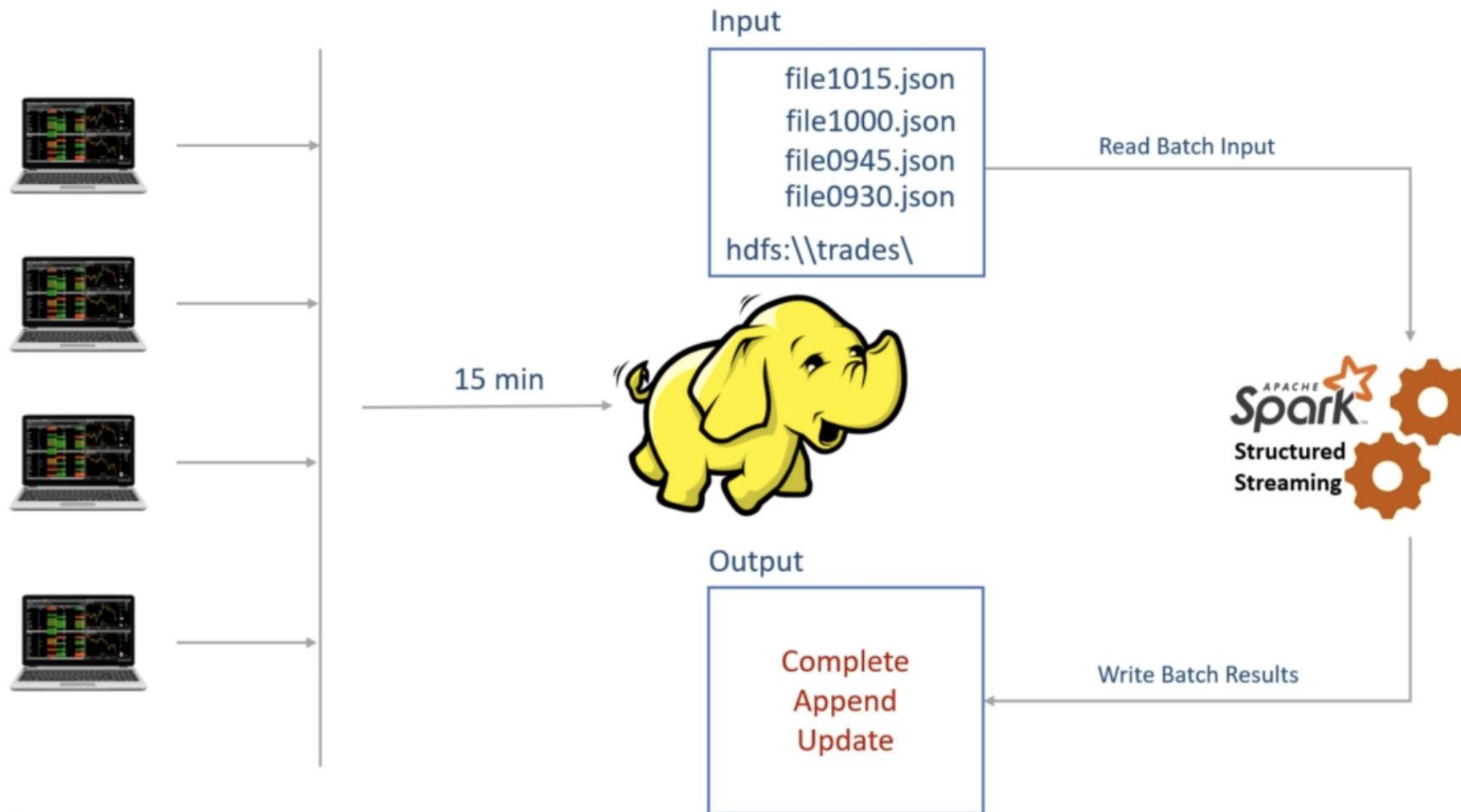
Spark Structured Streaming Framework



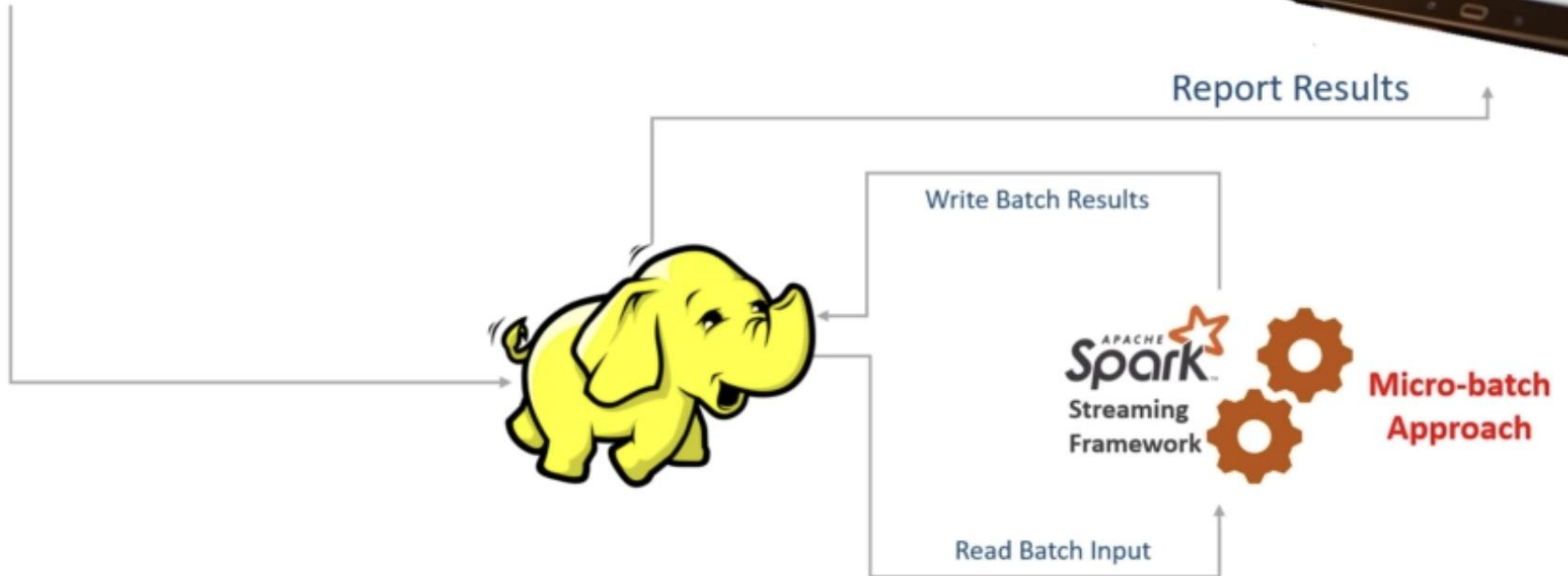
Spark Structured Streaming Framework



Spark Structured Streaming Framework



Spark Structured Streaming Framework



Spark Structured Streaming Framework



1. Automatic Looping between micro-batches.
2. Batch start and end position management.
3. Intermediate state management
4. Combining results to the previous batch results.
5. Fault tolerance and Restart management



Spark Streaming Sources

1. Socket source
2. Rate source
3. File source
4. Kafka source



Output Modes

1. Append
2. Update
3. Complete



Append Mode

Allowed only when Spark knows the following
Record will not update or change in future

Output Modes !!!Attenzione!!!



- L'**output mode "append"** in Spark Structured Streaming **non può essere utilizzato con funzioni di aggregazione** perché l'append richiede che ogni riga prodotta sia definitiva e non debba essere modificata in futuro. Tuttavia, nelle funzioni di aggregazione, i risultati intermedi potrebbero cambiare man mano che arrivano nuovi dati, il che rende impossibile garantire che le righe prodotte siano definitive al momento della scrittura.
- **Modalità di Output Alternativa:**
 - **Update Mode:** In questa modalità, Spark può aggiornare i risultati già calcolati quando arrivano nuovi dati, il che rende possibile l'aggiornamento degli aggregati esistenti.
 - **Complete Mode:** In questa modalità, Spark riscrive completamente tutti i risultati dell'aggregazione ogni volta che riceve nuovi dati.

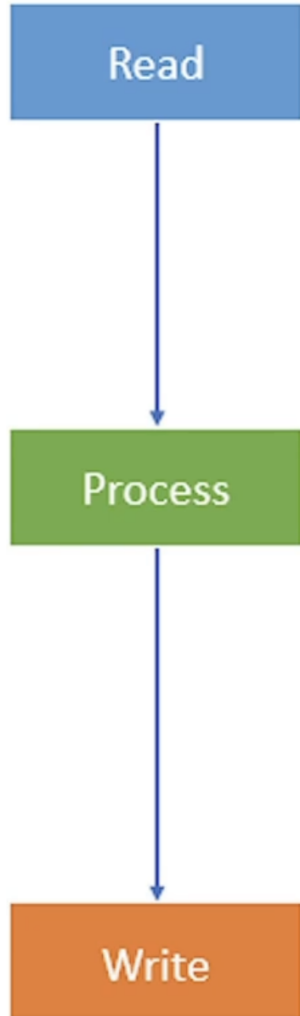
Spark Structured Streaming Template



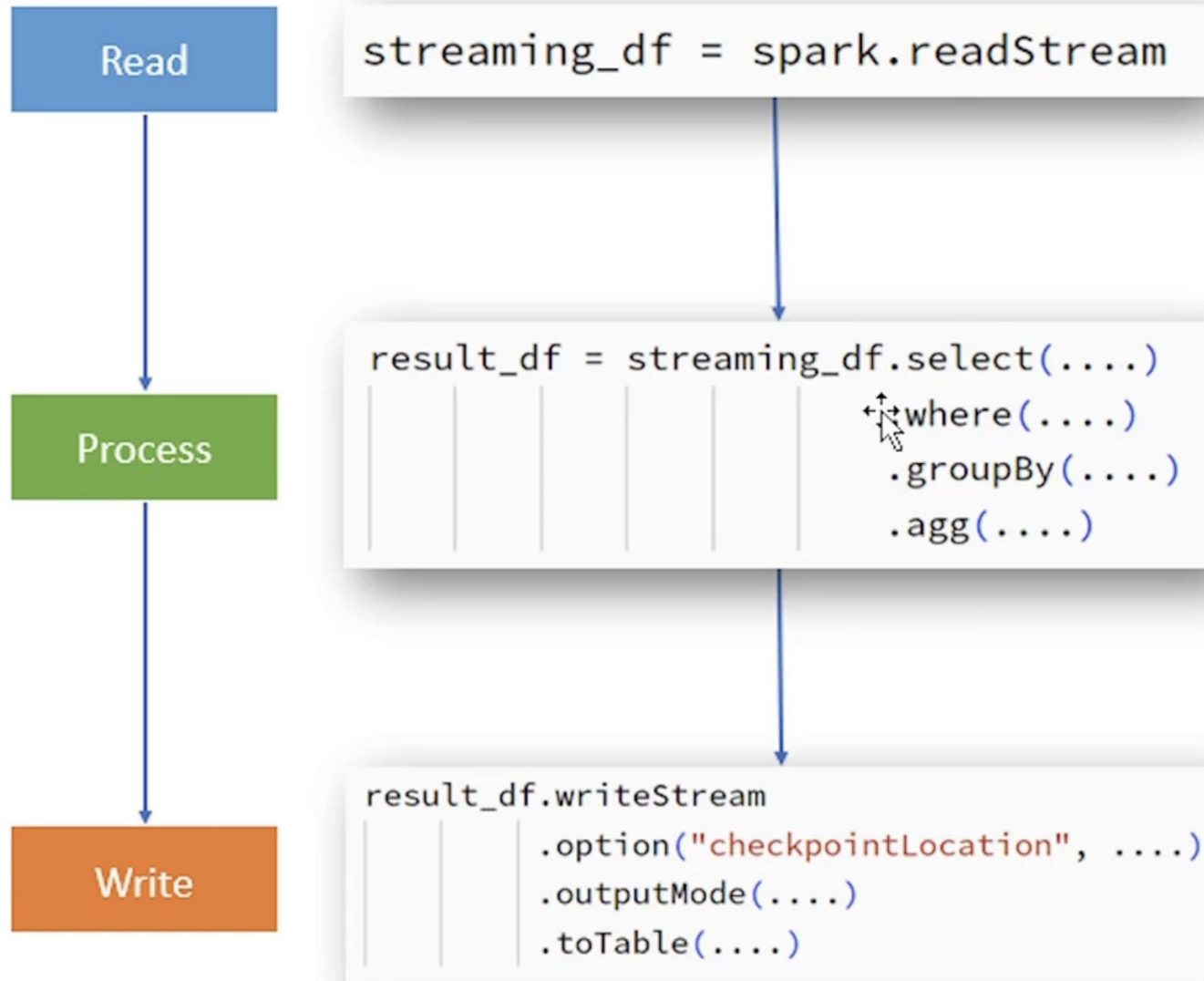
Typical structure of a Spark Streaming application

1. Read Streaming Source – Input Dataframe
2. Process and Transform input DataFrame – Output DataFrame
3. Write the output – Streaming Sink

Spark Structured Streaming Template



Spark Structured Streaming Template



Spark Structured Streaming Execution Plan



```
# Read
lines_df = spark.readStream \
    .format("socket") \
    .option("host", "localhost") \
    .option("port", "9999") \
    .load()

# Transform
words_df = lines_df.select(expr("explode(split(value, ' ')) as word"))
counts_df = words_df.groupBy("word").count()

# Sink
word_count_query = counts_df.writeStream \
    .format("console") \
    .option("checkpointLocation", "chk-point-dir") \
    .outputMode("complete") \
    .start()
```

Spark SQL
Engine

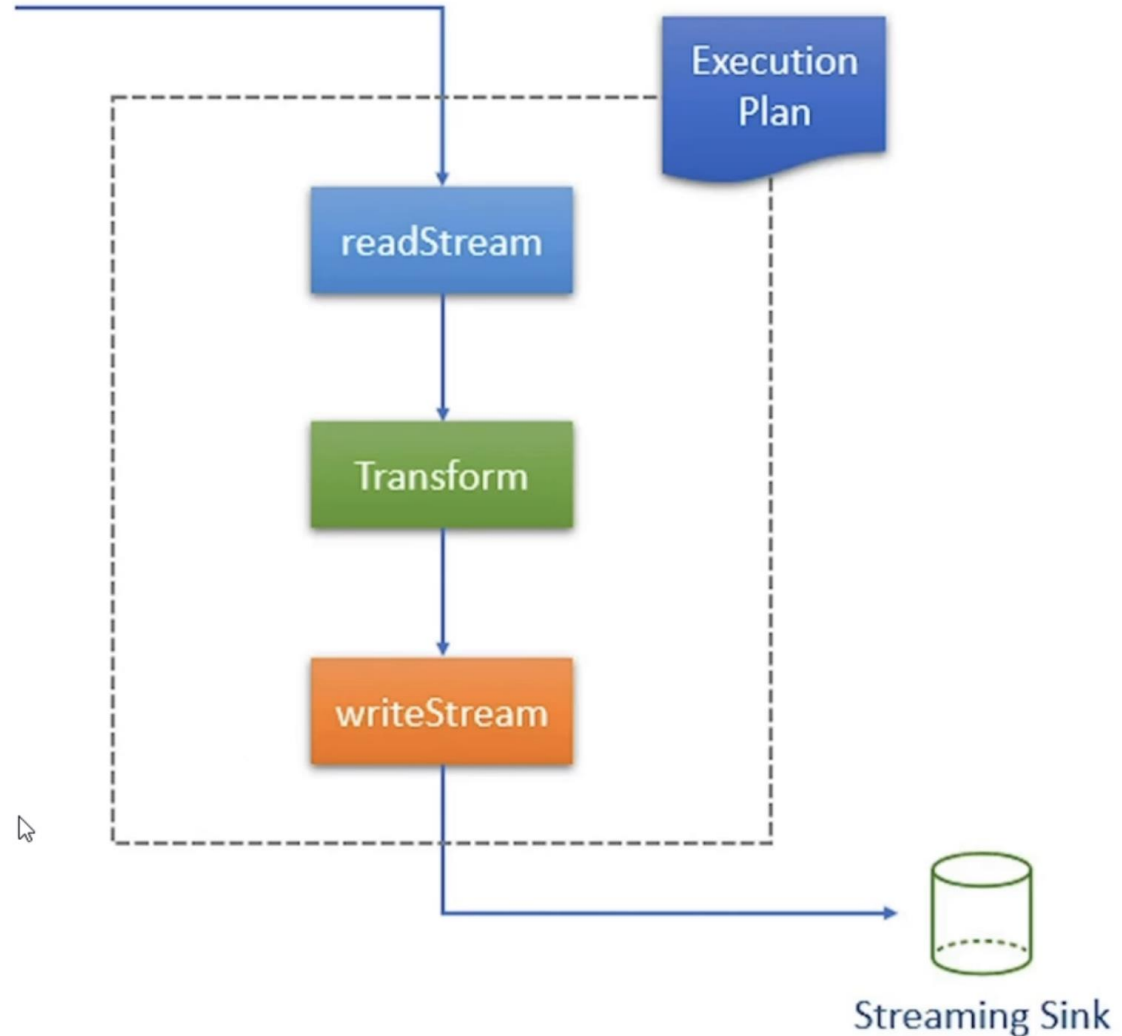
Execution
Plan

Spark Structured Streaming Template

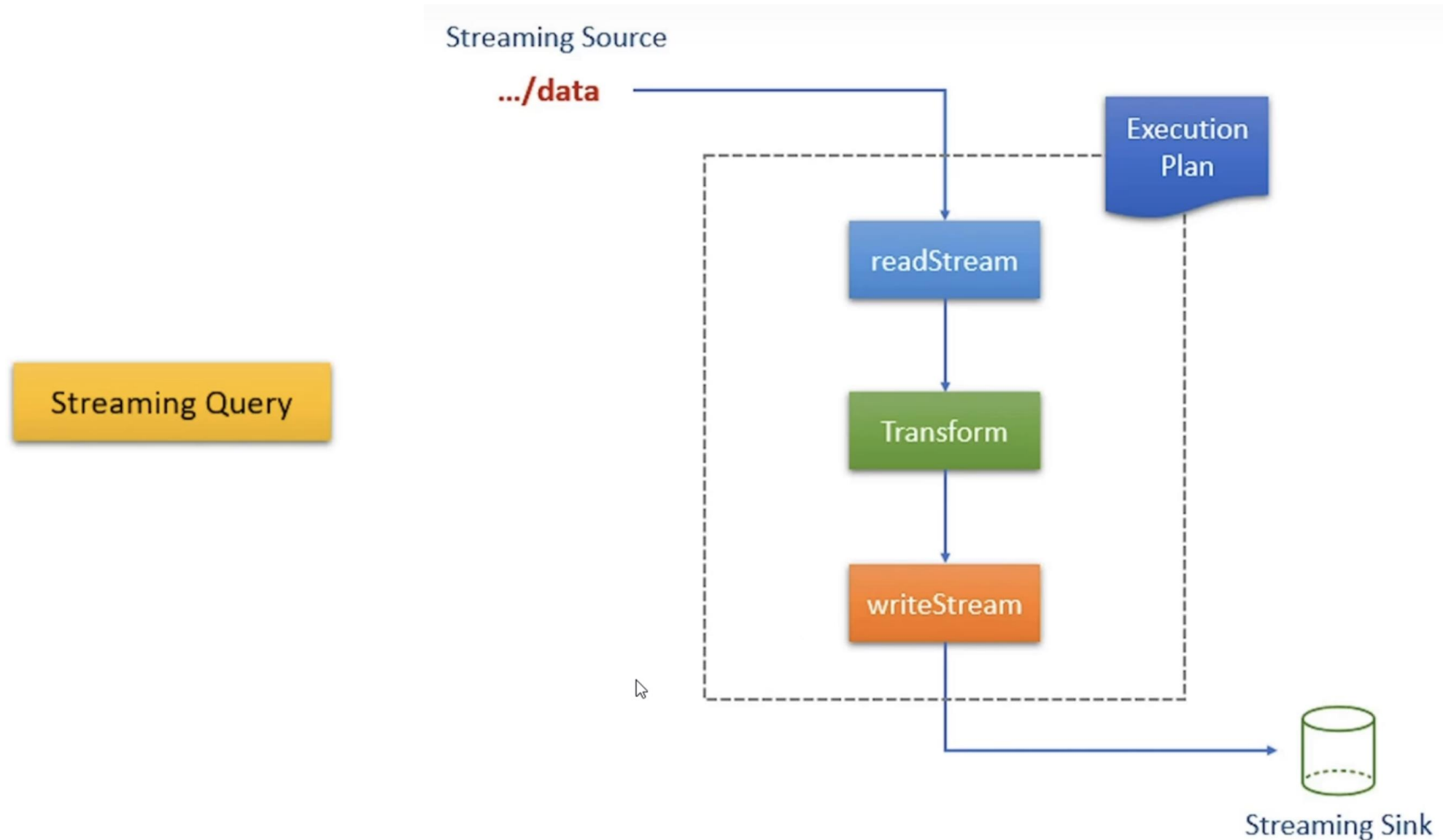


Streaming Source

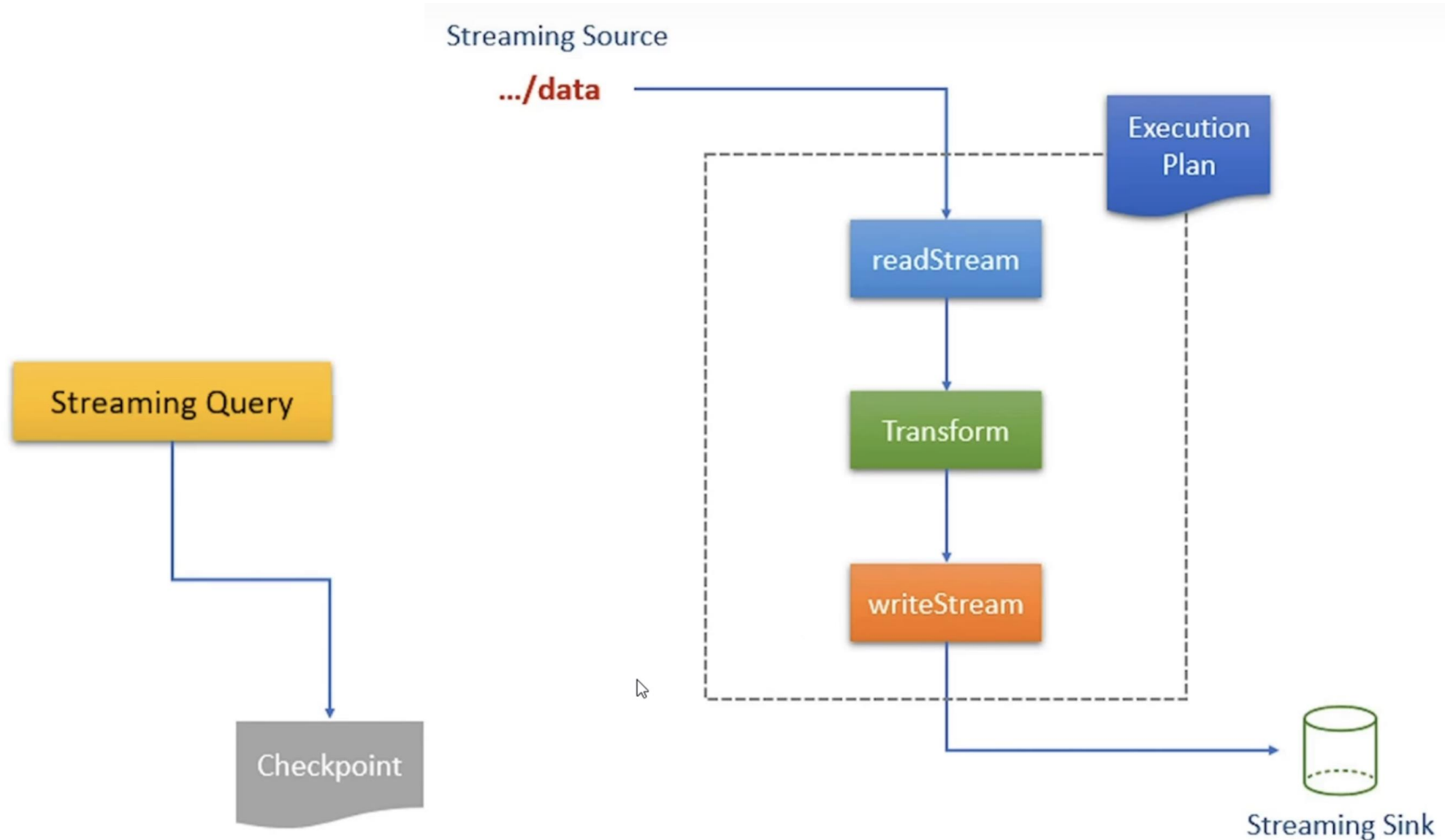
.../data



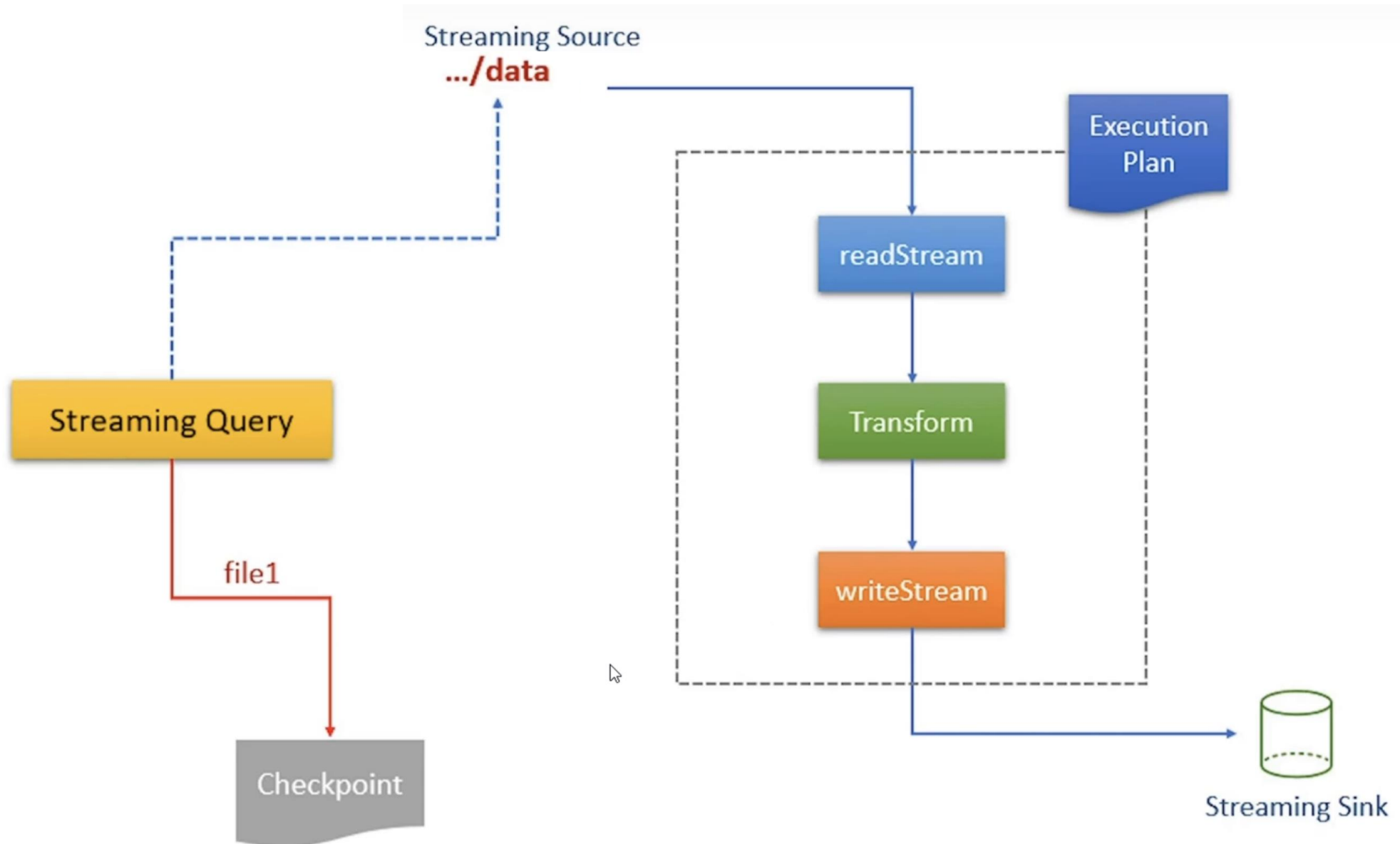
Spark Structured Streaming Template



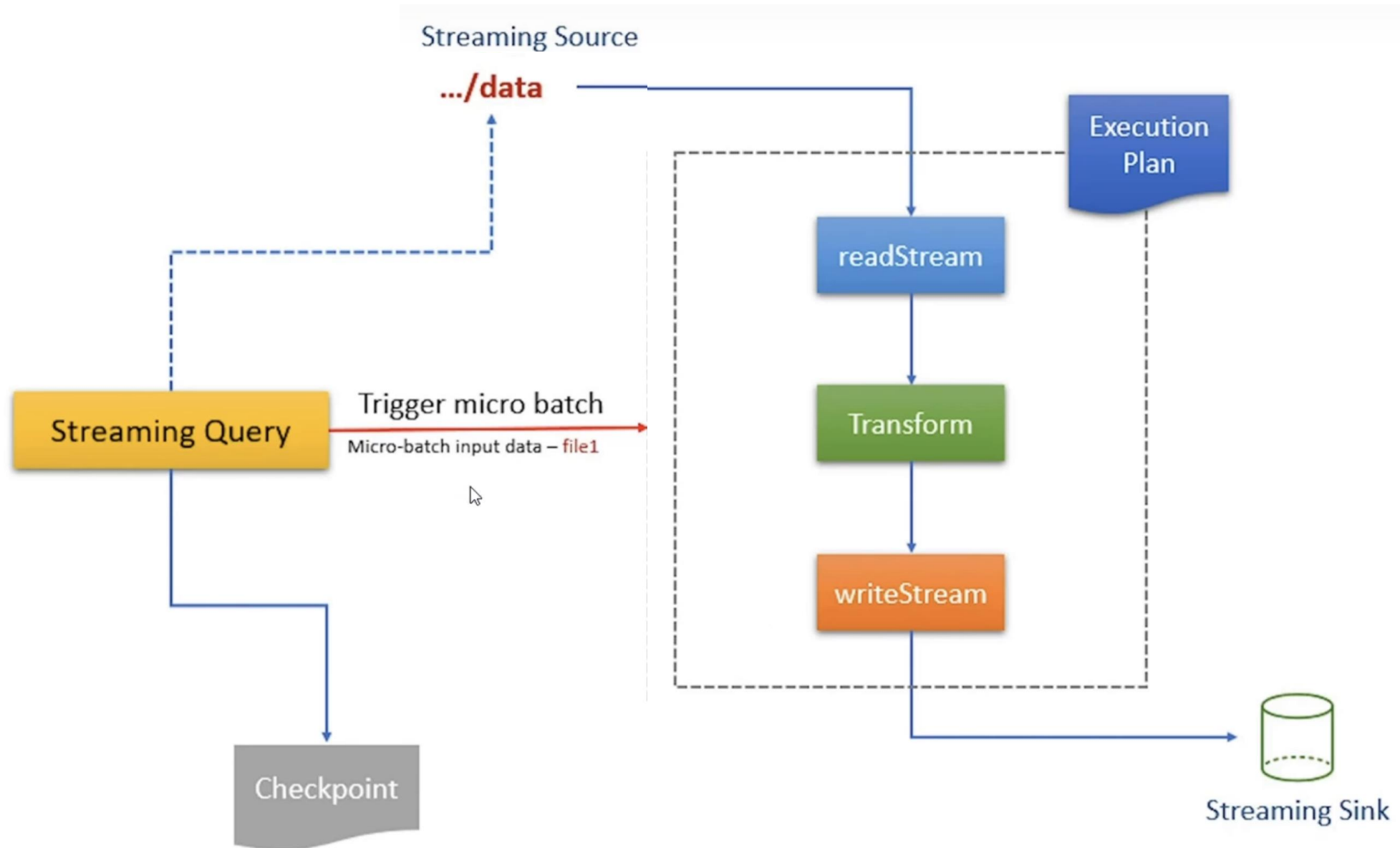
Spark Structured Streaming Template



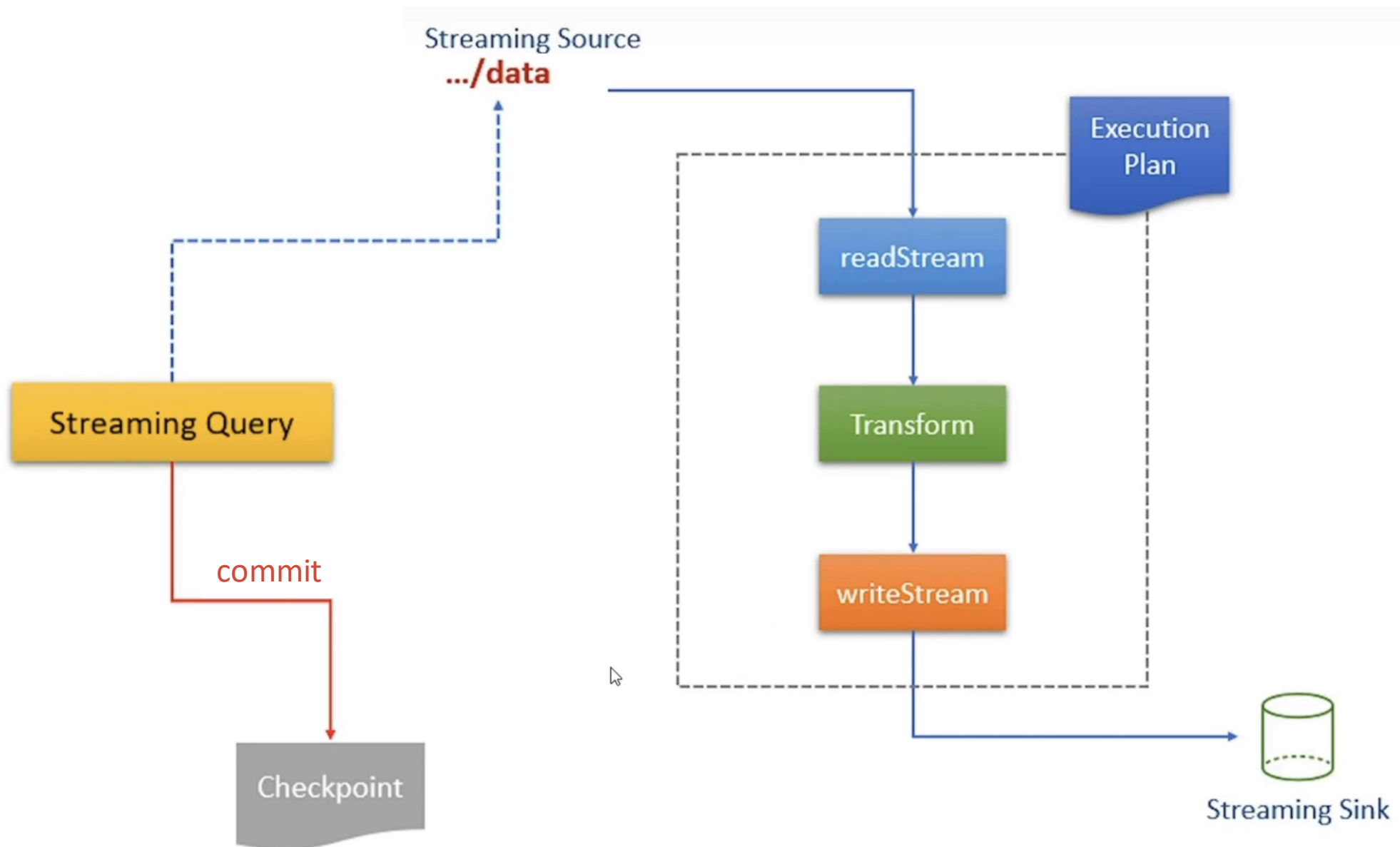
Spark Structured Streaming Template



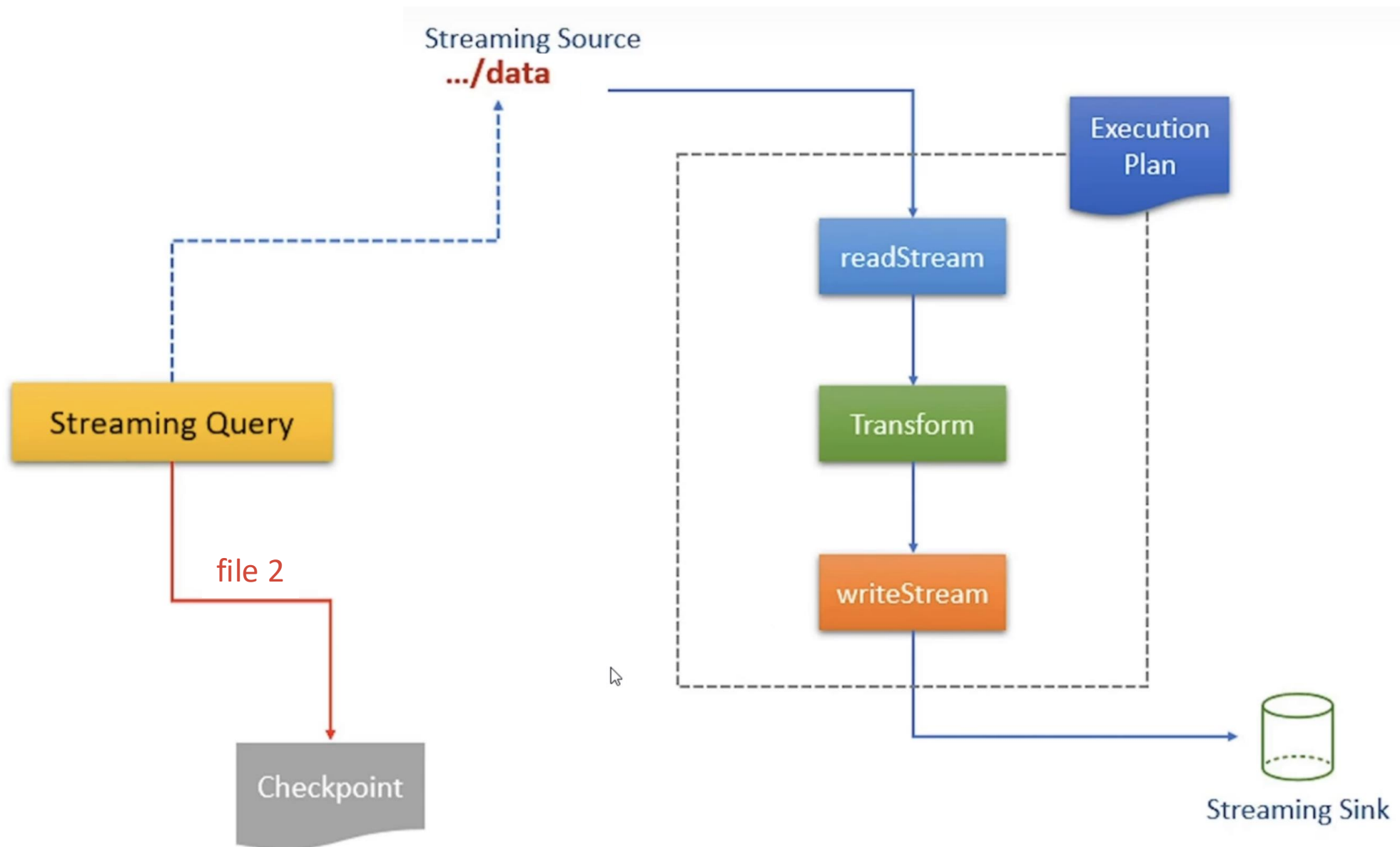
Spark Structured Streaming Template



Spark Structured Streaming Template



Spark Structured Streaming Template



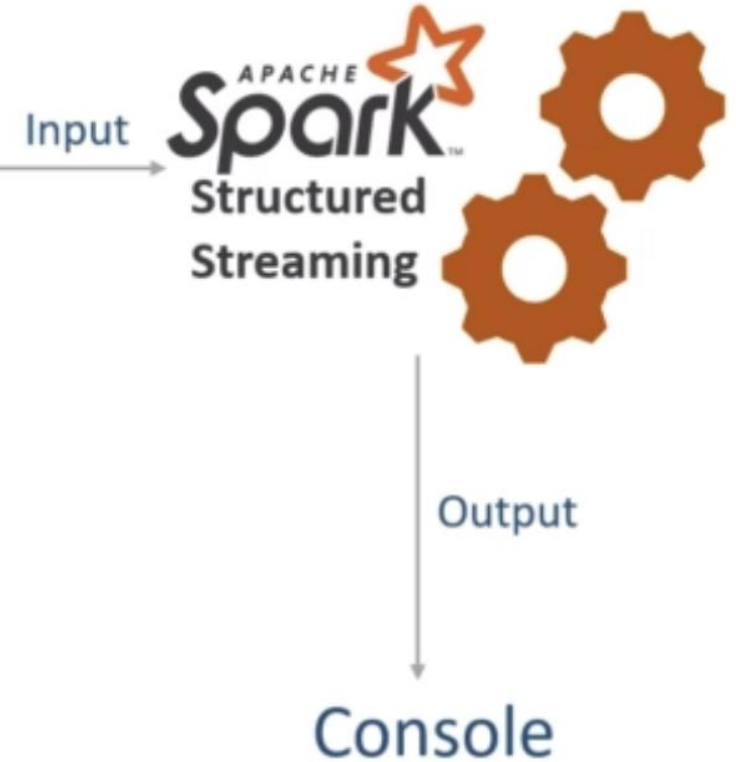
Socket Source Example



```
Command Prompt - ncat -lk 9999
Microsoft Windows [Version 10.0.19041.450]
(c) 2020 Microsoft Corporation. All rights reserved.
C:\Users\prash>ncat -lk 9999
```

TCP Socket port 9999

<https://nmap.org/download.html>



SHOW WORD COUNT PROJECT



Spark Structured Streaming



- **Streaming Query**

Un thread in background (coordinatore) che gestisce l'esecuzione di un job di streaming.

- **Micro-batch**

La query di streaming esegue micro-batch in un ciclo continuo, e ogni batch è inizializzato con un dataset batch.

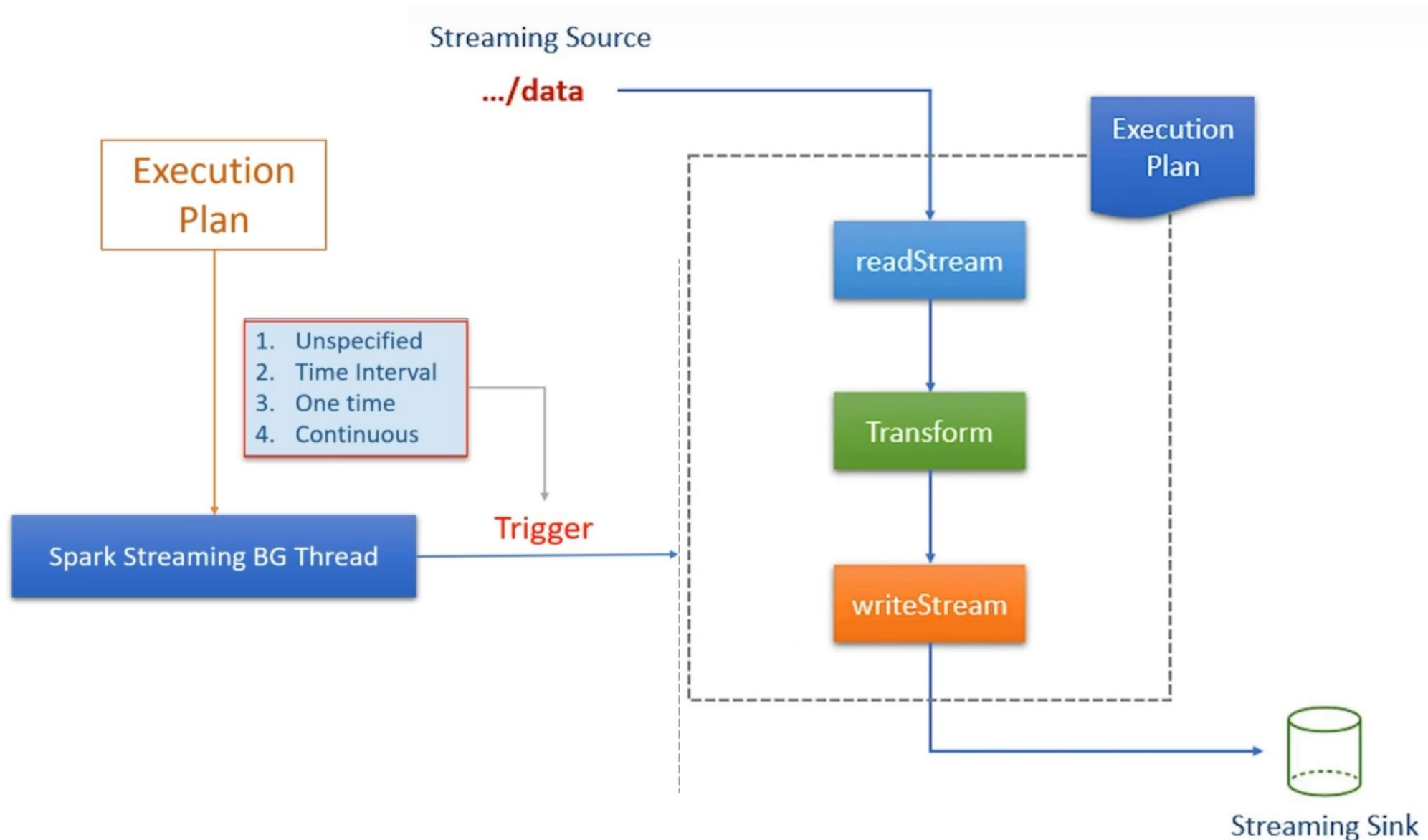
- **Checkpoint**

Il checkpoint contiene le informazioni sui dati dei micro-batch.

- **Implicitamente Incrementale**

La query di streaming garantisce la lettura solo dei nuovi dati in ciascun micro-batch.

Spark Structured Streaming Triggers



Spark Structured Streaming Triggers



01

Unspecified

micro-batches will be generated as soon as the previous micro-batch has completed processing

02

Fixed interval

micro-batches will be kicked off at user-specified intervals

03

Available-now

one-time micro-batch trigger to process all the available data and then stop on its own



```
df.writeStream \  
  .format("delta") \  
  .option("checkpointLocation", "/chkpt/dir") \  
  .toTable("table name")
```

ASAP - Seconds

```
df.writeStream \  
  .format("delta") \  
  .option("checkpointLocation", "/chkpt/dir") \  
  .trigger(processingTime='2 seconds') \  
  .toTable("table name")
```

Collect and Process - Minutes

```
df.writeStream \  
  .format("delta") \  
  .option("checkpointLocation", "/chkpt/dir") \  
  .trigger(availableNow=True) \  
  .toTable("table name")
```

Incremental Batch - Hours

File Source Example



- SHOW PROJECT

Spark Structured Streaming Fault Tolerance



Spark Structured Streaming Fault Tolerance



1. Failure
2. Maintenance

Background
Streaming Thread

Micro-batch

1. Read Stream
2. Process Stream
3. Write Stream

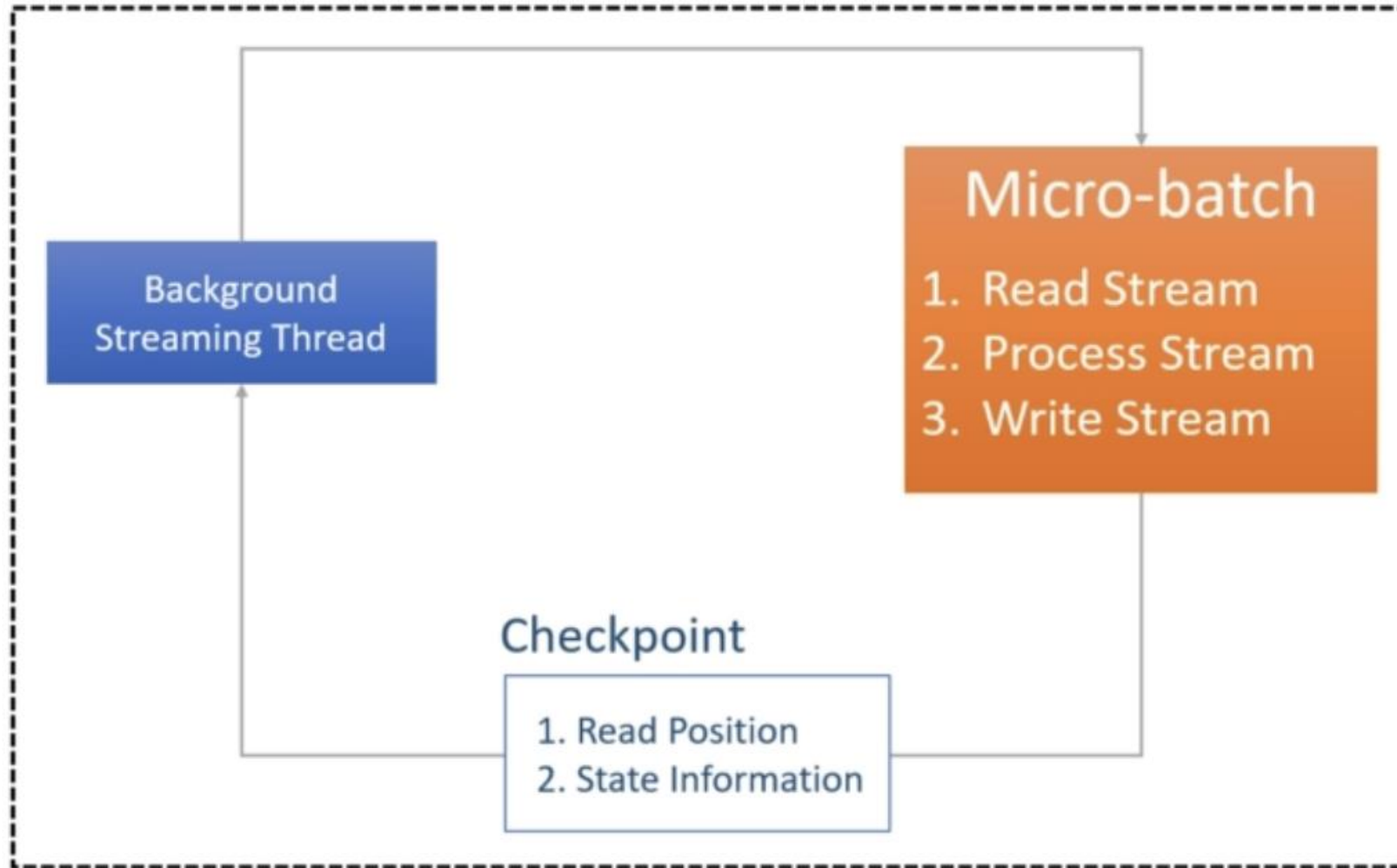
Spark Structured Streaming Fault Tolerance



Restart with Exactly Once Feature

1. Do not miss any input records
2. Do not create duplicate output records

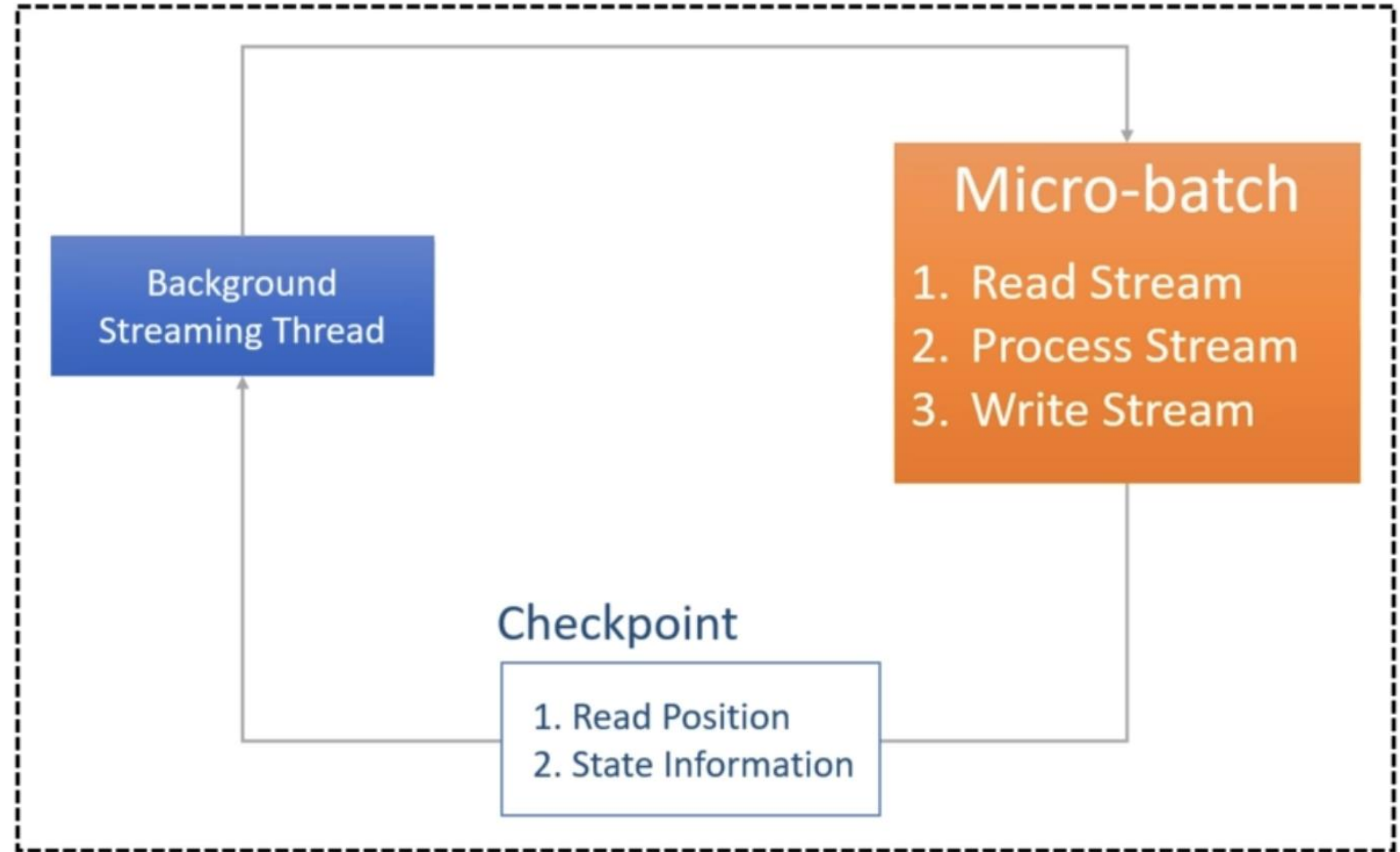
Spark Structured Streaming Fault Tolerance



Spark Structured Streaming Fault Tolerance



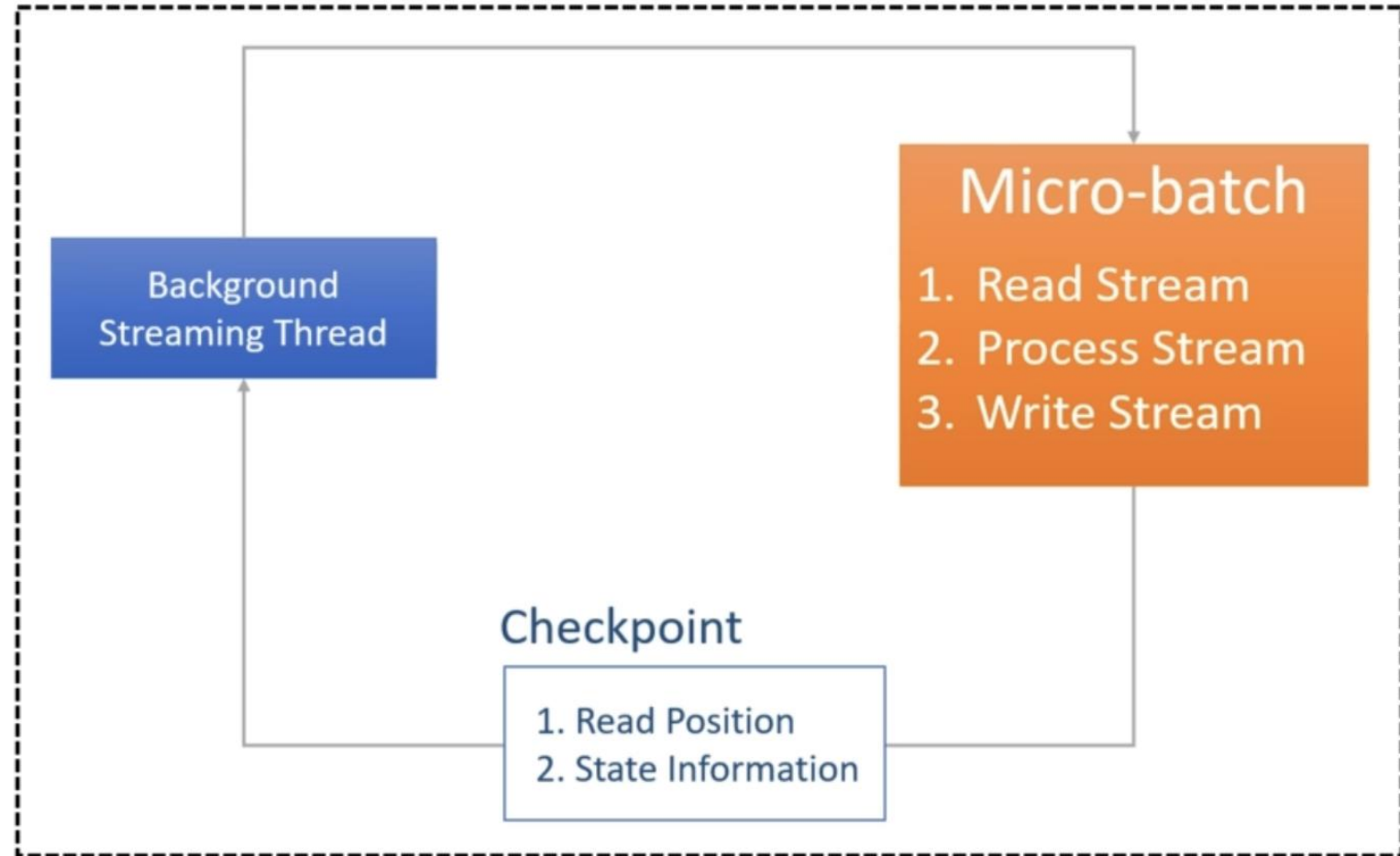
1. Restart with same checkpoint
2. Use a replayable source
3. Use Deterministic computation
4. Use an idempotent sink



Spark Structured Streaming Fault Tolerance



1. Restart with same checkpoint
2. Use a replayable source
3. Use Deterministic computation
4. Use an idempotent sink



What about bug fixes?

Kafka Source Example



- SHOW PROJECT

Kafka Source and Sink Example



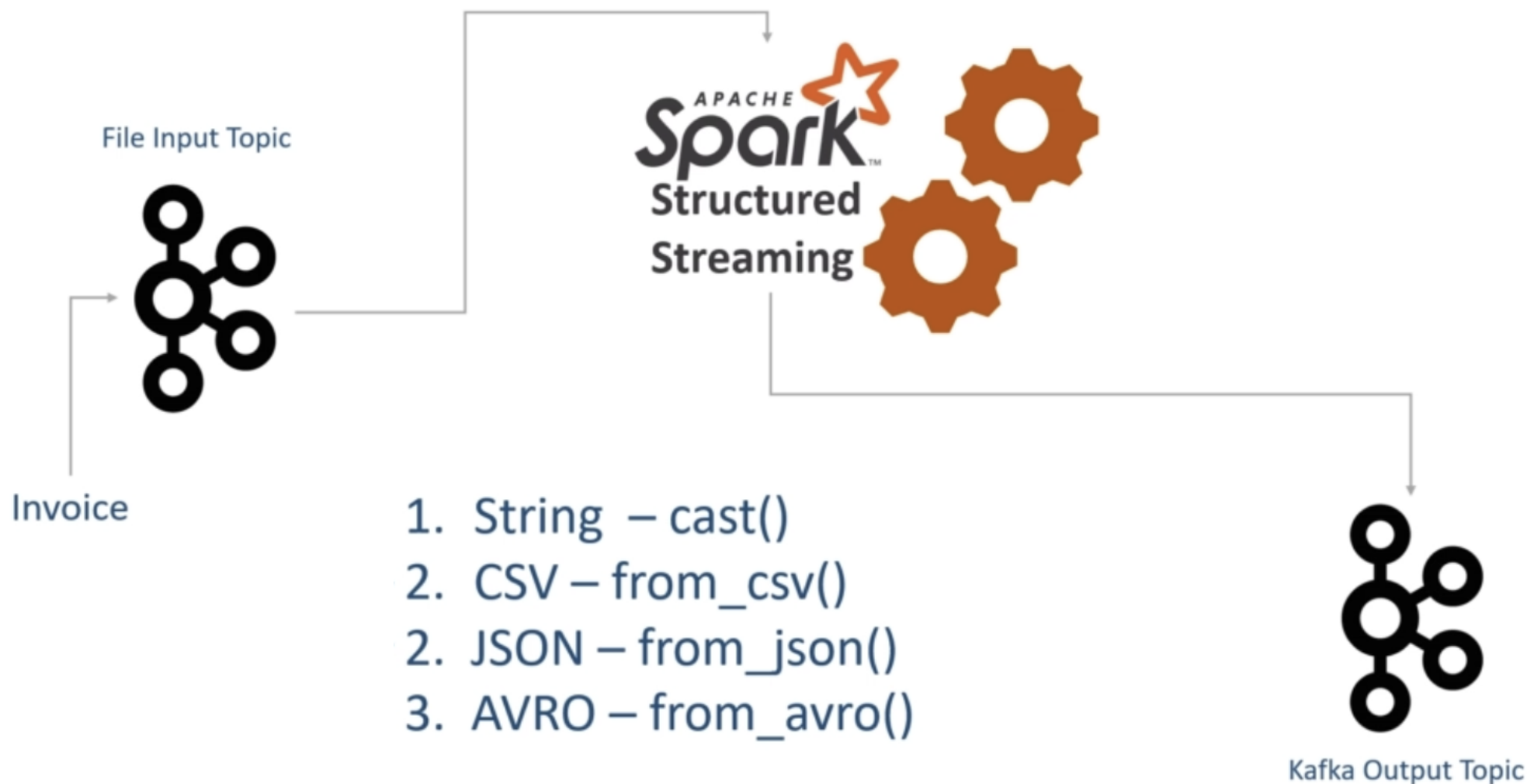
Kafka Source and Sink Example



```
{"CustomerCardNo": "4629185211", "TotalAmount": 11114.0, "EarnedLoyaltyPoints": 2222.8}
```



Kafka Source and Sink Example



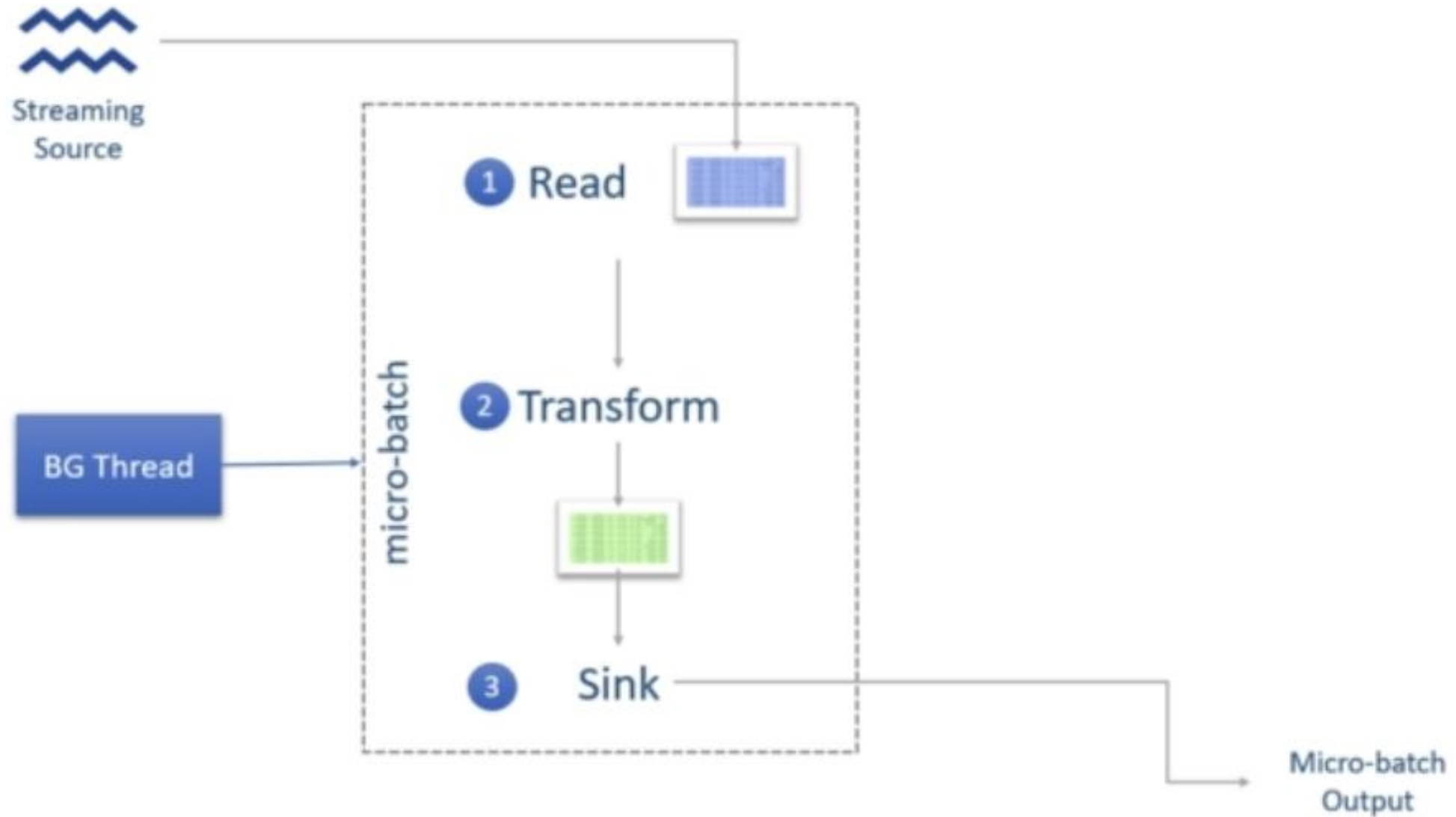
Kafka Source and Sink + File Sink Example



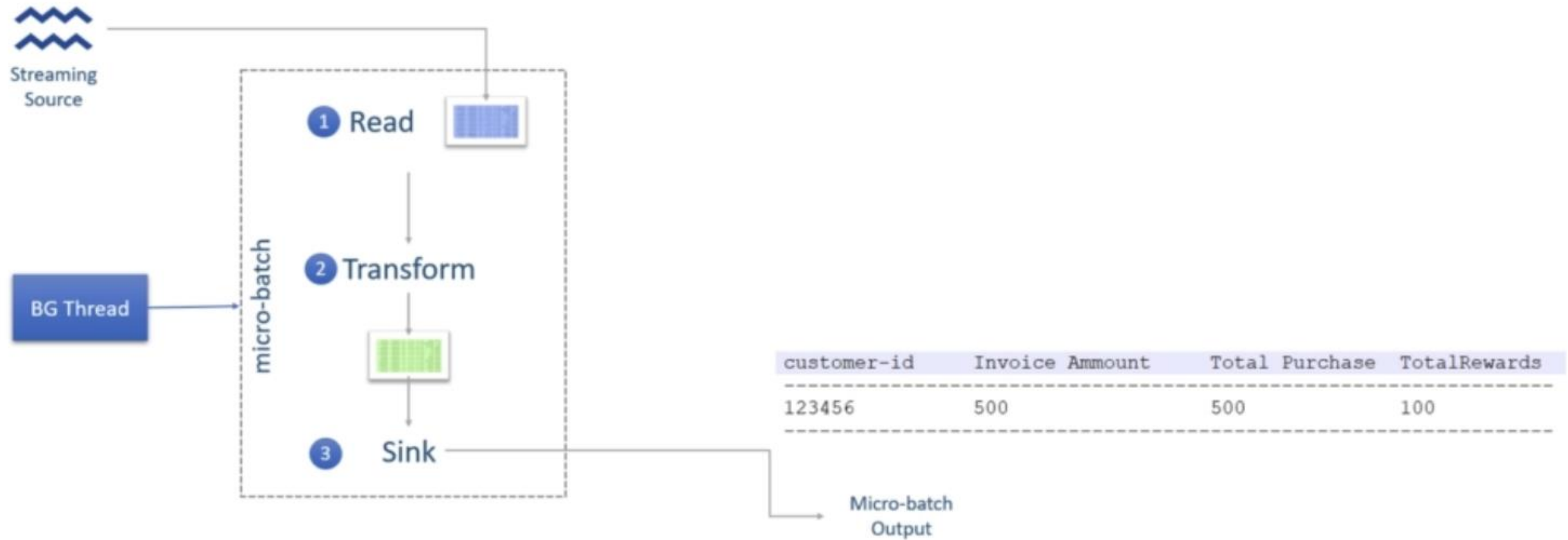


1. Stateless transformations
2. Stateful transformations

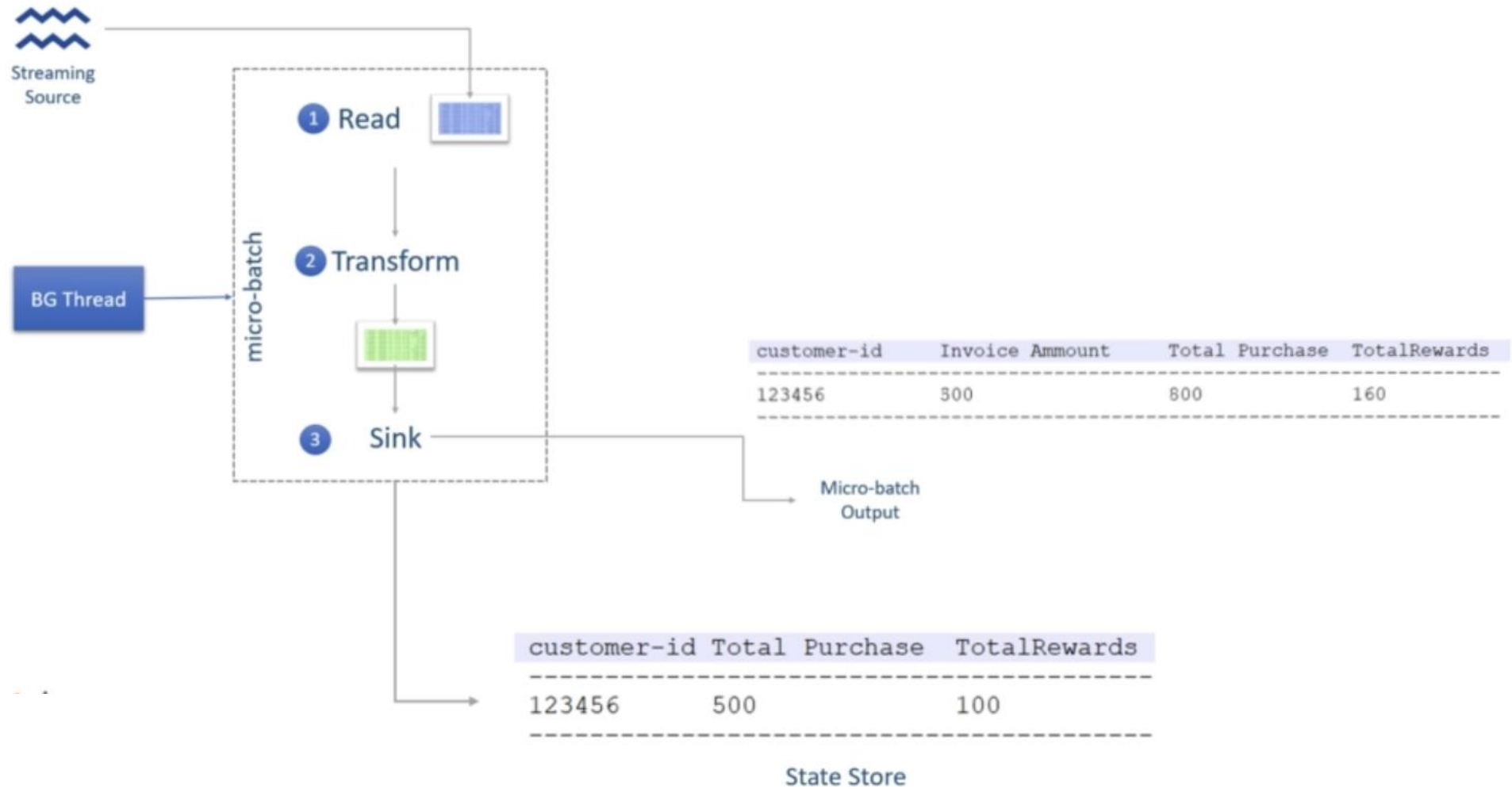
State Store



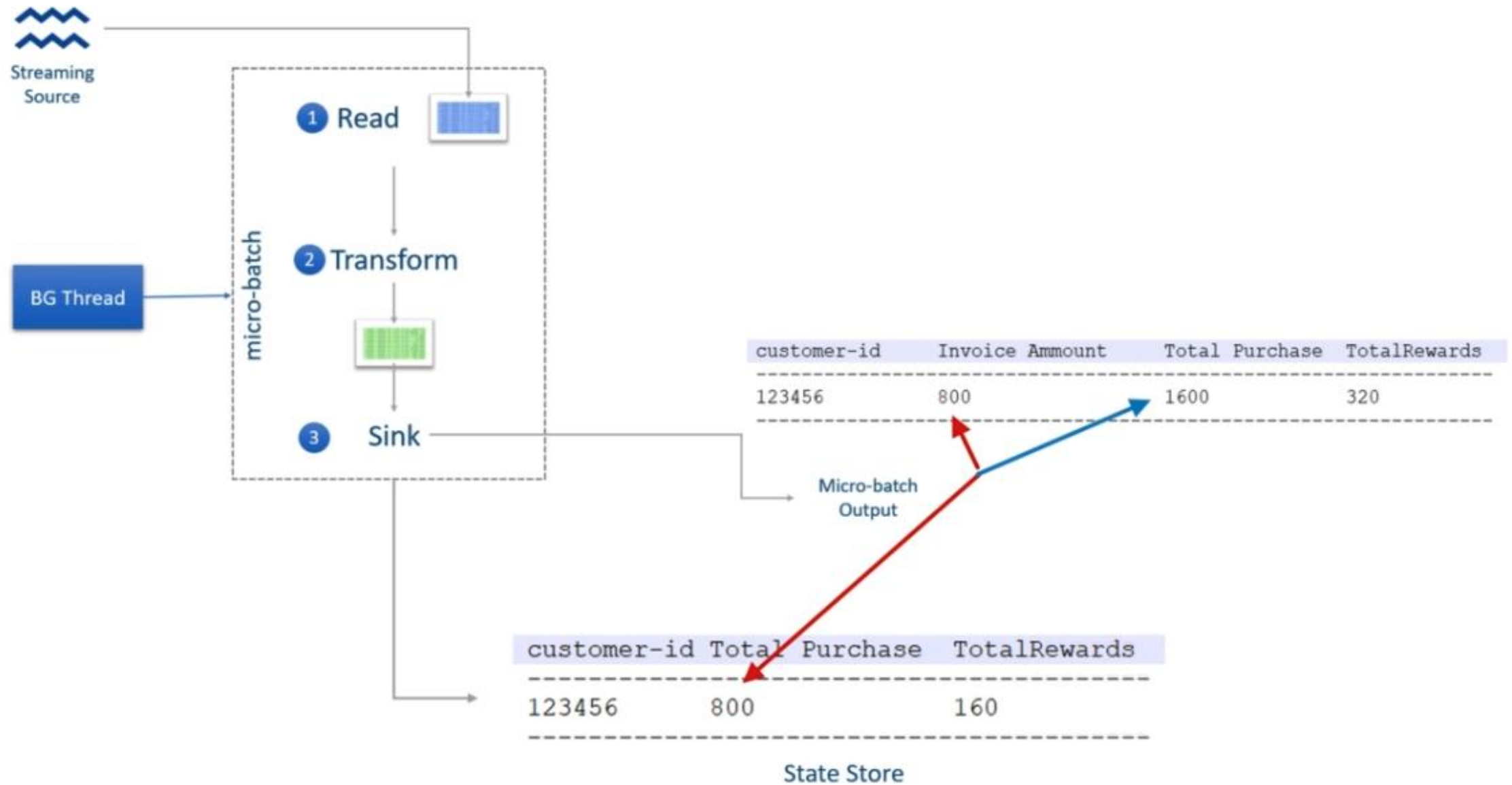
State Store



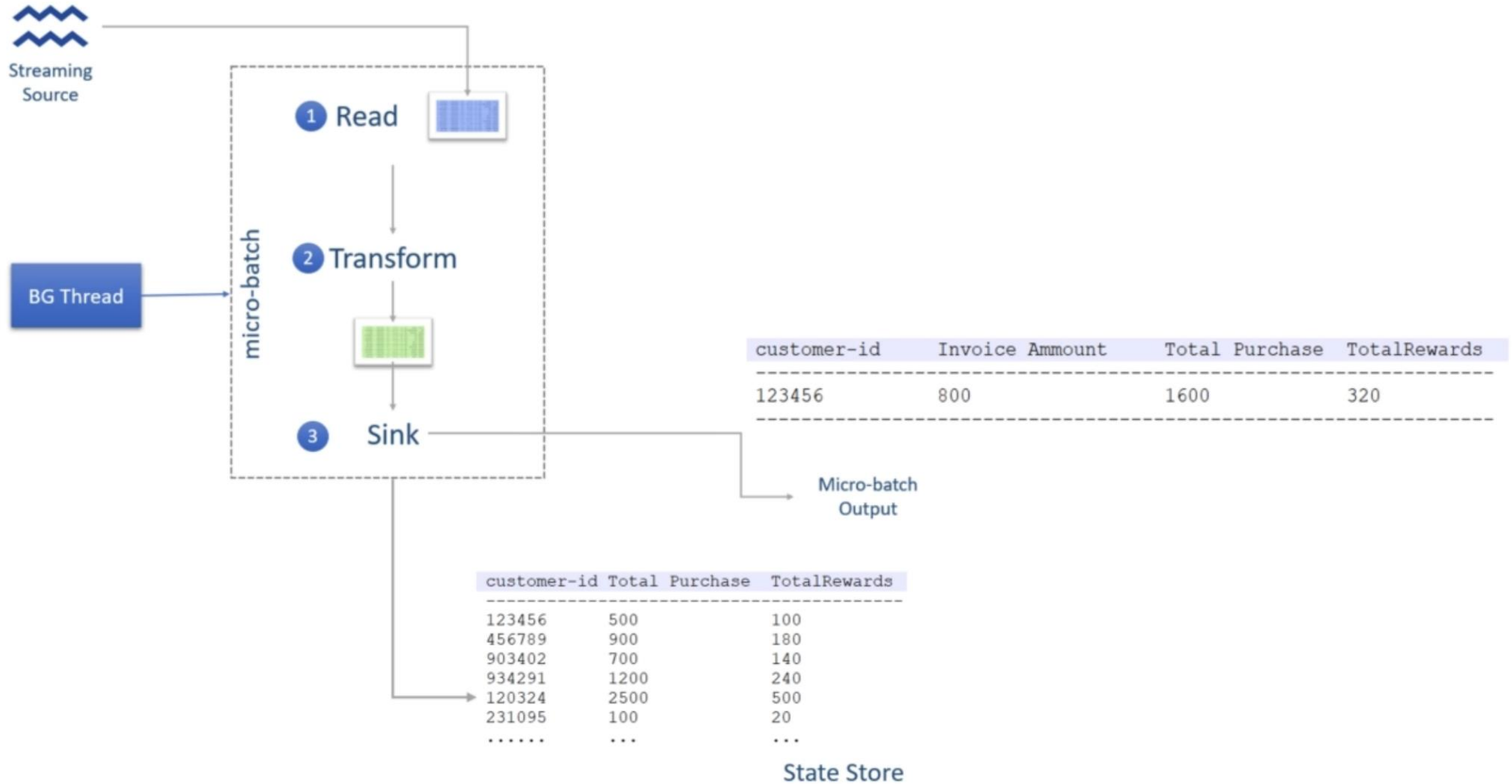
State Store



State Store



State Store





1. Stateless transformation

`select()`, `filter()`, `map()`,
`flatMap()`, `explode()`

2. Stateful transformations

Grouping, Aggregations
Windowing, and Joins



1. Stateless transformation
Complete output mode not supported
2. Stateful transformations
Excessive state causes out of memory



Stateful transformations

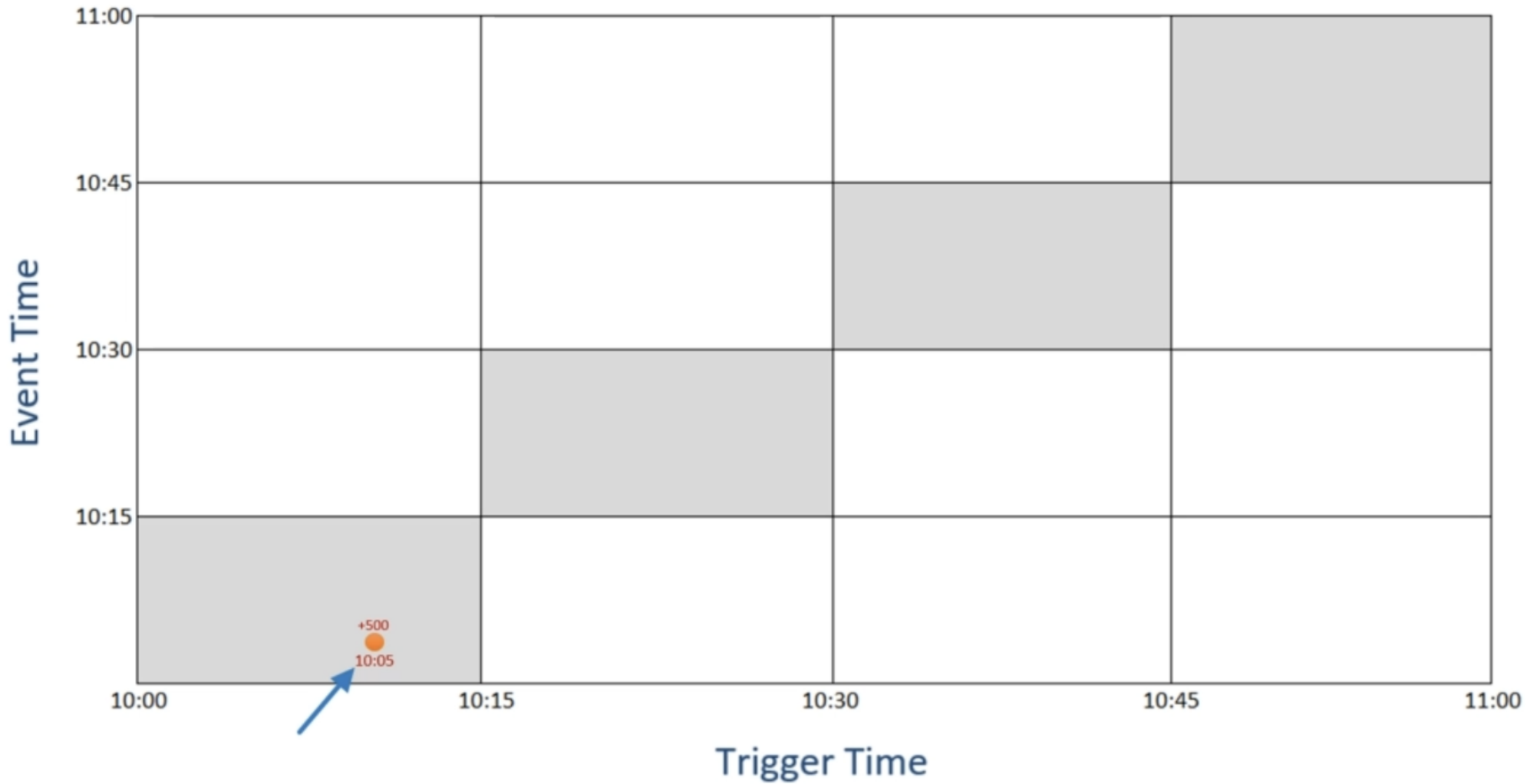
- • Managed Stateful operations
- Unmanaged Stateful operations



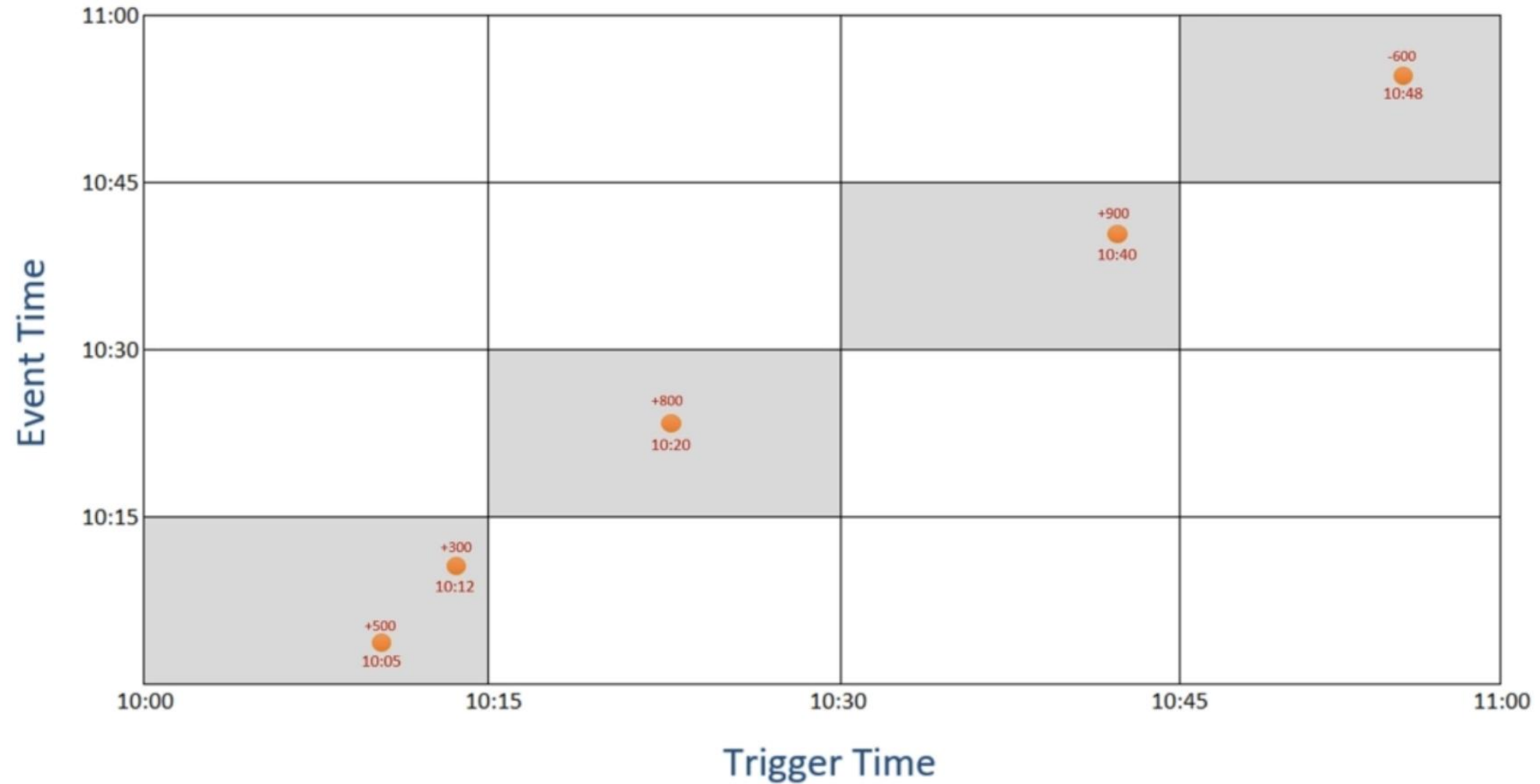
Aggregations

- Continuous aggregation
- Time-bound aggregations

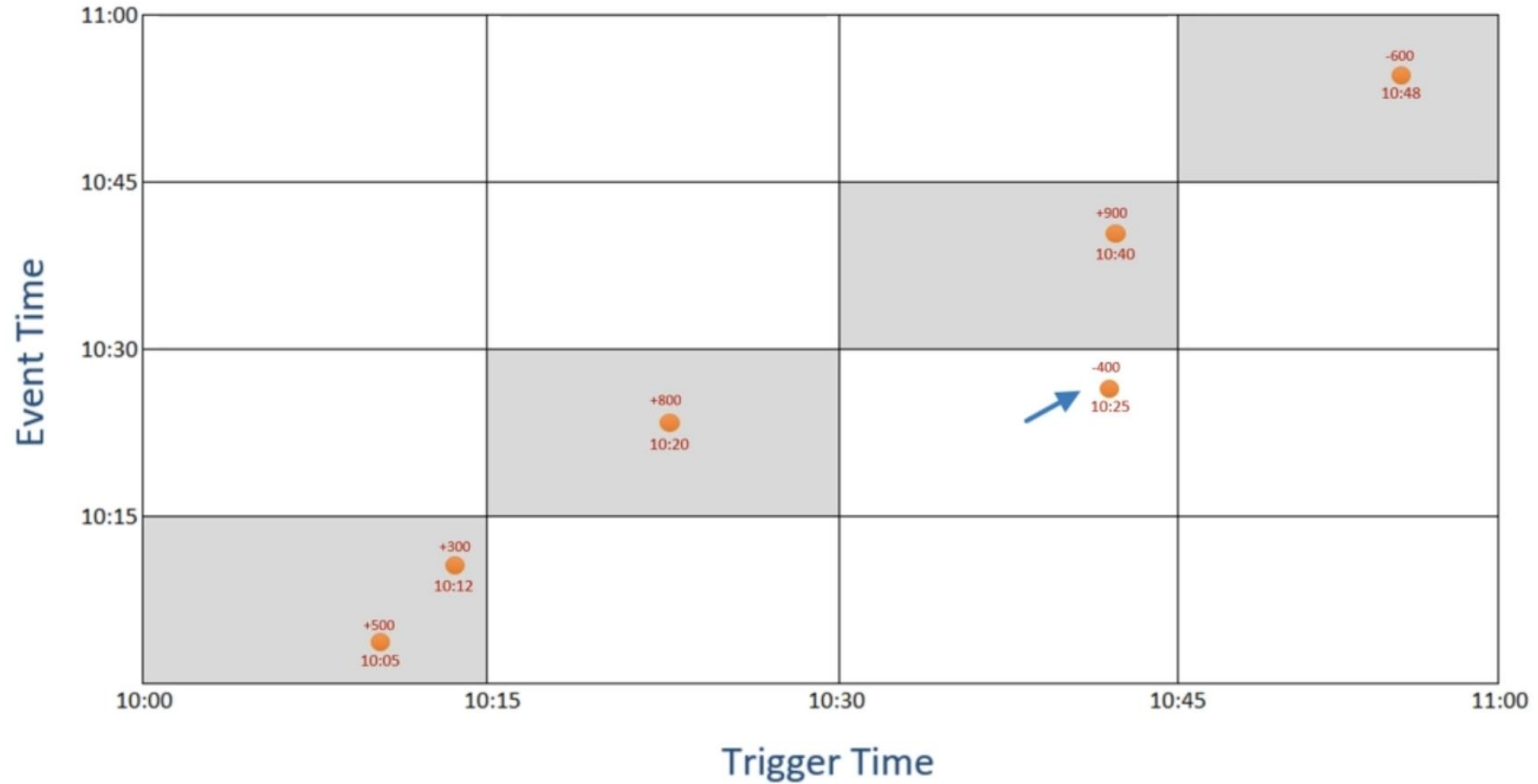
Time-bounded Aggregations



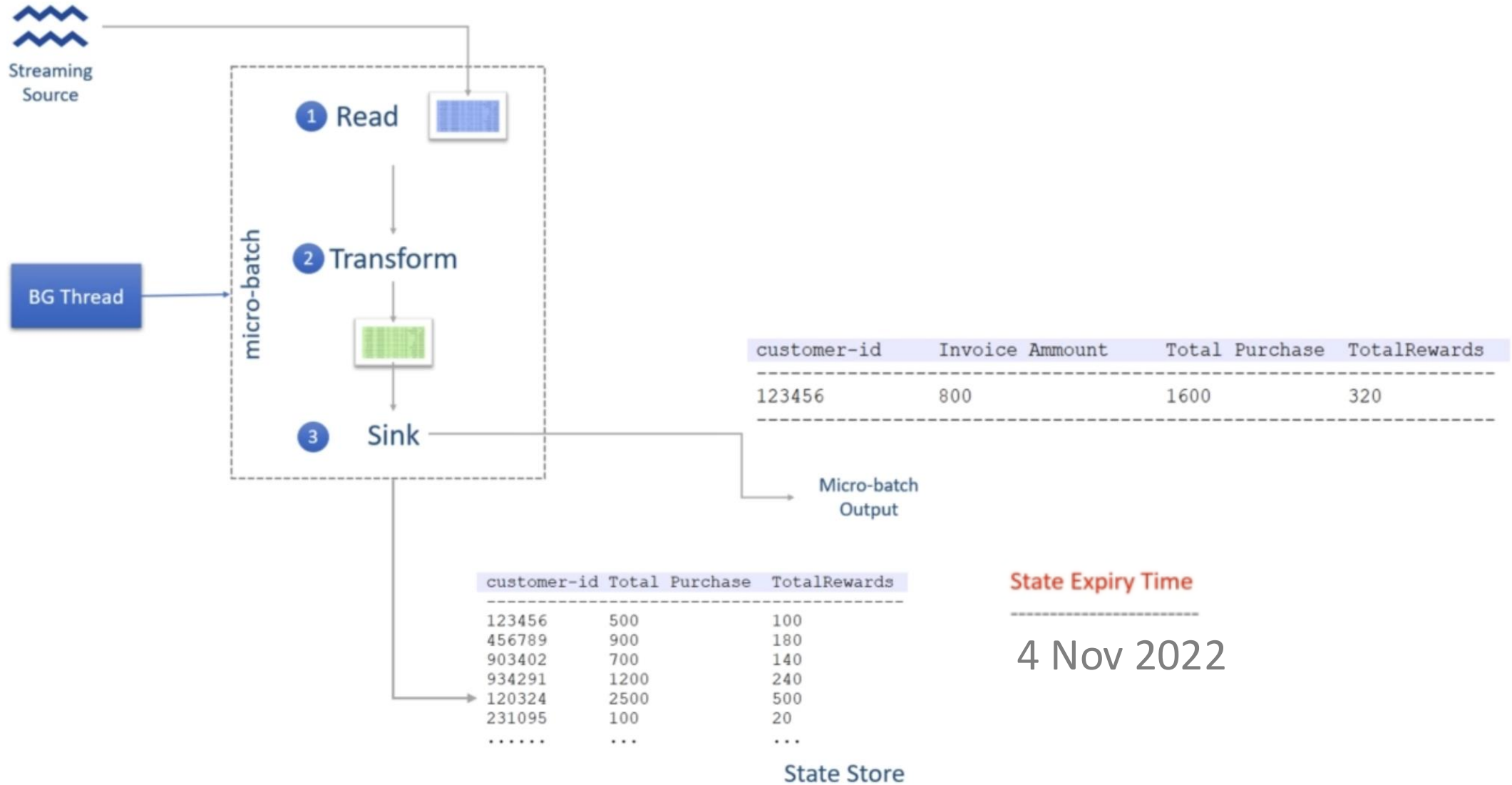
Time-bounded Aggregations



Time-bounded Aggregations



State Store





1. Stateless transformation

Complete output mode not supported

2. Stateful transformations

Excessive state causes out of memory

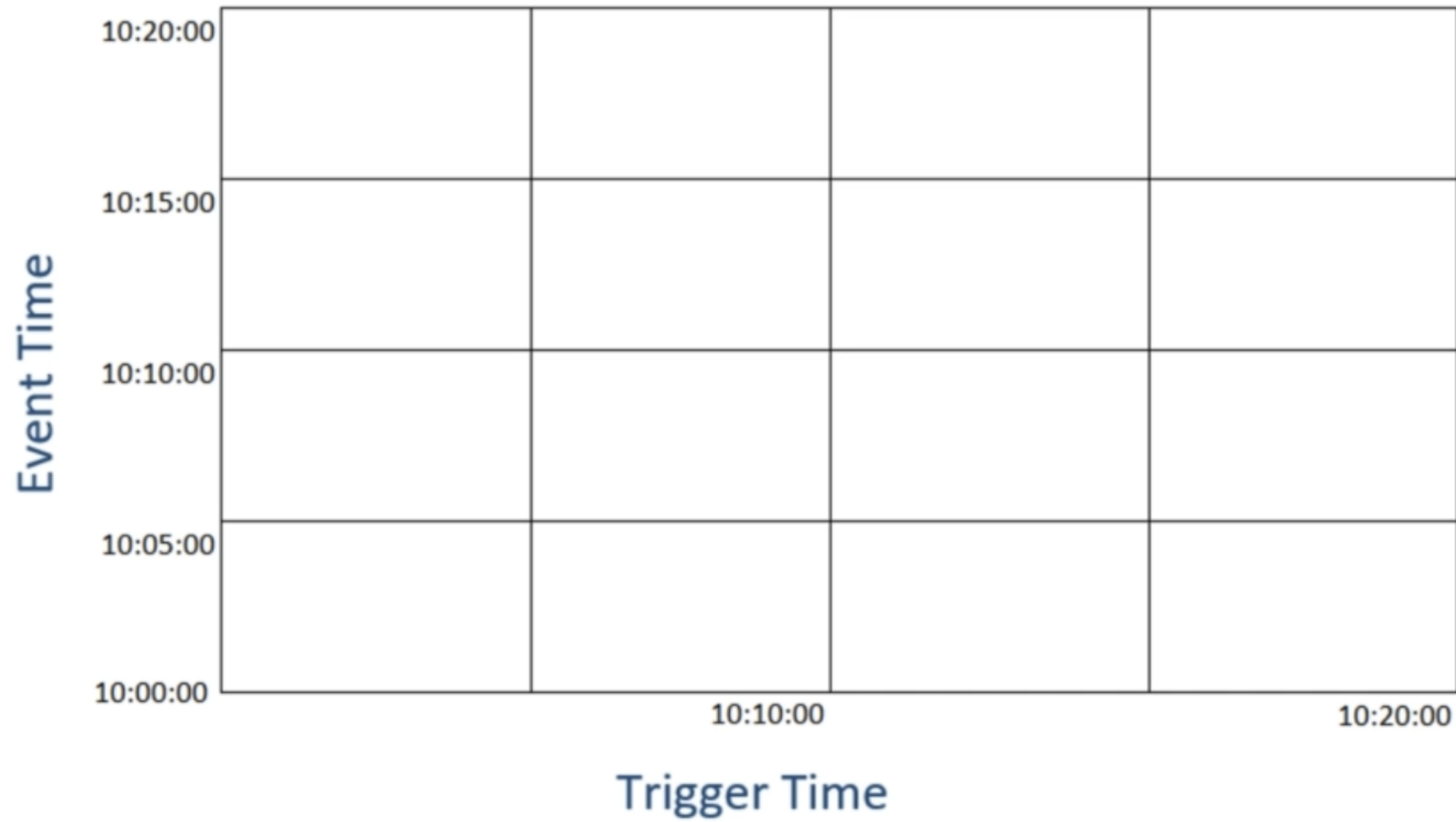
1. Unbounded State
2. Time Bound State



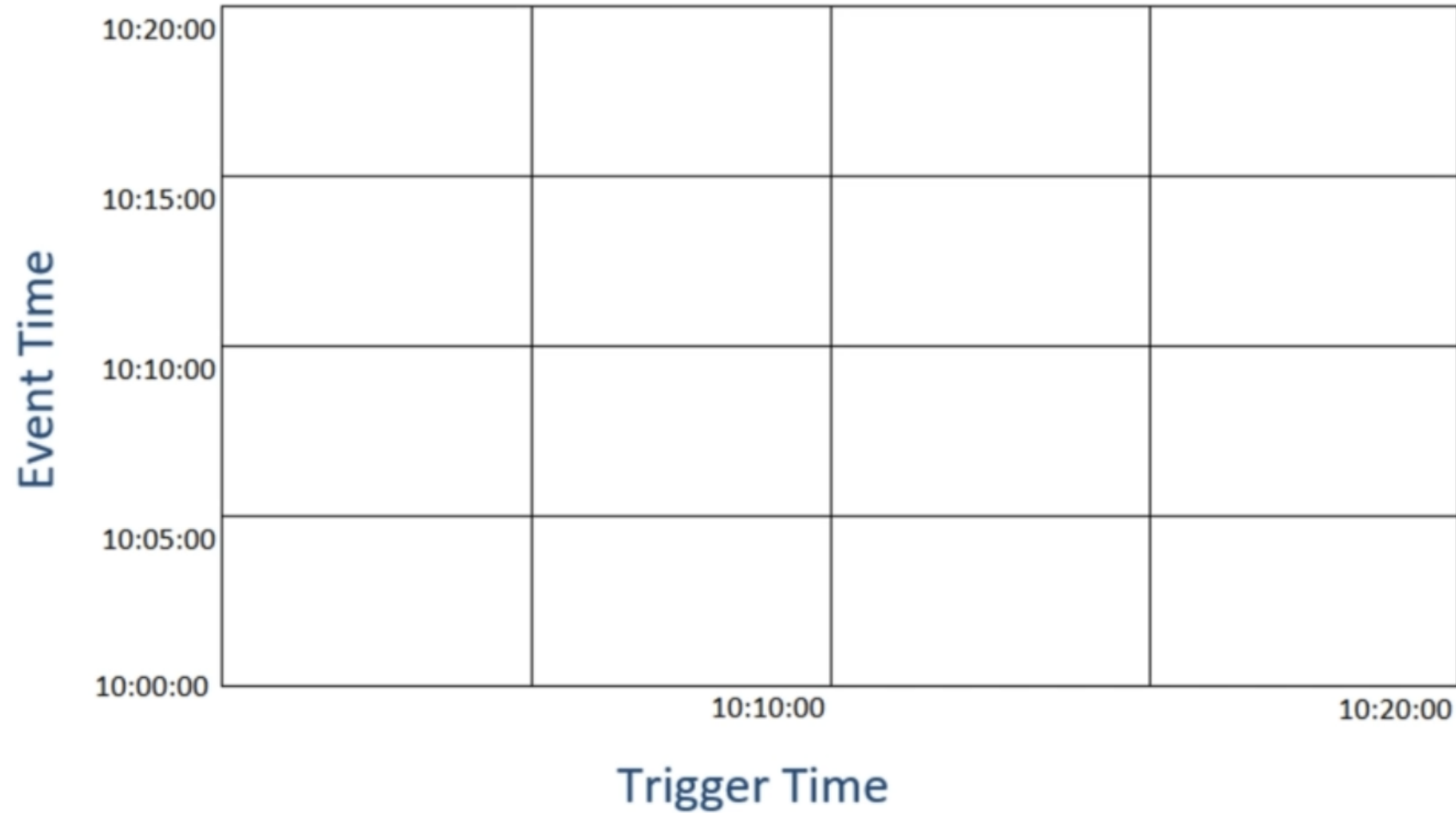
Windowing Aggregates

1. Tumbling Time Window
2. Sliding Time Window

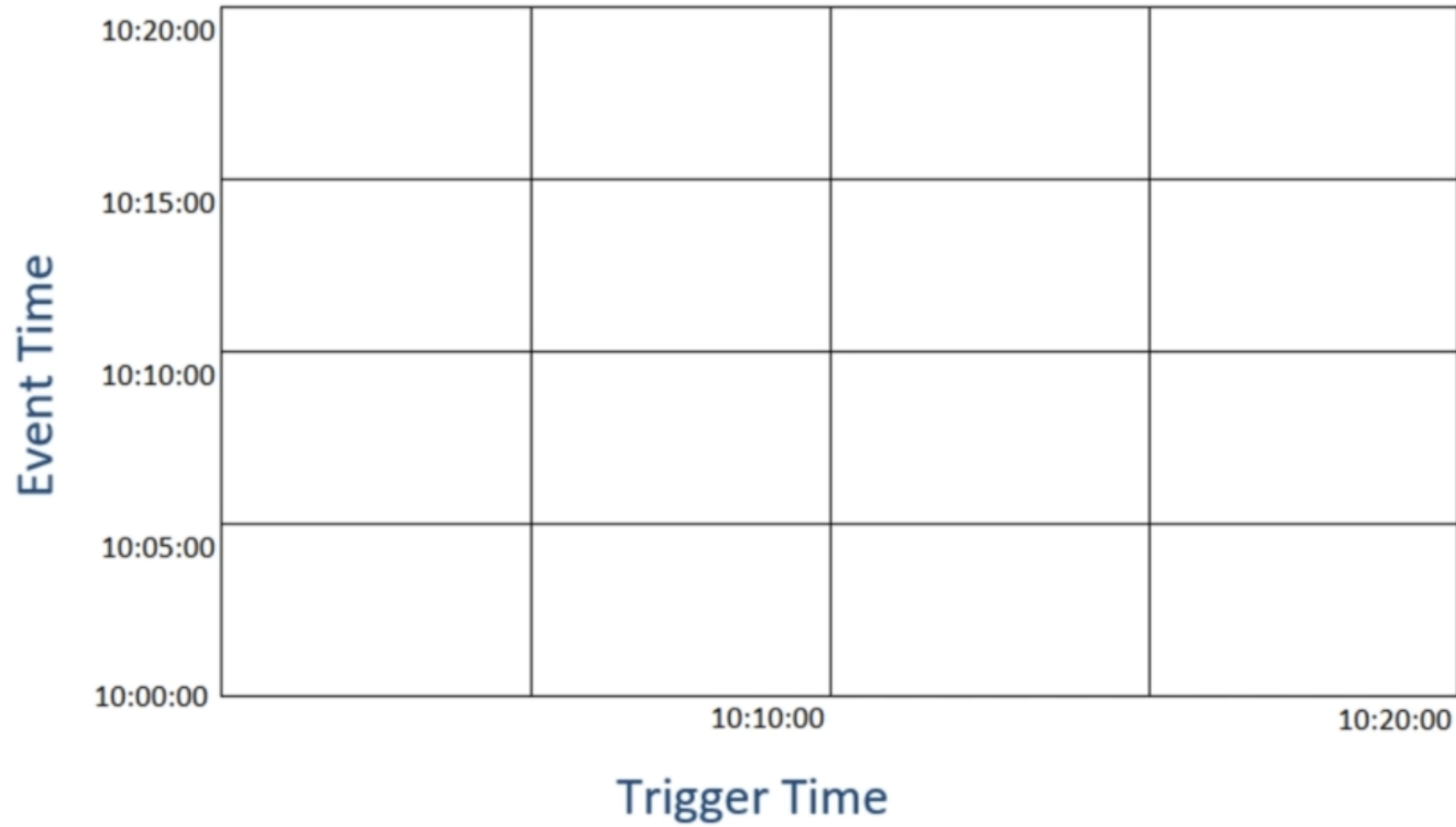
Manage State Store



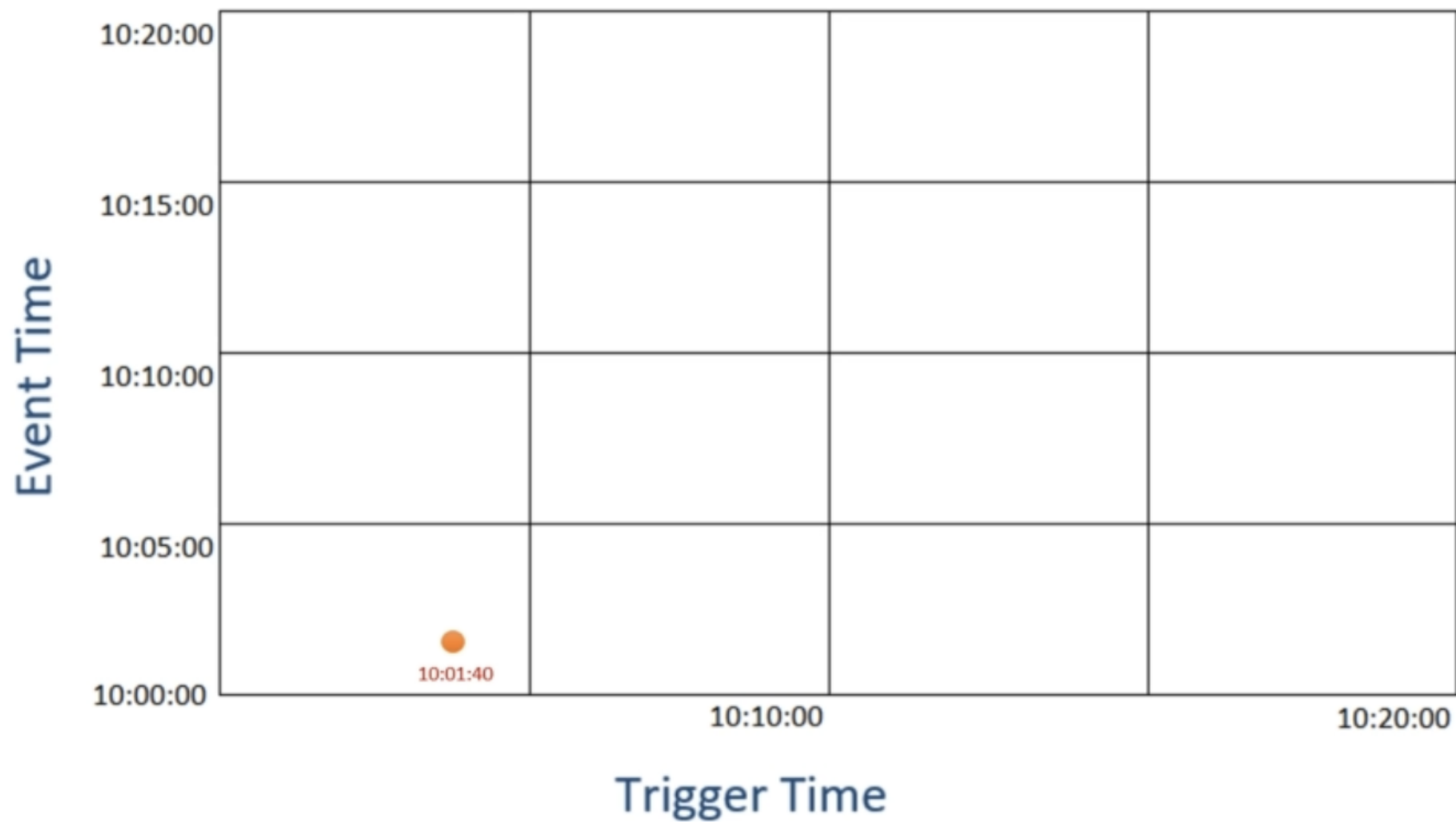
Event Time vs Trigger Time



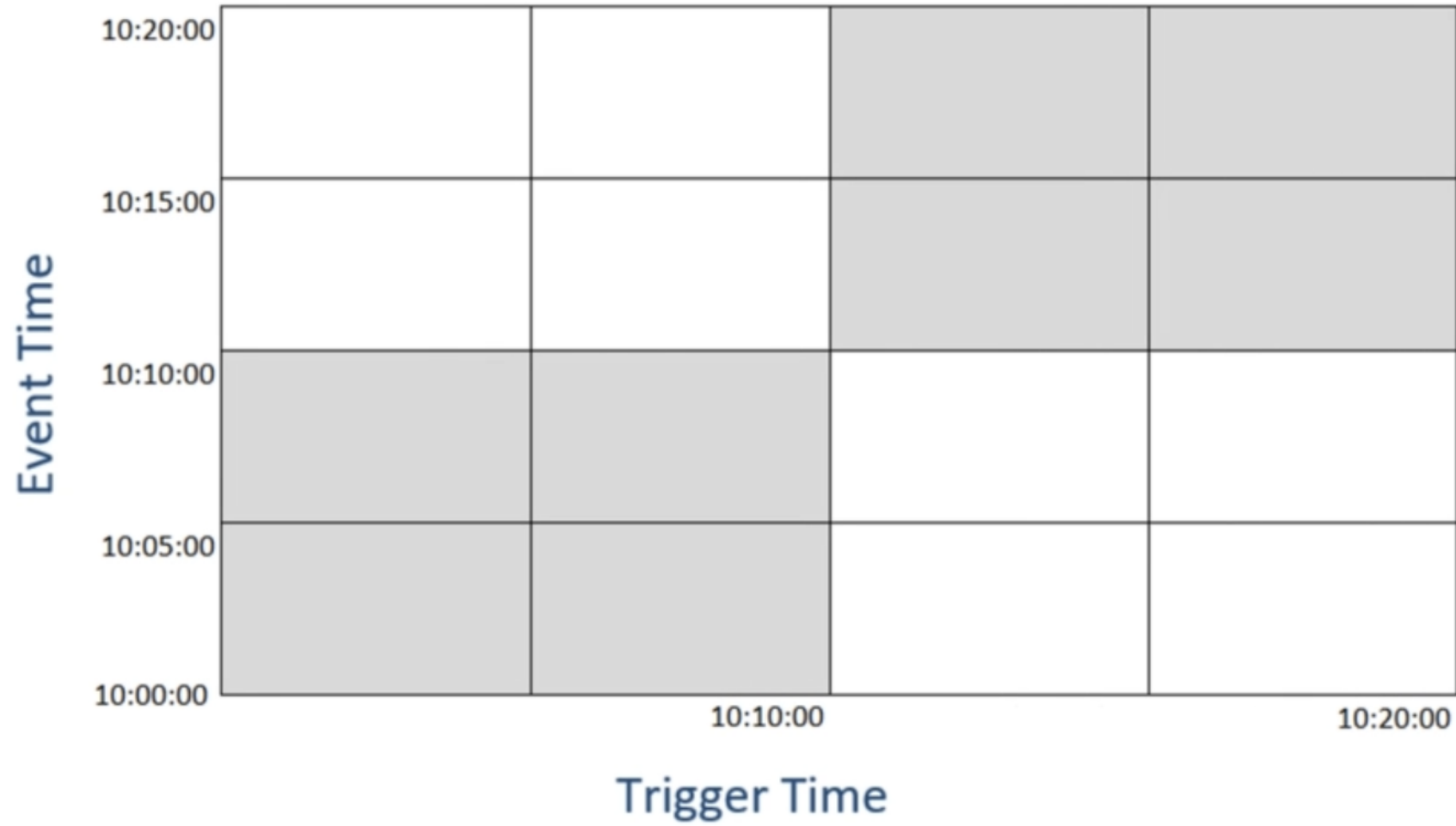
Event Time vs Trigger Time



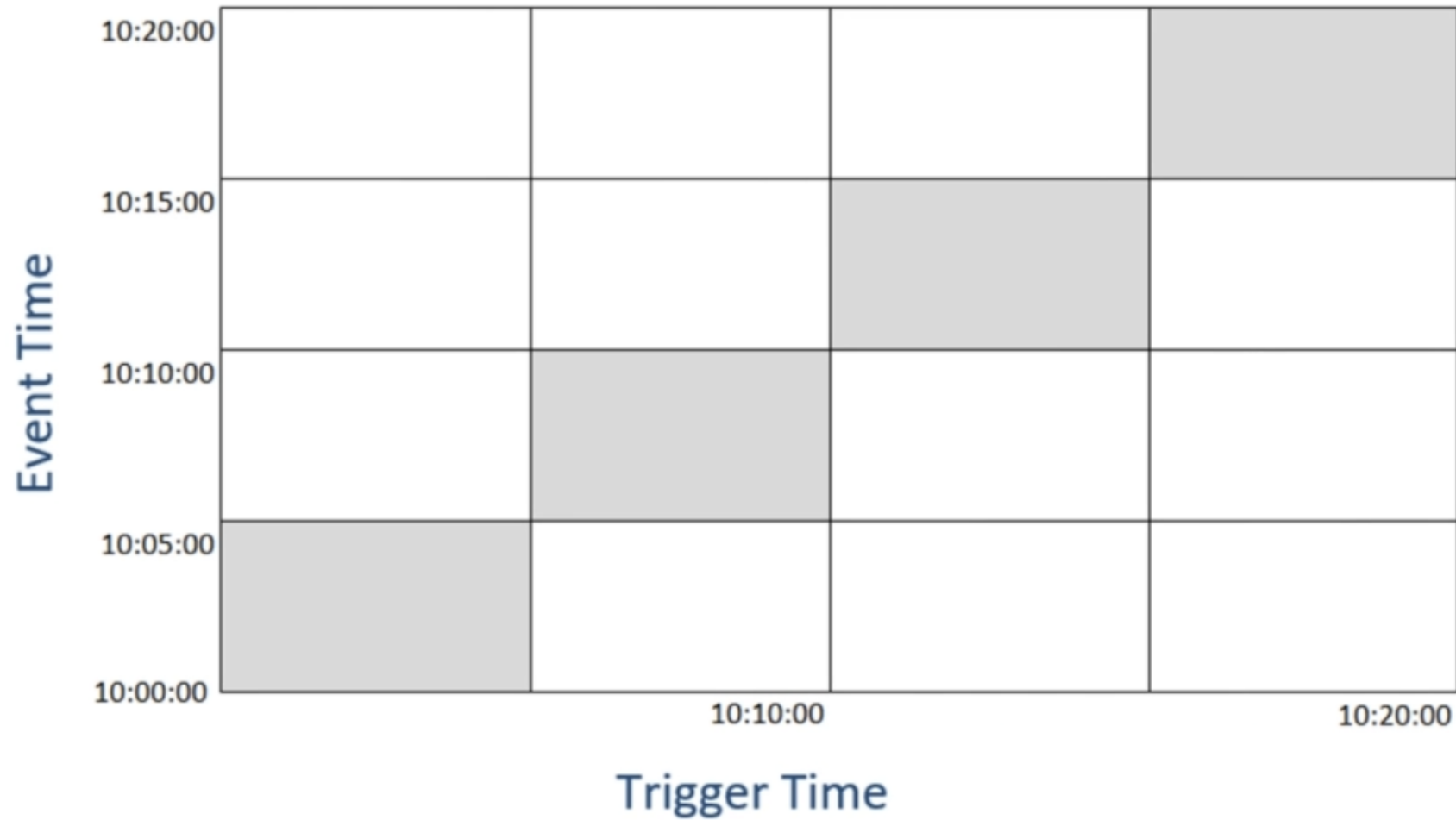
Event Time vs Trigger Time



Event Time vs Trigger Time



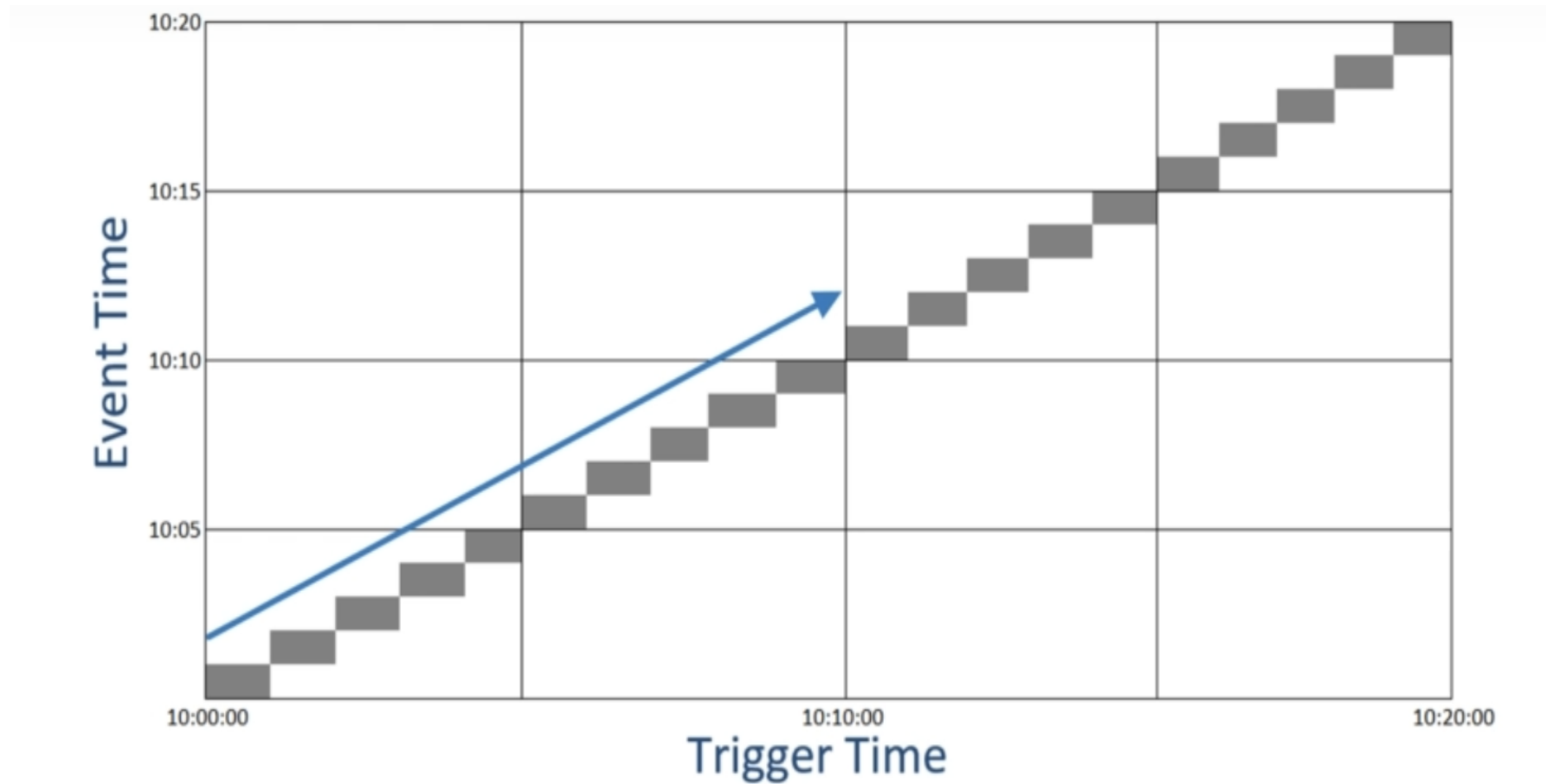
Event Time vs Trigger Time



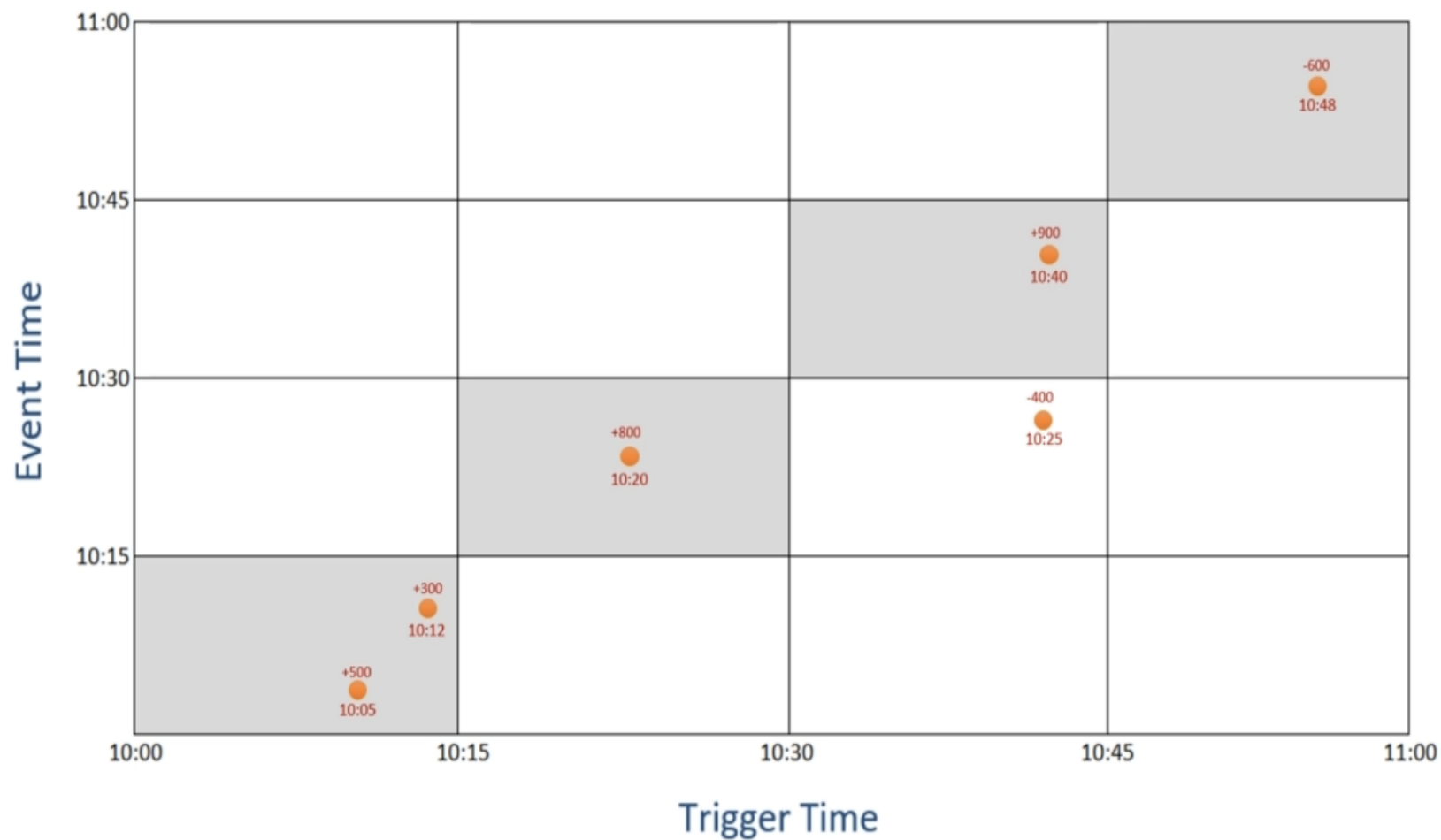
Event Time vs Trigger Time



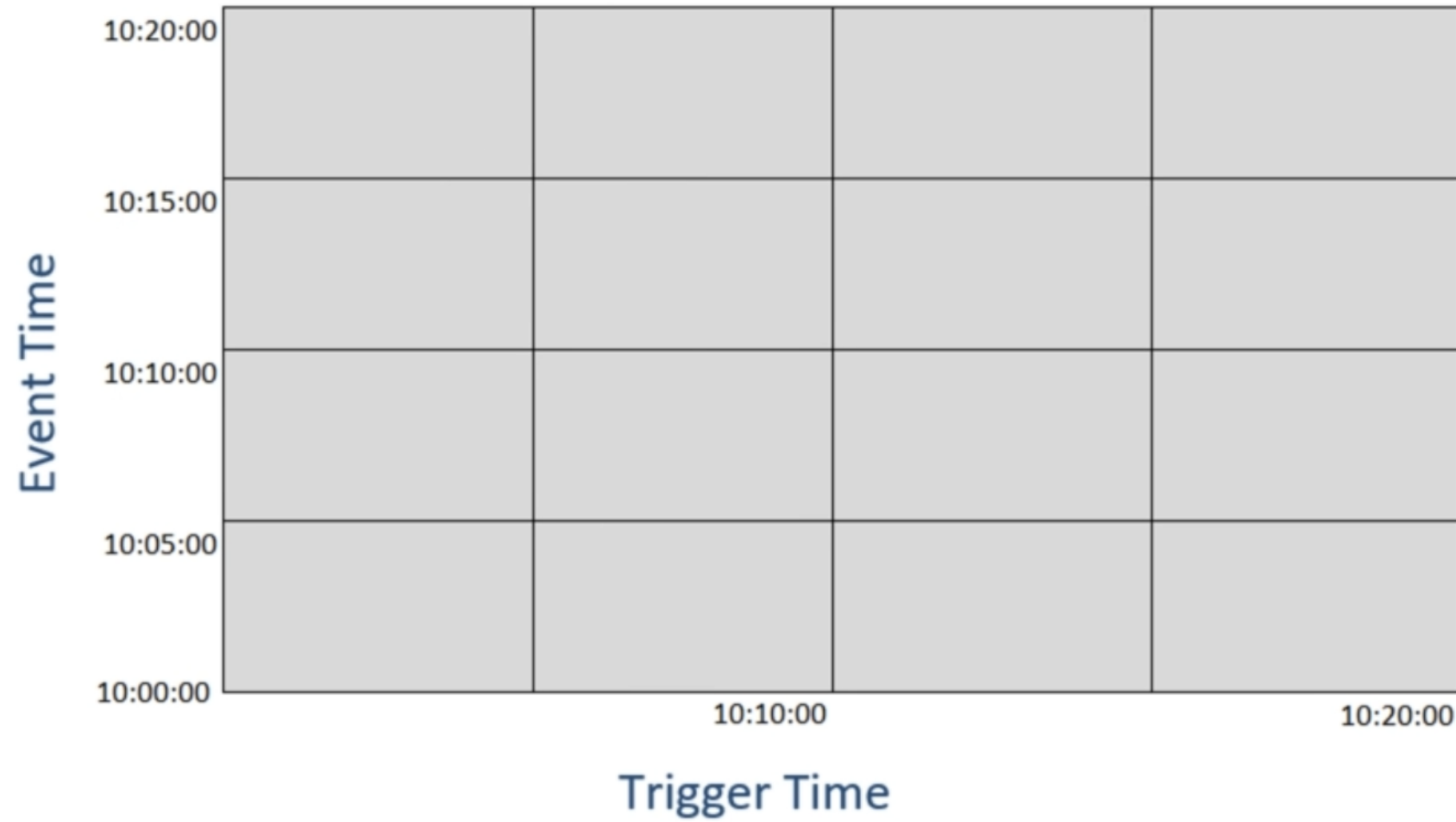
Event Time vs Trigger Time



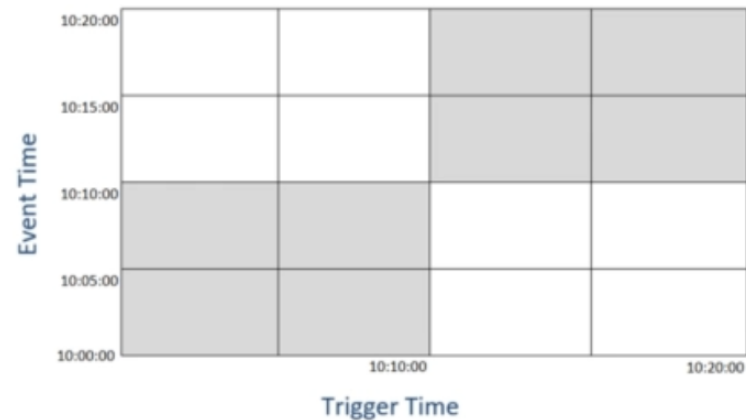
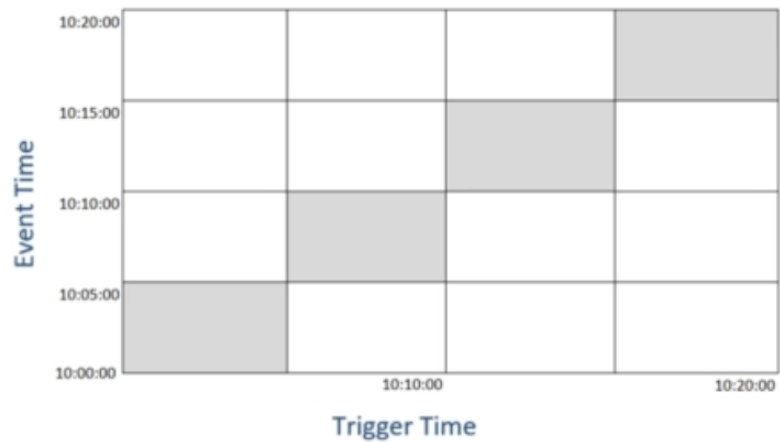
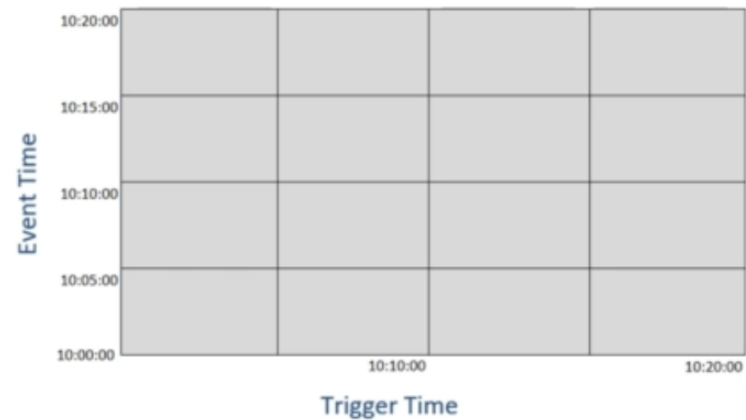
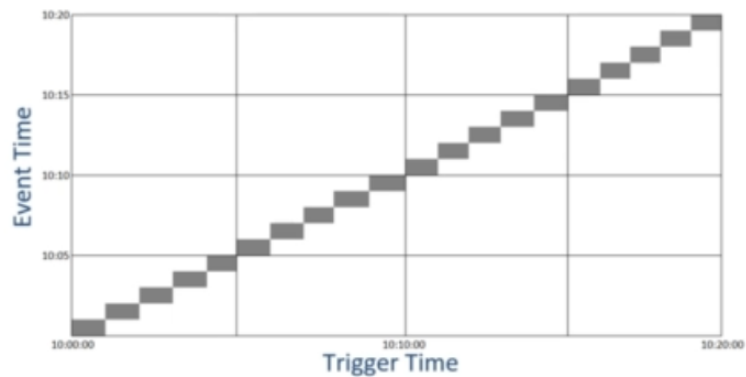
Event Time vs Trigger Time



Event Time vs Trigger Time



Event Time vs Trigger Time



Tumbling Time Windows

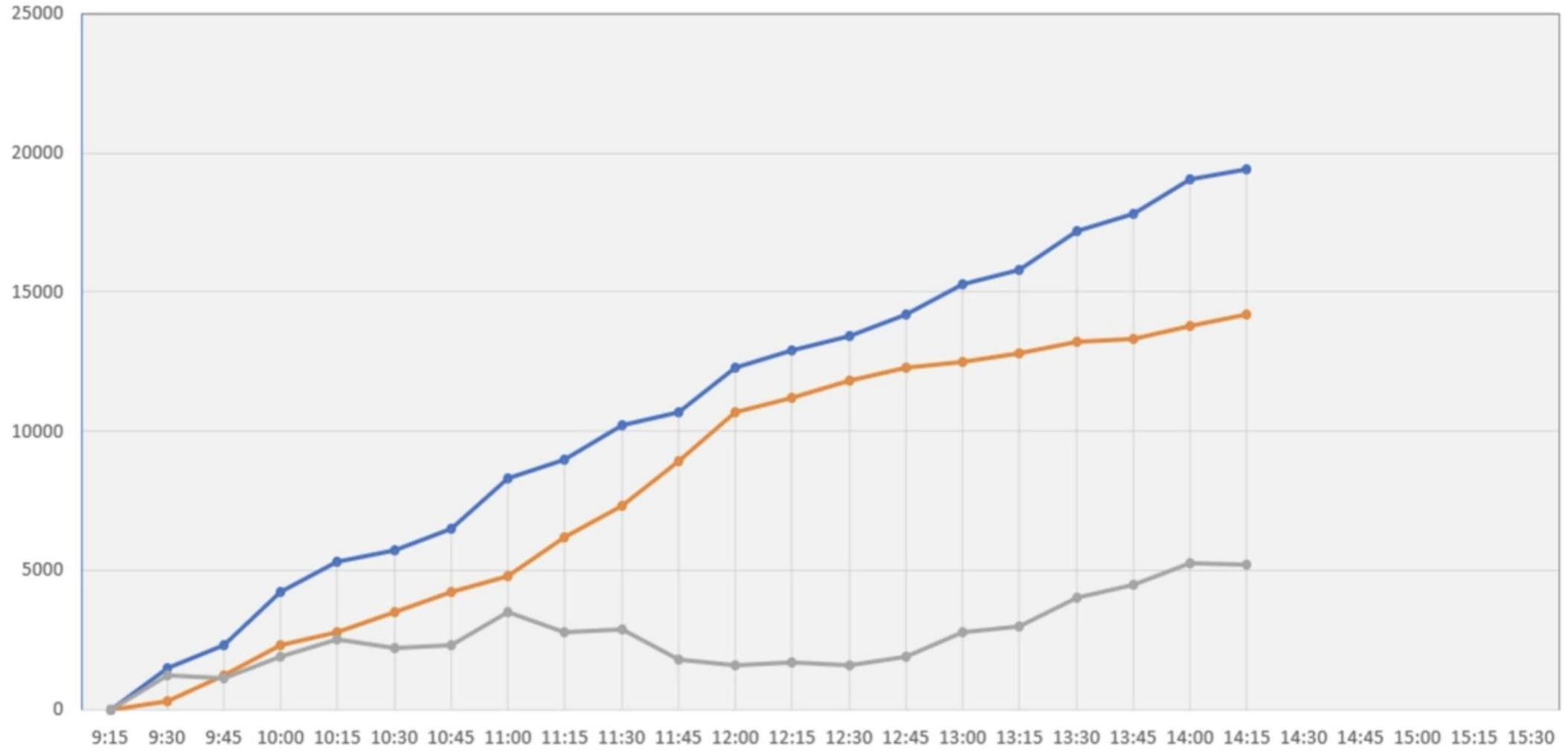


```
{"TransactionTime": "2019-02-05 10:01:12",  
"TransactionType": "BUY",  
"TransactionAmount": 500,  
"BrokerCode": "ABX"}
```

```
{"TransactionTime": "2019-02-05 10:05:30",  
"TransactionType": "SELL",  
"TransactionAmount": 300, ,  
"BrokerCode": "ABX"}
```

Start Time	End Time	Buy	Sell	Net Value
9:00	9:15	0	0	0
9:15	9:30	1500	300	1200
9:30	9:45	2300	1200	1100
9:45	10:00	4200	2300	1900
10:00	10:15	5300	2800	2500
10:15	10:30	5700	3500	2200
10:30	10:45	6500	4200	2300
10:45	11:00	8300	4800	3500

Tumbling Time Windows



Tumbling Time Windows



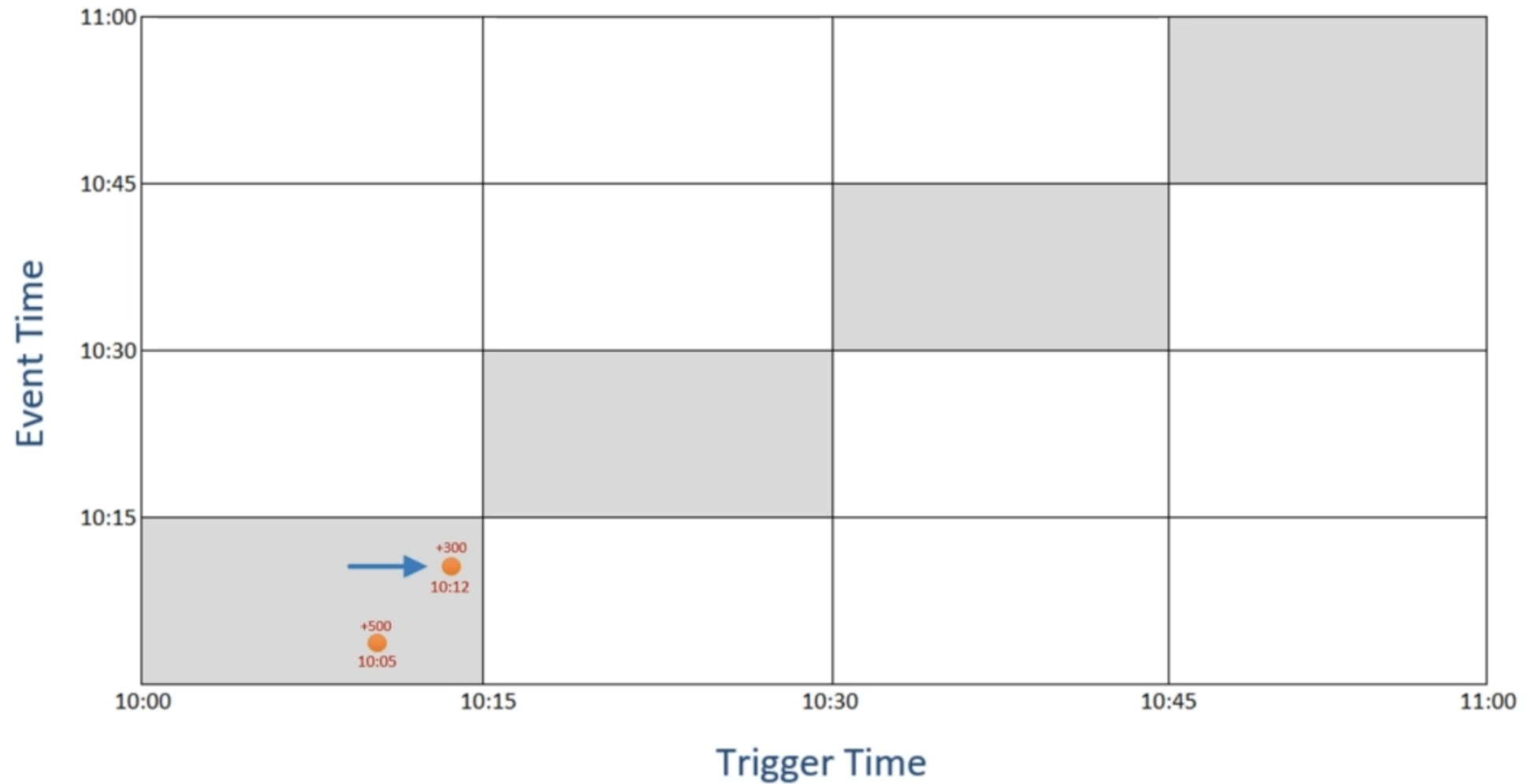
```
{ "TransactionTime": "2019-02-05 10:01:12",  
  "TransactionType": "BUY",  
  "TransactionAmount": 500,  
  "BrokerCode": "ABX" }  
  
{ "TransactionTime": "2019-02-05 10:05:30",  
  "TransactionType": "SELL",  
  "TransactionAmount": 300,  
  "BrokerCode": "ABX" }
```

Start Time	End Time	Buy	Sell	Net Value
9:00	9:15	0	0	0
9:15	9:30	1500	300	1200
9:30	9:45	2300	1200	1100
9:45	10:00	4200	2300	1900
10:00	10:15	5300	2800	2500
10:15	10:30	5700	3500	2200
10:30	10:45	6500	4200	2300
10:45	11:00	8300	4800	3500

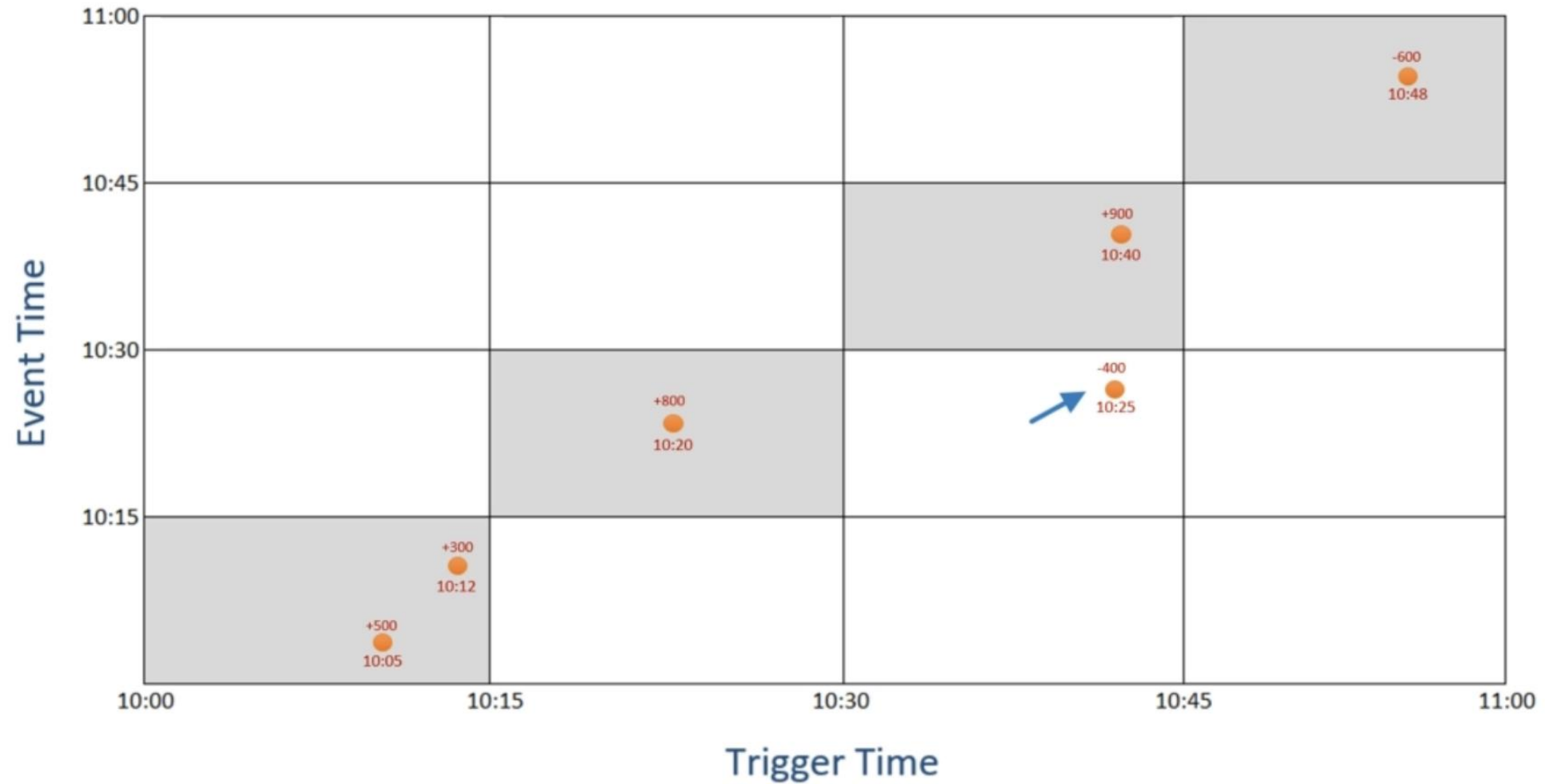
Tumbling Time Windows



Tumbling Time Windows



Tumbling Time Windows



Tumbling Time Windows



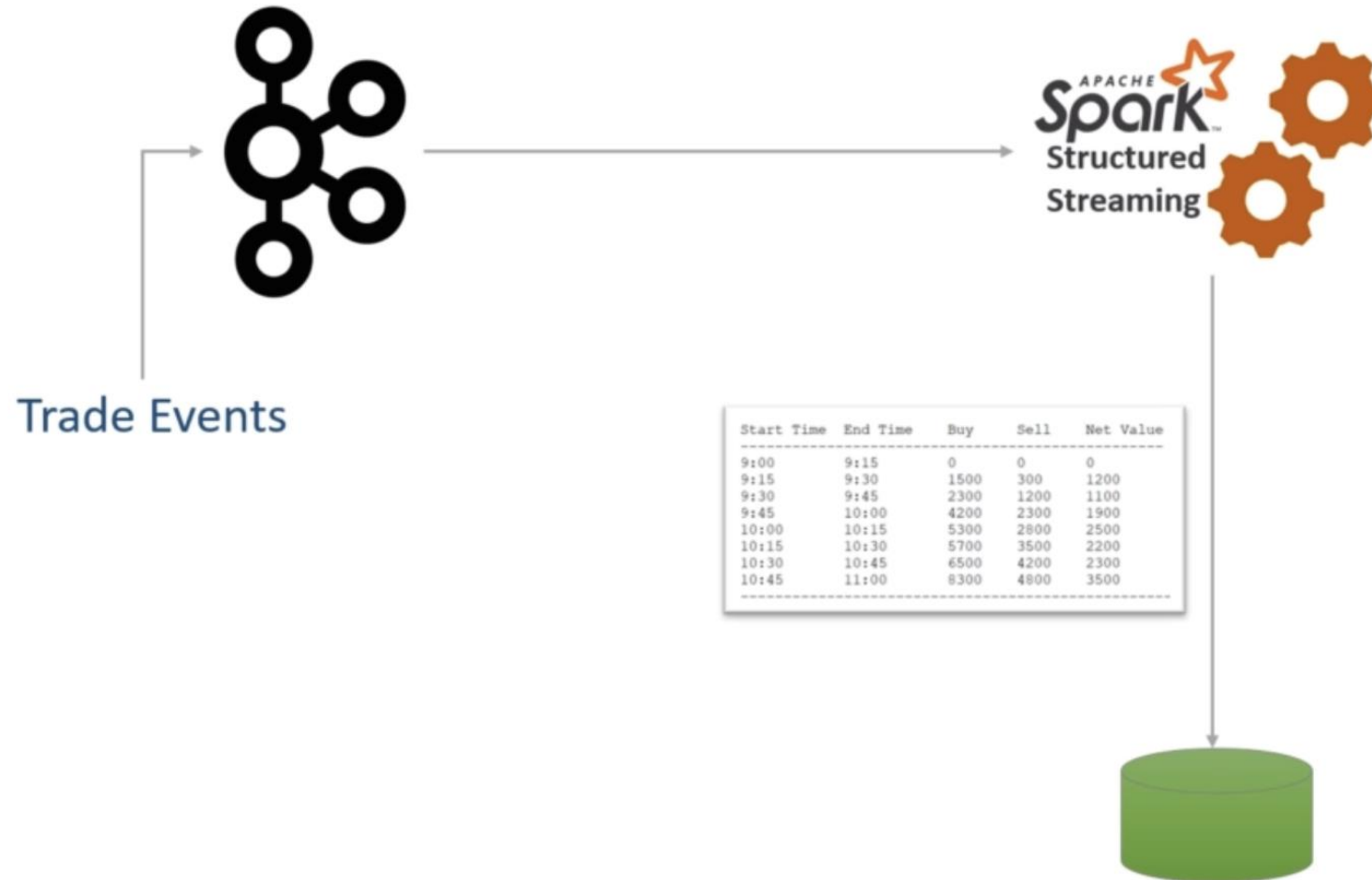
```
{ "TransactionTime": "2019-02-05 10:01:12",  
  "TransactionType": "BUY",  
  "TransactionAmount": 500,  
  "BrokerCode": "ABX" }
```

```
{ "TransactionTime": "2019-02-05 10:05:30",  
  "TransactionType": "SELL",  
  "TransactionAmount": 300, ,  
  "BrokerCode": "ABX" }
```

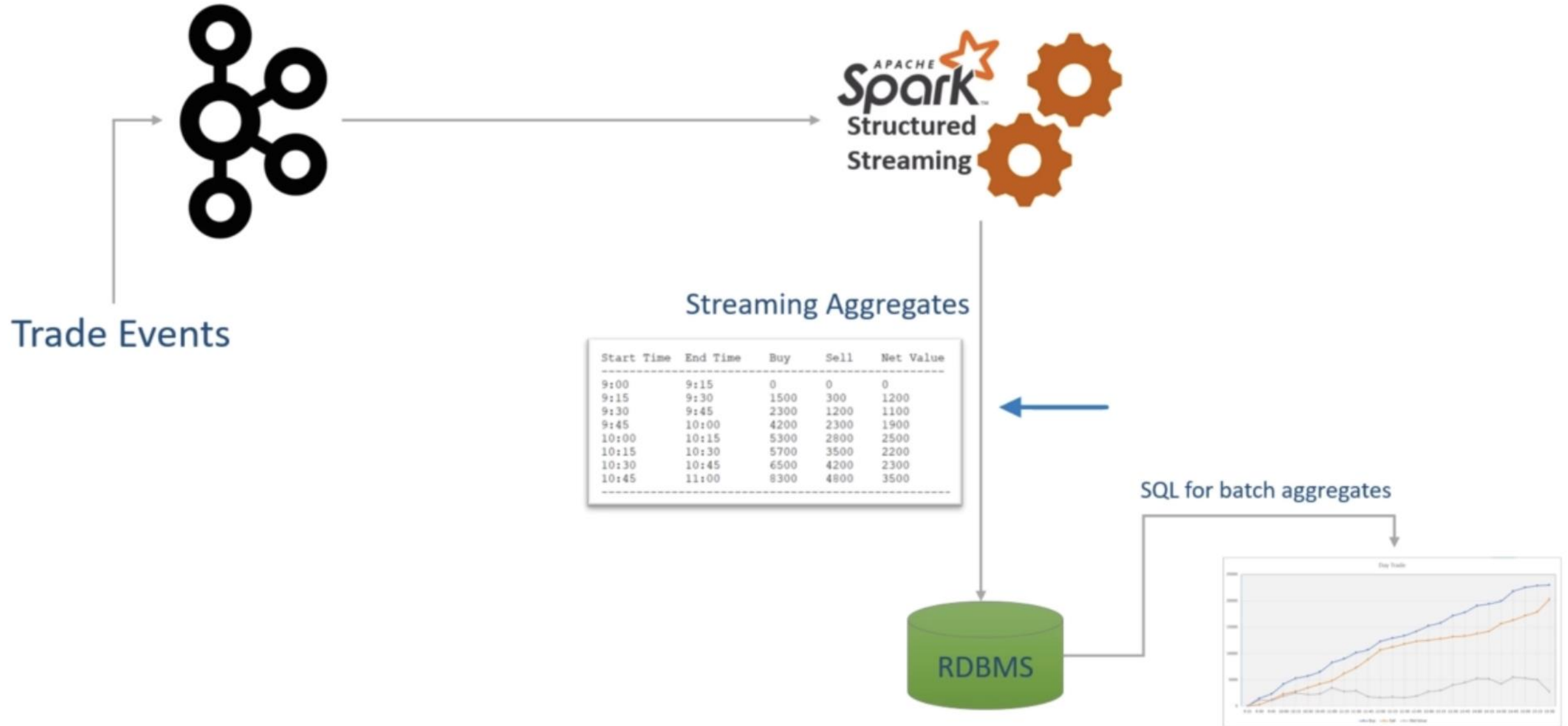
Start Time	End Time	Buy	Sell	Net Value

9:00	9:15	0	0	0
9:15	9:30	1500	300	1200
9:30	9:45	2300	1200	1100
9:45	10:00	4200	2300	1900
10:00	10:15	5300	2800	2500
10:15	10:30	5700	3500	2200
10:30	10:45	6500	4200	2300
10:45	11:00	8300	4800	3500

Tumbling Time Windows



Tumbling Time Windows

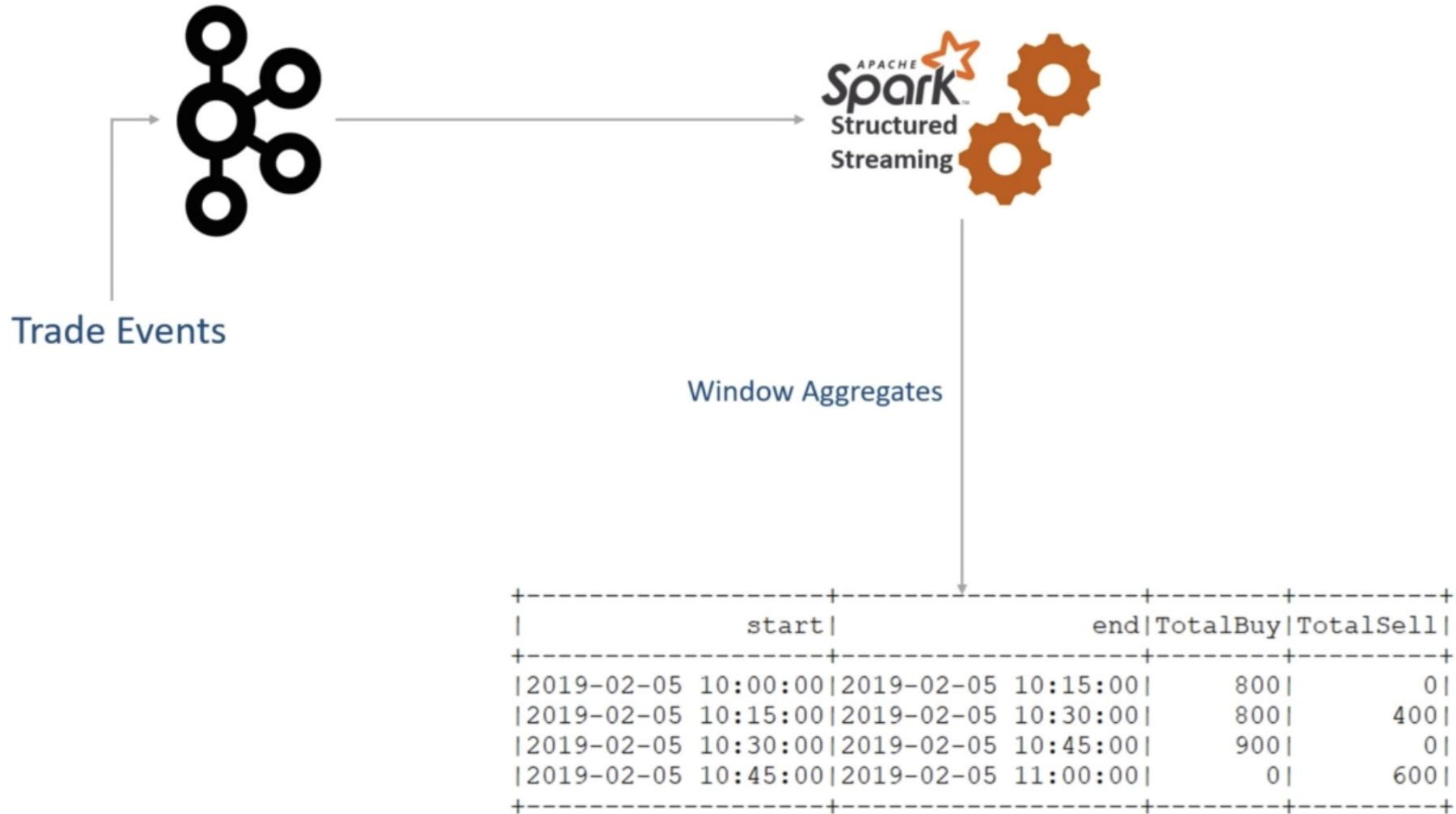




Summary

1. Tumbling Window Aggregates
2. What happens to late records
3. Limitations of Spark Streaming

Tumbling Time Windows

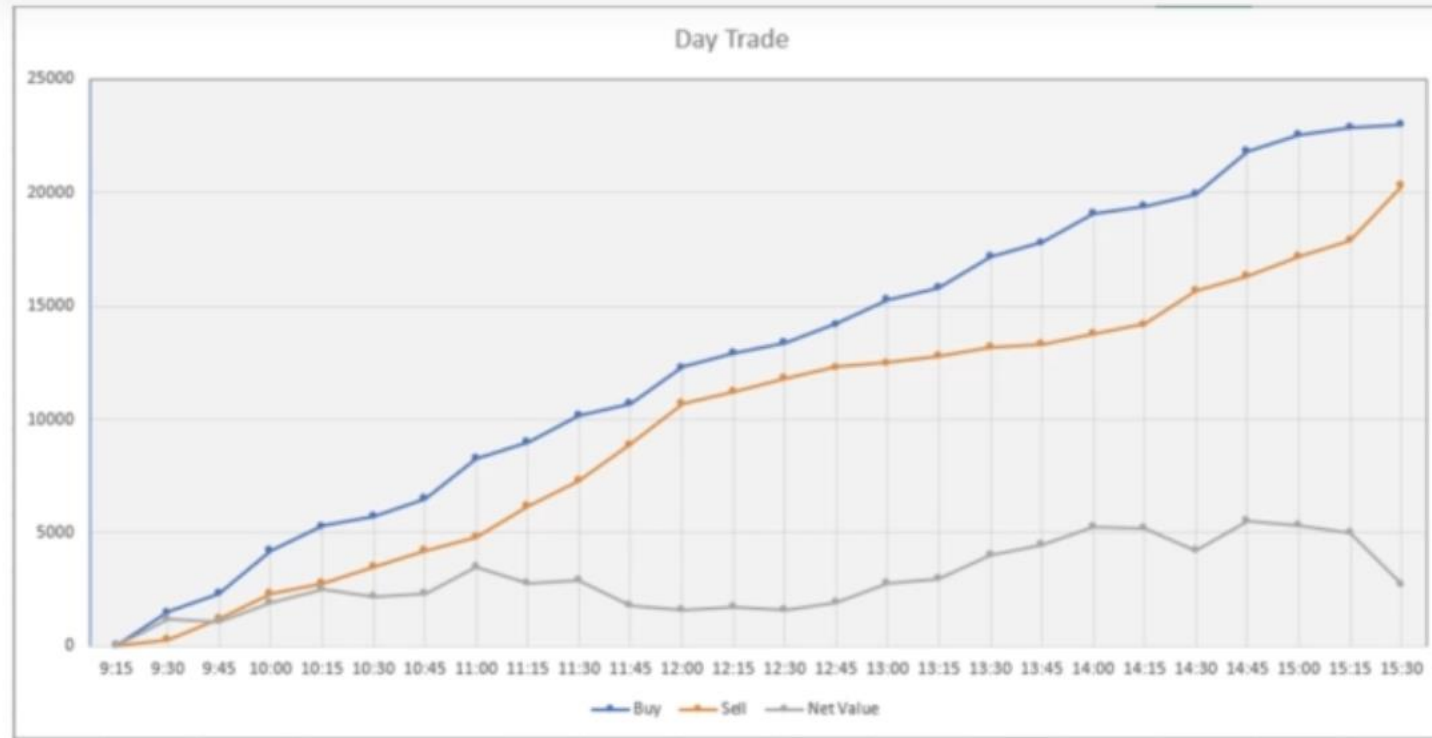




Watermark

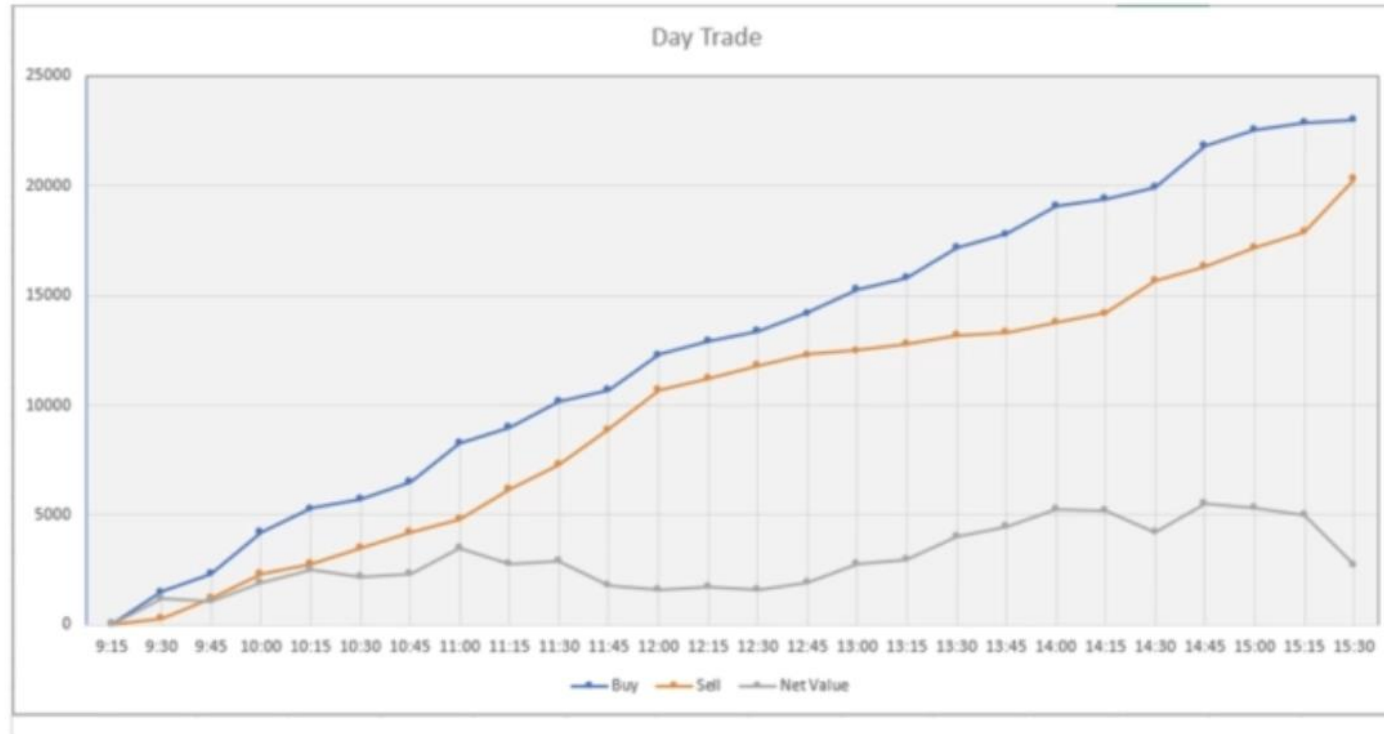
- 1. What is the maximum possible delay?
- 2. When late records are not relevant?

Watermark



9.30 ————— 6 hrs —————> 3.30

Watermark



Watermark = 6 hours

Watermark



```
trade_df = value_df.select("value.*") \
    .withColumn("CreatedTime", to_timestamp(col("CreatedTime"), "yyyy-MM-dd HH:mm:ss")) \
    .withColumn("Buy", expr("case when Type == 'BUY' then Amount else 0 end")) \
    .withColumn("Sell", expr("case when Type == 'SELL' then Amount else 0 end"))
```

```
window_agg_df = trade_df \
    .withWatermark("CreatedTime", "")
    .groupBy(window(col("CreatedTime"), "15 minute")) \
    .agg(sum("Buy").alias("TotalBuy"),
         sum("Sell").alias("TotalSell"))
```

```
output_df = window_agg_df.select("window.start", "window.end", "TotalBuy", "TotalSell")
```

```
window_query = output_df.writeStream \
    .format("console") \
    .outputMode("update") \
    .option("checkpointLocation", "chk-point-dir") \
    .trigger(processingTime="1 minute") \
    .start()
```

```
logger.info("Waiting for Query")
window_query.awaitTermination()
```

```
window_agg_df = trade_df \
    .withWatermark()
    .groupBy(window(col("CreatedTime"), "15 minute")) \
    .agg(sum("Buy").alias("TotalBuy"),
         sum("Sell").alias("TotalSell"))
```

Watermark



```
trade_df = value_df.select("value.*") \
    .withColumn("CreateTime", to_timestamp(col("CreateTime"), "yyyy-MM-dd HH:mm:ss")) \
    .withColumn("Buy", expr("case when Type == 'BUY' then Amount else 0 end")) \
    .withColumn("Sell", expr("case when Type == 'SELL' then Amount else 0 end"))
```

```
window_agg_df = trade_df \
    .withWatermark("CreateTime", "30 minute") \
    .groupBy(window(col("CreateTime"), "15 minute")) \
    .agg(sum("Buy").alias("TotalBuy"),
         sum("Sell").alias("TotalSell"))
```

```
output_df = window_agg_df.select("window.start", "window.end", "TotalBuy", "TotalSell")
```

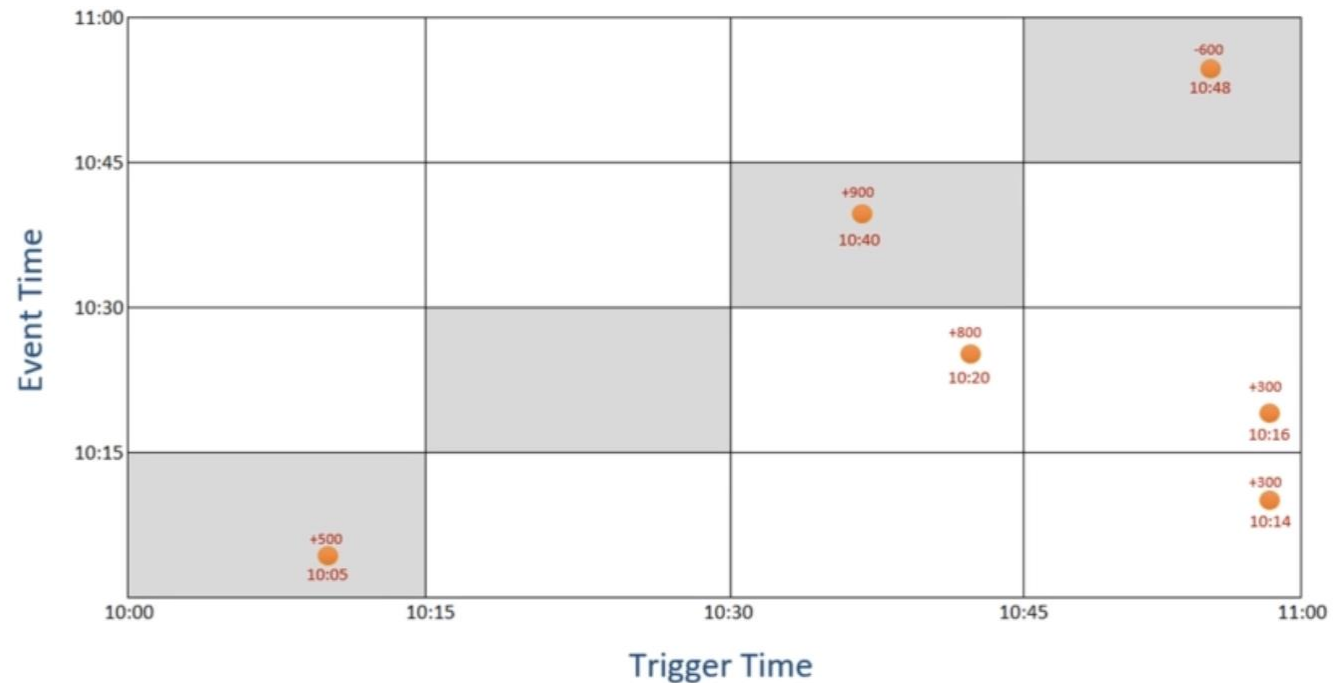
```
window_query = output_df.writeStream \
    .format("console") \
    .outputMode("update") \
    .option("checkpointLocation", "chk-point-dir") \
    .trigger(processingTime="1 minute") \
    .start()
```

```
logger.info("Waiting for Query")
window_query.awaitTermination()
```

Watermark



```
{ "CreatedTime": "2019-02-05 10:05:00", "Type": "BUY", "Amount": 500, "BrokerCode": "ABX" }  
{ "CreatedTime": "2019-02-05 10:40:00", "Type": "BUY", "Amount": 900, "BrokerCode": "ABX" }  
{ "CreatedTime": "2019-02-05 10:20:00", "Type": "BUY", "Amount": 800, "BrokerCode": "ABX" }  
{ "CreatedTime": "2019-02-05 10:48:00", "Type": "SELL", "Amount": 600, "BrokerCode": "ABX" }  
  
{ "CreatedTime": "2019-02-05 10:14:00", "Type": "SELL", "Amount": 300, "BrokerCode": "ABX" }  
{ "CreatedTime": "2019-02-05 10:16:00", "Type": "SELL", "Amount": 300, "BrokerCode": "ABX" }
```





- 1. Watermark is the key for state store cleanup
- 2. Events within the watermark is taken
- 3. Event outside the watermark may or may not be taken



State Store Cleanup

- 1. Setting a watermark
- 2. Using an appropriate output mode

Watermark vs Output Modes



Output Modes

1. Complete
2. Update
3. Append

Watermark Allows Append Mode With Aggregations



Append Mode

Allowed only when Spark knows the following
Record will not update or change in future



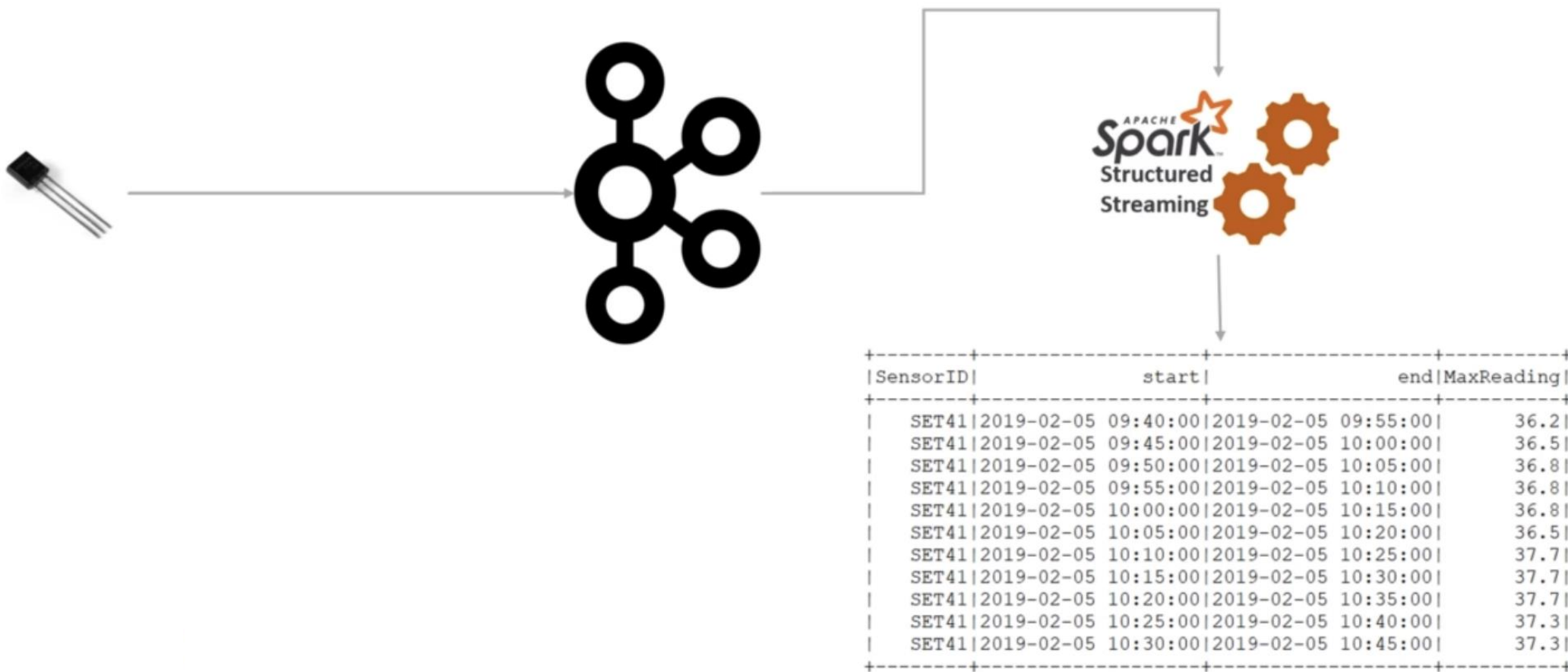
Time bound aggregates

1. Tumbling window
- 2. Sliding window

Sliding Windows



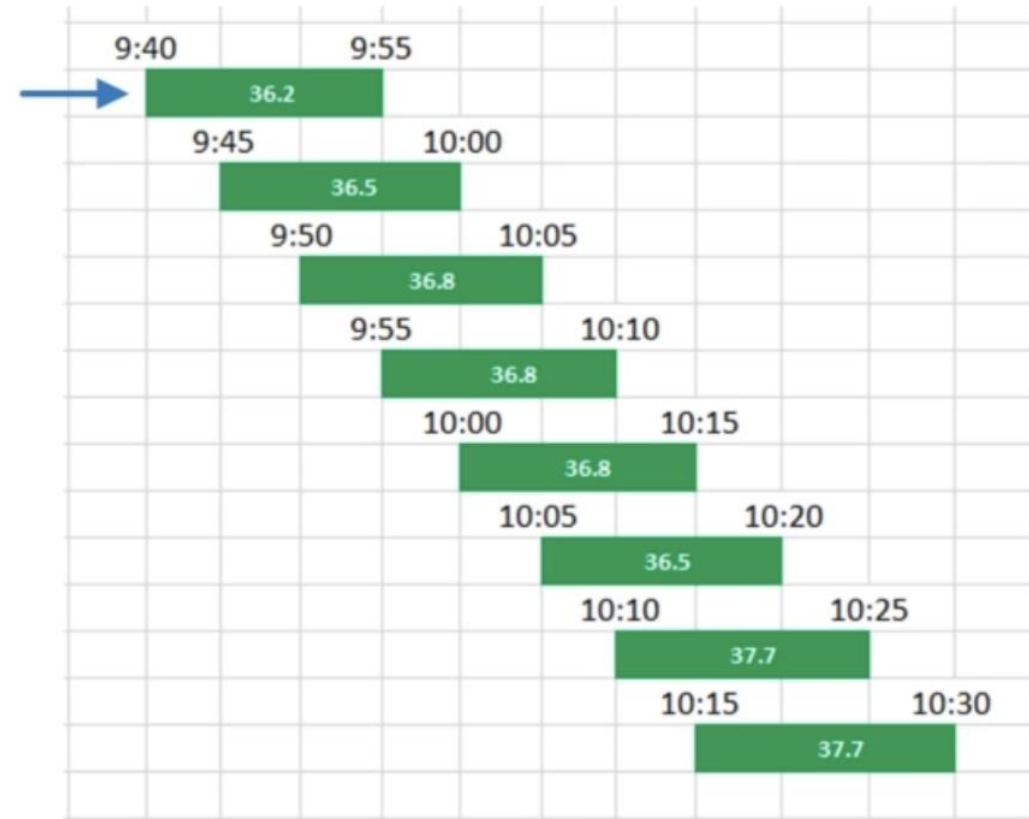
SET41:{"CreatedTime": "2019-02-05 09:54:00", "Reading": 36.2}



Sliding Windows



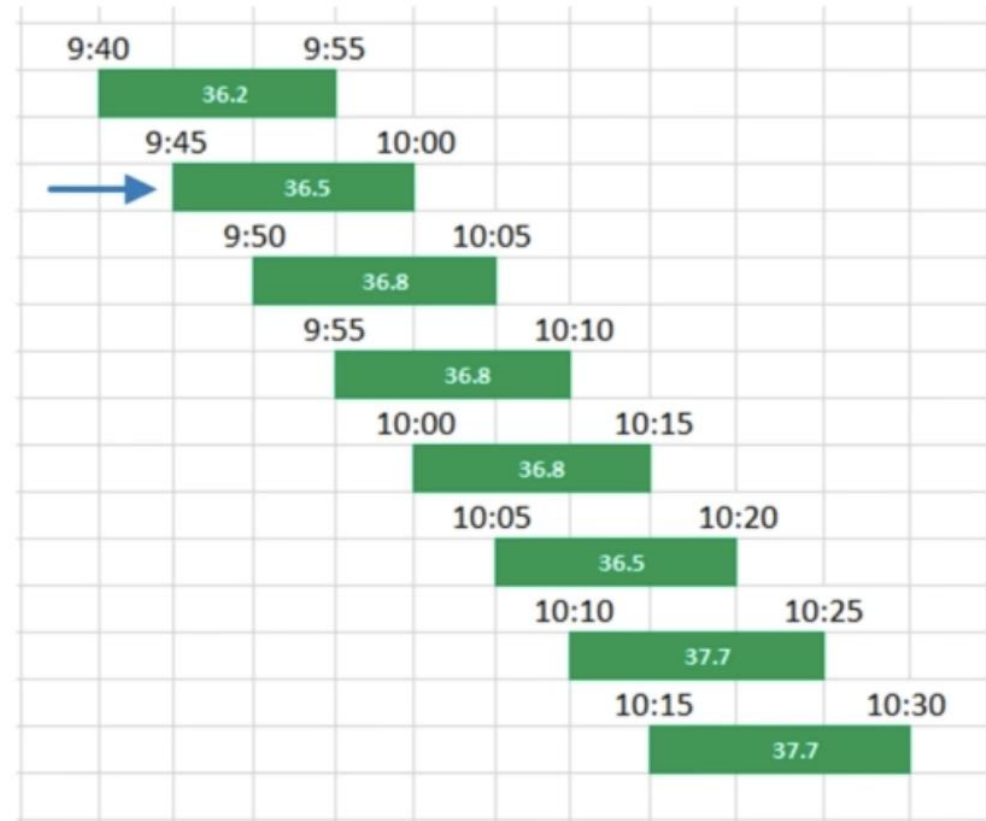
SensorID	start	end	MaxReading
SET41	2019-02-05 09:40:00	2019-02-05 09:55:00	36.2
SET41	2019-02-05 09:45:00	2019-02-05 10:00:00	36.5
SET41	2019-02-05 09:50:00	2019-02-05 10:05:00	36.8
SET41	2019-02-05 09:55:00	2019-02-05 10:10:00	36.8
SET41	2019-02-05 10:00:00	2019-02-05 10:15:00	36.8
SET41	2019-02-05 10:05:00	2019-02-05 10:20:00	36.5
SET41	2019-02-05 10:10:00	2019-02-05 10:25:00	37.7
SET41	2019-02-05 10:15:00	2019-02-05 10:30:00	37.7
SET41	2019-02-05 10:20:00	2019-02-05 10:35:00	37.7
SET41	2019-02-05 10:25:00	2019-02-05 10:40:00	37.3
SET41	2019-02-05 10:30:00	2019-02-05 10:45:00	37.3



Sliding Windows



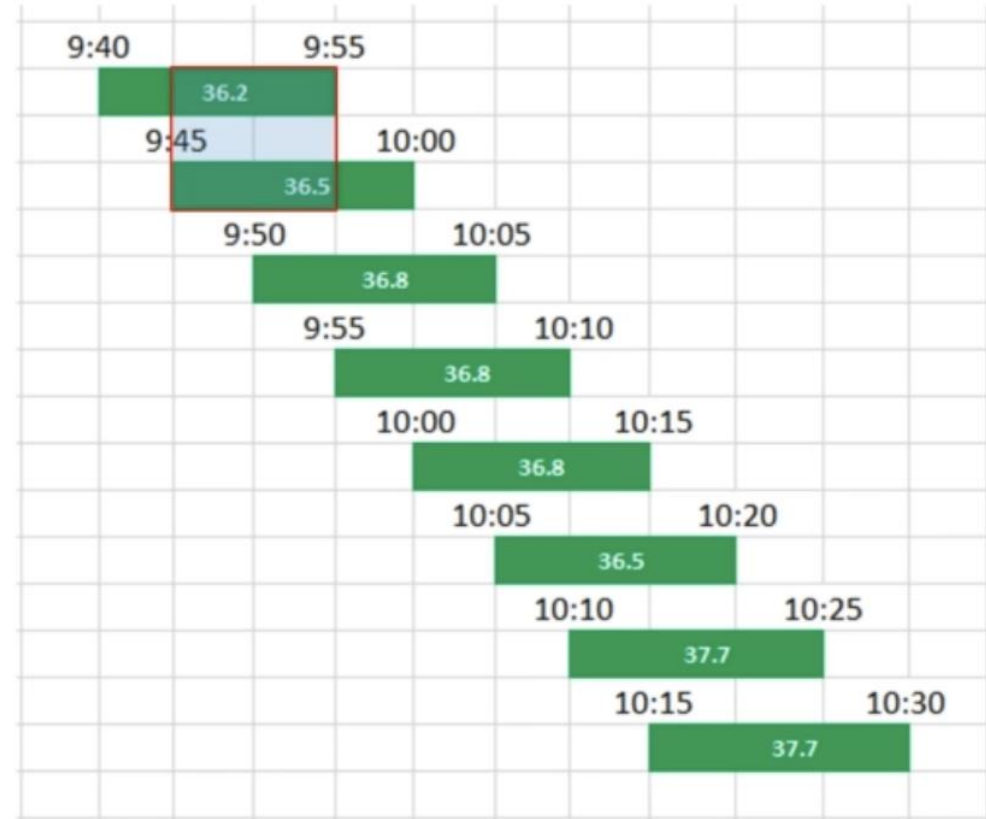
SensorID	start	end	MaxReading
SET41	2019-02-05 09:40:00	2019-02-05 09:55:00	36.2
SET41	2019-02-05 09:45:00	2019-02-05 10:00:00	36.5
SET41	2019-02-05 09:50:00	2019-02-05 10:05:00	36.8
SET41	2019-02-05 09:55:00	2019-02-05 10:10:00	36.8
SET41	2019-02-05 10:00:00	2019-02-05 10:15:00	36.8
SET41	2019-02-05 10:05:00	2019-02-05 10:20:00	36.5
SET41	2019-02-05 10:10:00	2019-02-05 10:25:00	37.7
SET41	2019-02-05 10:15:00	2019-02-05 10:30:00	37.7
SET41	2019-02-05 10:20:00	2019-02-05 10:35:00	37.7
SET41	2019-02-05 10:25:00	2019-02-05 10:40:00	37.3
SET41	2019-02-05 10:30:00	2019-02-05 10:45:00	37.3



Sliding Windows



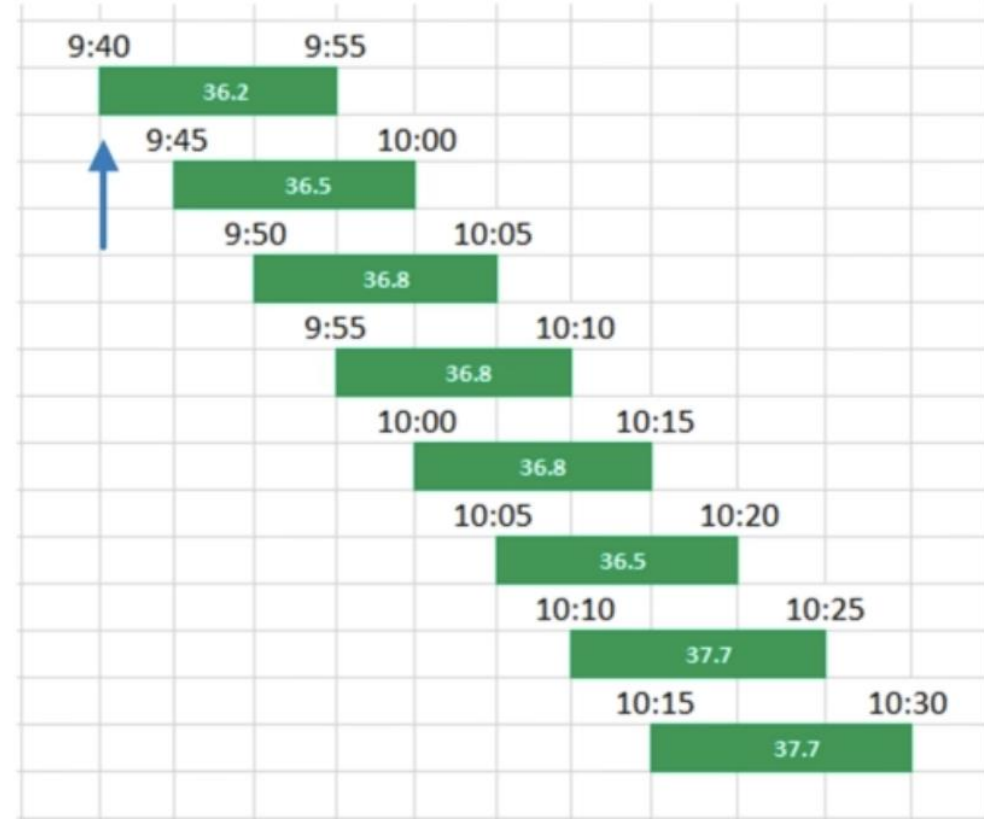
SensorID	start	end	MaxReading
SET41	2019-02-05 09:40:00	2019-02-05 09:55:00	36.2
SET41	2019-02-05 09:45:00	2019-02-05 10:00:00	36.5
SET41	2019-02-05 09:50:00	2019-02-05 10:05:00	36.8
SET41	2019-02-05 09:55:00	2019-02-05 10:10:00	36.8
SET41	2019-02-05 10:00:00	2019-02-05 10:15:00	36.8
SET41	2019-02-05 10:05:00	2019-02-05 10:20:00	36.5
SET41	2019-02-05 10:10:00	2019-02-05 10:25:00	37.7
SET41	2019-02-05 10:15:00	2019-02-05 10:30:00	37.7
SET41	2019-02-05 10:20:00	2019-02-05 10:35:00	37.7
SET41	2019-02-05 10:25:00	2019-02-05 10:40:00	37.3
SET41	2019-02-05 10:30:00	2019-02-05 10:45:00	37.3



Sliding Windows



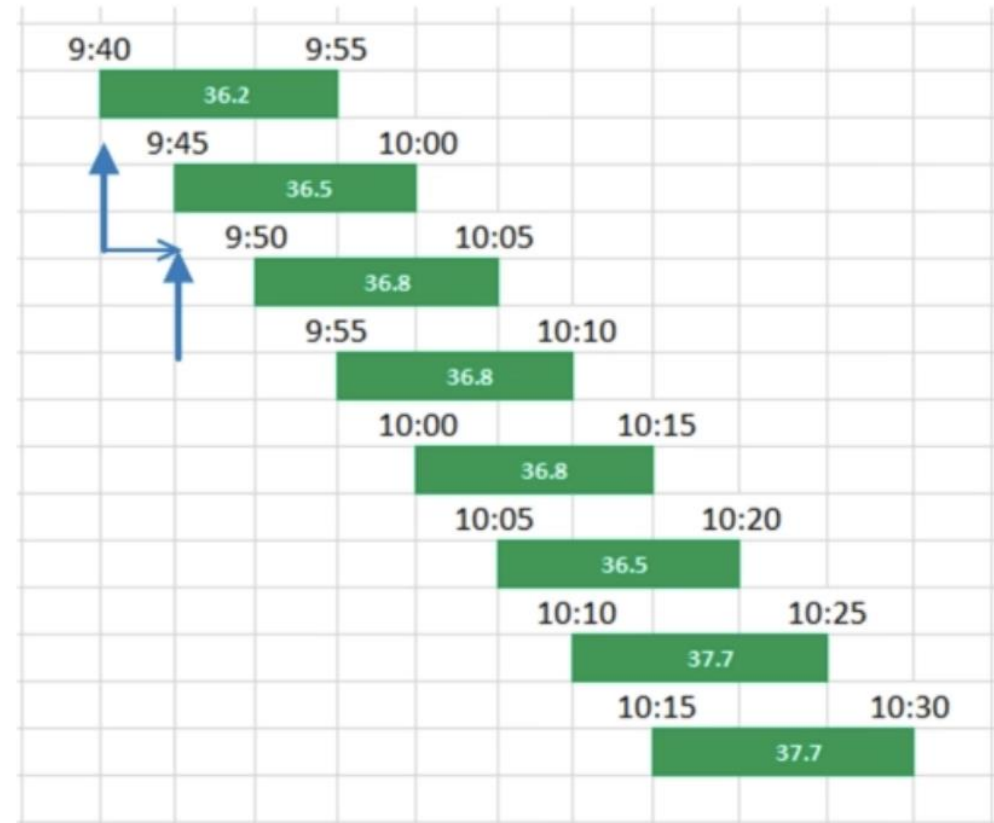
SensorID	start	end	MaxReading
SET41	2019-02-05 09:40:00	2019-02-05 09:55:00	36.2
SET41	2019-02-05 09:45:00	2019-02-05 10:00:00	36.5
SET41	2019-02-05 09:50:00	2019-02-05 10:05:00	36.8
SET41	2019-02-05 09:55:00	2019-02-05 10:10:00	36.8
SET41	2019-02-05 10:00:00	2019-02-05 10:15:00	36.8
SET41	2019-02-05 10:05:00	2019-02-05 10:20:00	36.5
SET41	2019-02-05 10:10:00	2019-02-05 10:25:00	37.7
SET41	2019-02-05 10:15:00	2019-02-05 10:30:00	37.7
SET41	2019-02-05 10:20:00	2019-02-05 10:35:00	37.7
SET41	2019-02-05 10:25:00	2019-02-05 10:40:00	37.3
SET41	2019-02-05 10:30:00	2019-02-05 10:45:00	37.3



Sliding Windows



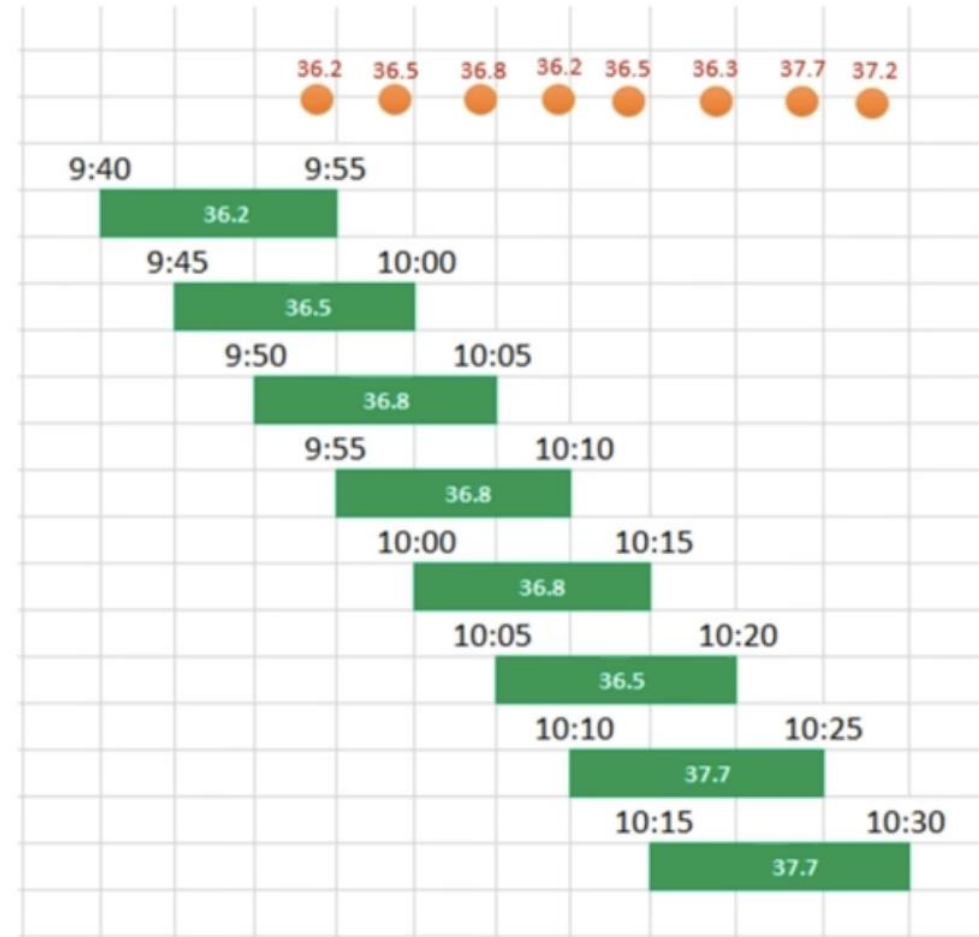
SensorID	start	end	MaxReading
SET41	2019-02-05 09:40:00	2019-02-05 09:55:00	36.2
SET41	2019-02-05 09:45:00	2019-02-05 10:00:00	36.5
SET41	2019-02-05 09:50:00	2019-02-05 10:05:00	36.8
SET41	2019-02-05 09:55:00	2019-02-05 10:10:00	36.8
SET41	2019-02-05 10:00:00	2019-02-05 10:15:00	36.8
SET41	2019-02-05 10:05:00	2019-02-05 10:20:00	36.5
SET41	2019-02-05 10:10:00	2019-02-05 10:25:00	37.7
SET41	2019-02-05 10:15:00	2019-02-05 10:30:00	37.7
SET41	2019-02-05 10:20:00	2019-02-05 10:35:00	37.7
SET41	2019-02-05 10:25:00	2019-02-05 10:40:00	37.3
SET41	2019-02-05 10:30:00	2019-02-05 10:45:00	37.3



Sliding Windows



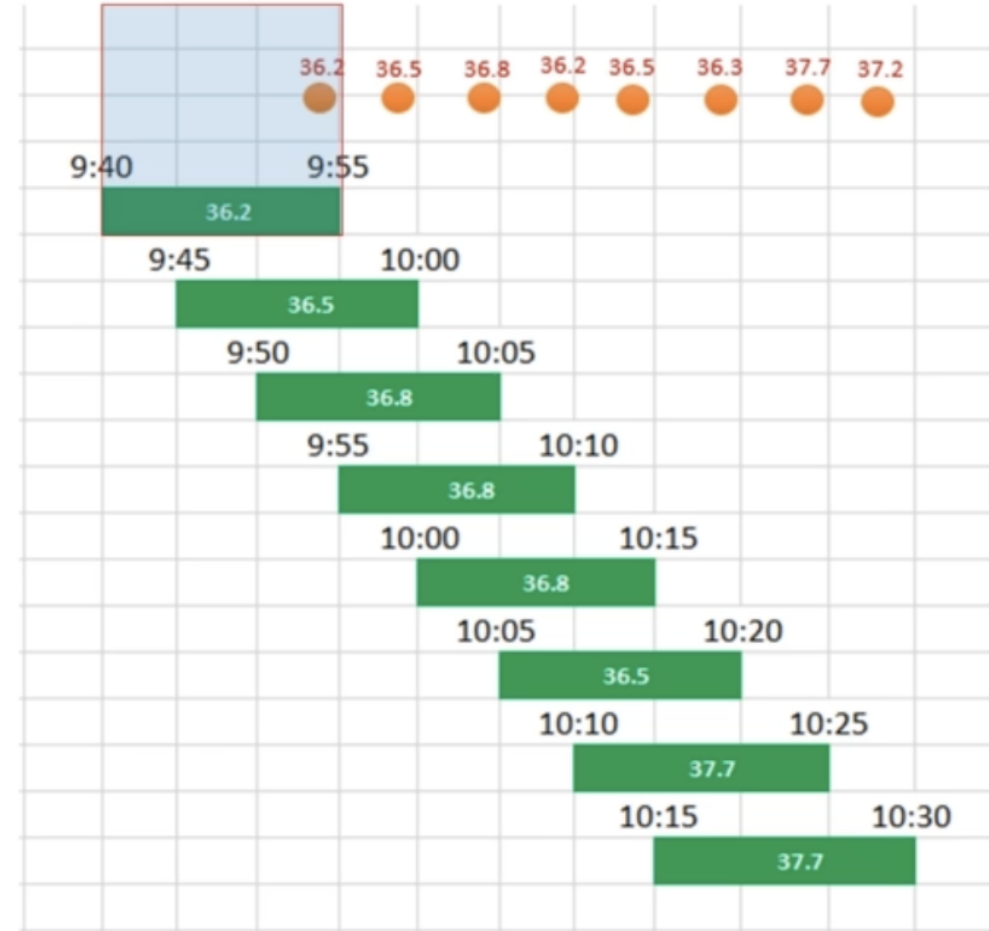
SensorID	start	end	MaxReading
SET41	2019-02-05 09:40:00	2019-02-05 09:55:00	36.2
SET41	2019-02-05 09:45:00	2019-02-05 10:00:00	36.5
SET41	2019-02-05 09:50:00	2019-02-05 10:05:00	36.8
SET41	2019-02-05 09:55:00	2019-02-05 10:10:00	36.8
SET41	2019-02-05 10:00:00	2019-02-05 10:15:00	36.8
SET41	2019-02-05 10:05:00	2019-02-05 10:20:00	36.5
SET41	2019-02-05 10:10:00	2019-02-05 10:25:00	37.7
SET41	2019-02-05 10:15:00	2019-02-05 10:30:00	37.7
SET41	2019-02-05 10:20:00	2019-02-05 10:35:00	37.7
SET41	2019-02-05 10:25:00	2019-02-05 10:40:00	37.3
SET41	2019-02-05 10:30:00	2019-02-05 10:45:00	37.3



Sliding Windows



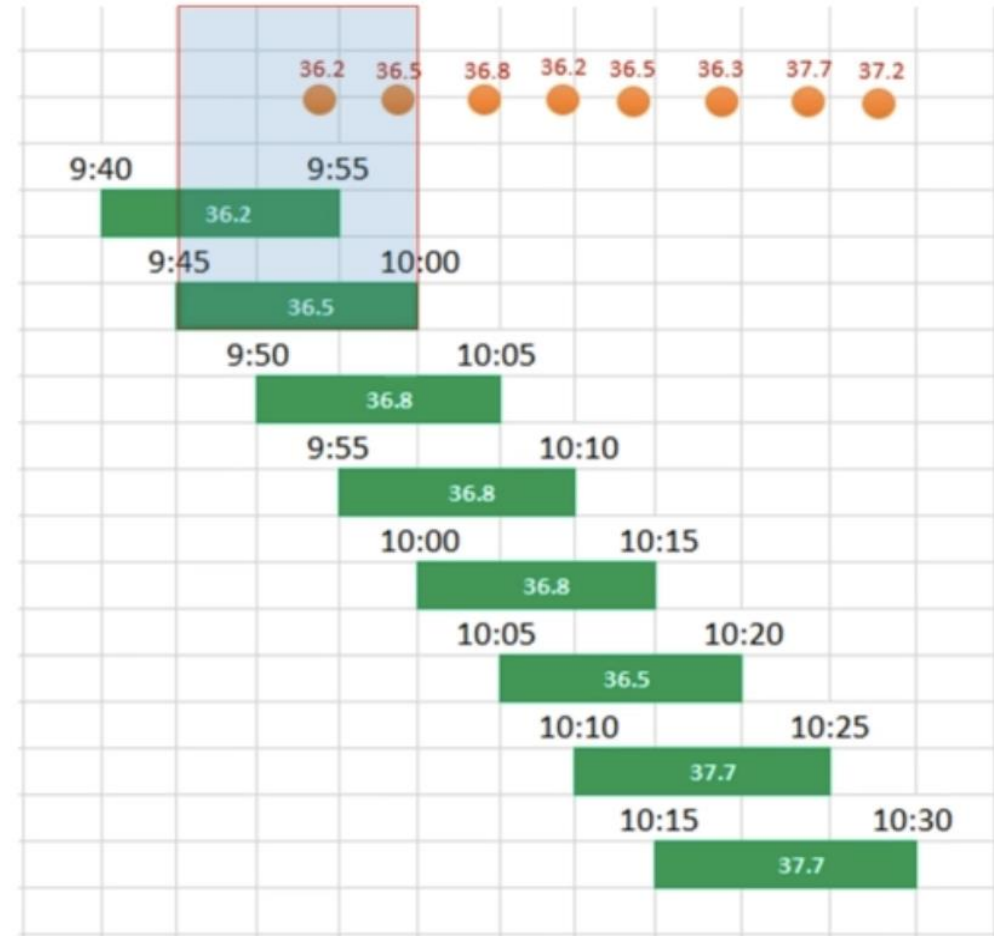
SensorID	start	end	MaxReading
SET41	2019-02-05 09:40:00	2019-02-05 09:55:00	36.2
SET41	2019-02-05 09:45:00	2019-02-05 10:00:00	36.5
SET41	2019-02-05 09:50:00	2019-02-05 10:05:00	36.8
SET41	2019-02-05 09:55:00	2019-02-05 10:10:00	36.8
SET41	2019-02-05 10:00:00	2019-02-05 10:15:00	36.8
SET41	2019-02-05 10:05:00	2019-02-05 10:20:00	36.5
SET41	2019-02-05 10:10:00	2019-02-05 10:25:00	37.7
SET41	2019-02-05 10:15:00	2019-02-05 10:30:00	37.7
SET41	2019-02-05 10:20:00	2019-02-05 10:35:00	37.7
SET41	2019-02-05 10:25:00	2019-02-05 10:40:00	37.3
SET41	2019-02-05 10:30:00	2019-02-05 10:45:00	37.3



Sliding Windows



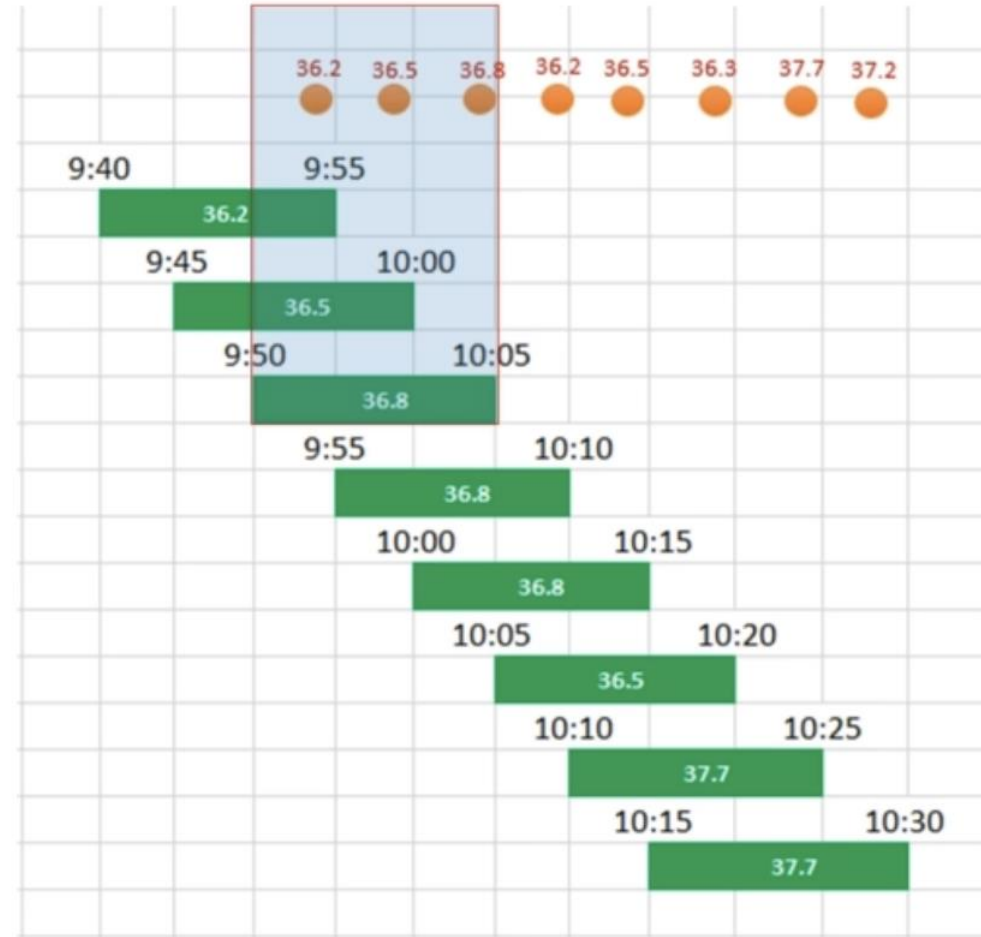
SensorID	start	end	MaxReading
SET41	2019-02-05 09:40:00	2019-02-05 09:55:00	36.2
SET41	2019-02-05 09:45:00	2019-02-05 10:00:00	36.5
SET41	2019-02-05 09:50:00	2019-02-05 10:05:00	36.8
SET41	2019-02-05 09:55:00	2019-02-05 10:10:00	36.8
SET41	2019-02-05 10:00:00	2019-02-05 10:15:00	36.8
SET41	2019-02-05 10:05:00	2019-02-05 10:20:00	36.5
SET41	2019-02-05 10:10:00	2019-02-05 10:25:00	37.7
SET41	2019-02-05 10:15:00	2019-02-05 10:30:00	37.7
SET41	2019-02-05 10:20:00	2019-02-05 10:35:00	37.7
SET41	2019-02-05 10:25:00	2019-02-05 10:40:00	37.3
SET41	2019-02-05 10:30:00	2019-02-05 10:45:00	37.3



Sliding Windows



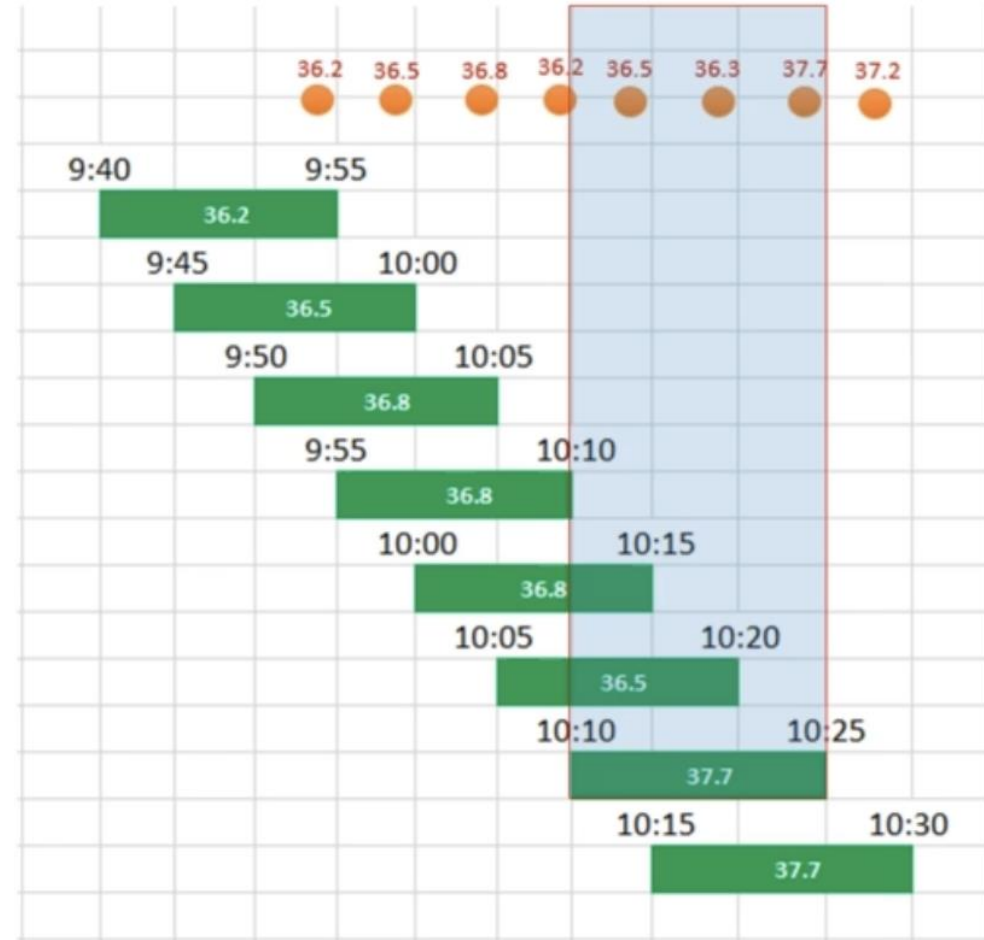
SensorID	start	end	MaxReading
SET41	2019-02-05 09:40:00	2019-02-05 09:55:00	36.2
SET41	2019-02-05 09:45:00	2019-02-05 10:00:00	36.5
SET41	2019-02-05 09:50:00	2019-02-05 10:05:00	36.8
SET41	2019-02-05 09:55:00	2019-02-05 10:10:00	36.8
SET41	2019-02-05 10:00:00	2019-02-05 10:15:00	36.8
SET41	2019-02-05 10:05:00	2019-02-05 10:20:00	36.5
SET41	2019-02-05 10:10:00	2019-02-05 10:25:00	37.7
SET41	2019-02-05 10:15:00	2019-02-05 10:30:00	37.7
SET41	2019-02-05 10:20:00	2019-02-05 10:35:00	37.7
SET41	2019-02-05 10:25:00	2019-02-05 10:40:00	37.3
SET41	2019-02-05 10:30:00	2019-02-05 10:45:00	37.3



Sliding Windows



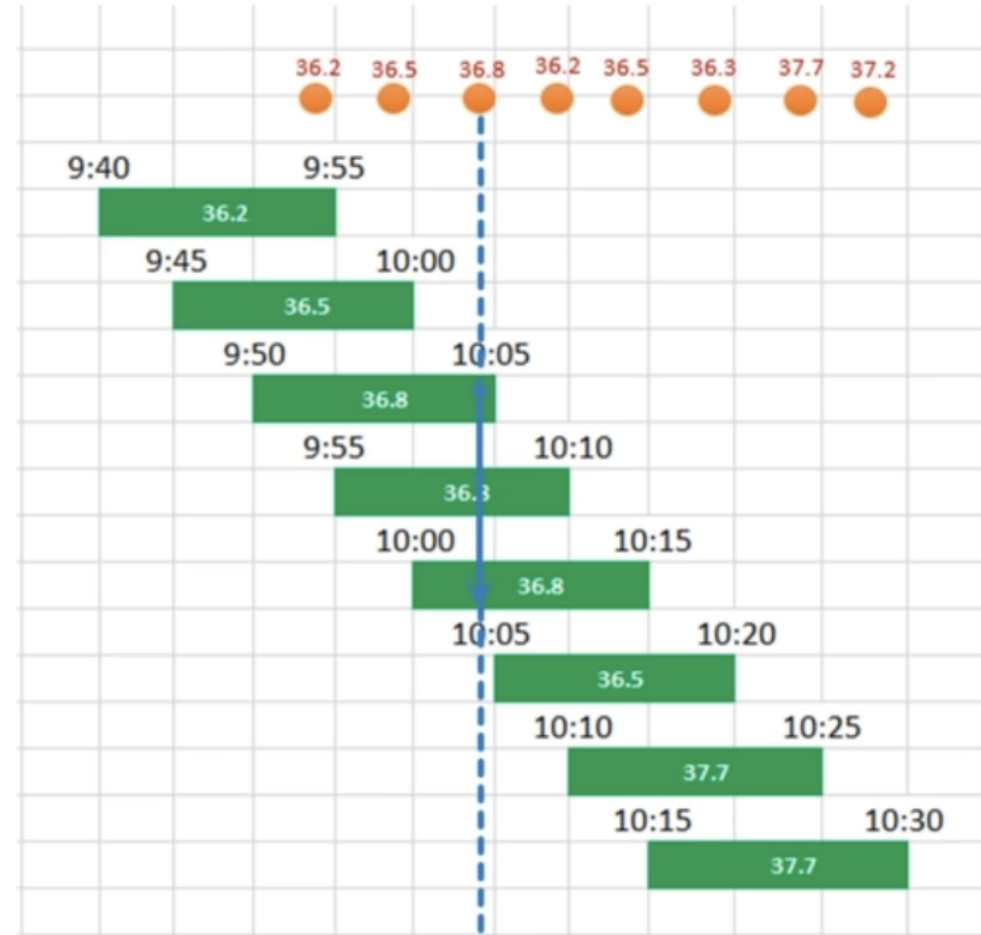
SensorID	start	end	MaxReading
SET41	2019-02-05 09:40:00	2019-02-05 09:55:00	36.2
SET41	2019-02-05 09:45:00	2019-02-05 10:00:00	36.5
SET41	2019-02-05 09:50:00	2019-02-05 10:05:00	36.8
SET41	2019-02-05 09:55:00	2019-02-05 10:10:00	36.8
SET41	2019-02-05 10:00:00	2019-02-05 10:15:00	36.8
SET41	2019-02-05 10:05:00	2019-02-05 10:20:00	36.5
SET41	2019-02-05 10:10:00	2019-02-05 10:25:00	37.7
SET41	2019-02-05 10:15:00	2019-02-05 10:30:00	37.7
SET41	2019-02-05 10:20:00	2019-02-05 10:35:00	37.7
SET41	2019-02-05 10:25:00	2019-02-05 10:40:00	37.3
SET41	2019-02-05 10:30:00	2019-02-05 10:45:00	37.3



Sliding Windows



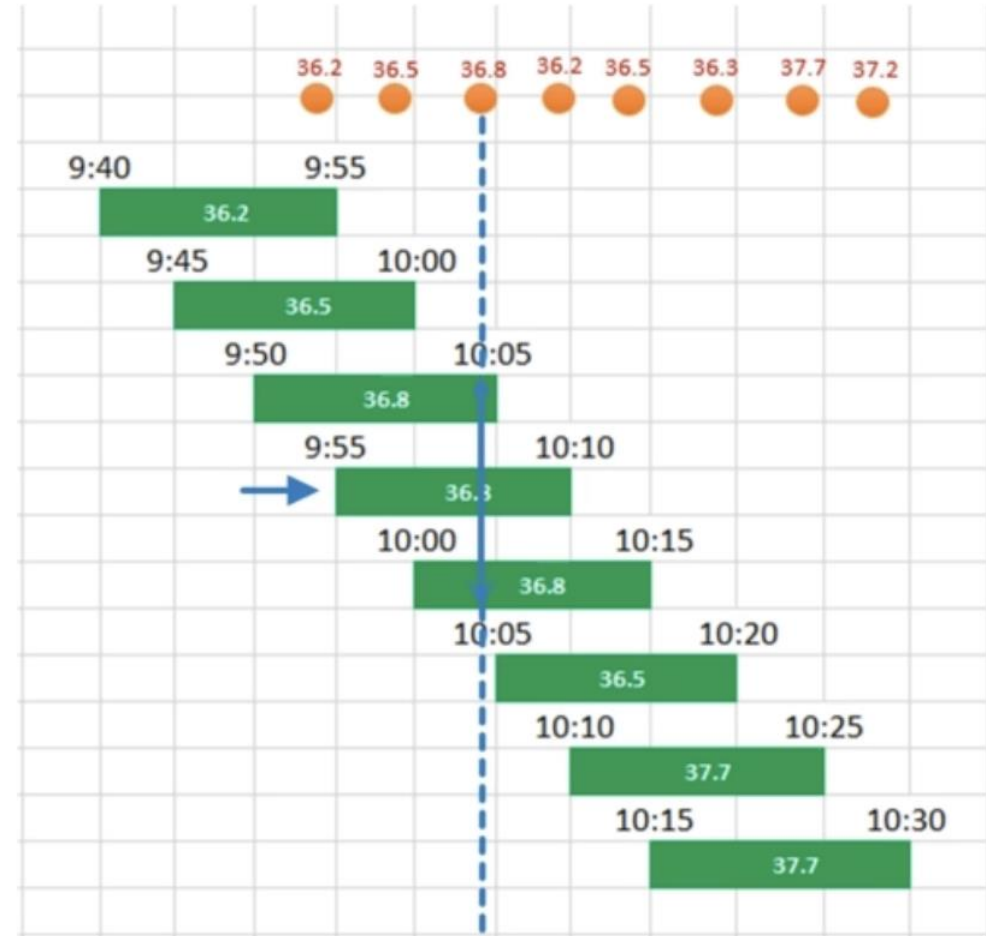
SensorID	start	end	MaxReading
SET41	2019-02-05 09:40:00	2019-02-05 09:55:00	36.2
SET41	2019-02-05 09:45:00	2019-02-05 10:00:00	36.5
SET41	2019-02-05 09:50:00	2019-02-05 10:05:00	36.8
SET41	2019-02-05 09:55:00	2019-02-05 10:10:00	36.8
SET41	2019-02-05 10:00:00	2019-02-05 10:15:00	36.8
SET41	2019-02-05 10:05:00	2019-02-05 10:20:00	36.5
SET41	2019-02-05 10:10:00	2019-02-05 10:25:00	37.7
SET41	2019-02-05 10:15:00	2019-02-05 10:30:00	37.7
SET41	2019-02-05 10:20:00	2019-02-05 10:35:00	37.7
SET41	2019-02-05 10:25:00	2019-02-05 10:40:00	37.3
SET41	2019-02-05 10:30:00	2019-02-05 10:45:00	37.3



Sliding Windows



SensorID	start	end	MaxReading
SET41	2019-02-05 09:40:00	2019-02-05 09:55:00	36.2
SET41	2019-02-05 09:45:00	2019-02-05 10:00:00	36.5
SET41	2019-02-05 09:50:00	2019-02-05 10:05:00	36.8
SET41	2019-02-05 09:55:00	2019-02-05 10:10:00	36.8
SET41	2019-02-05 10:00:00	2019-02-05 10:15:00	36.8
SET41	2019-02-05 10:05:00	2019-02-05 10:20:00	36.5
SET41	2019-02-05 10:10:00	2019-02-05 10:25:00	37.7
SET41	2019-02-05 10:15:00	2019-02-05 10:30:00	37.7
SET41	2019-02-05 10:20:00	2019-02-05 10:35:00	37.7
SET41	2019-02-05 10:25:00	2019-02-05 10:40:00	37.3
SET41	2019-02-05 10:30:00	2019-02-05 10:45:00	37.3



Sliding Windows



```
agg_df = sensor_df \  
    .withWatermark("CreatedTime", "30 minute") \  
    .groupBy(col("SensorID"),  
             window(col("CreatedTime"), "15 minute", "5 minute")) \  
    .agg(max("Reading").alias("MaxReading"))
```

Spark Structured Streaming and JOINs



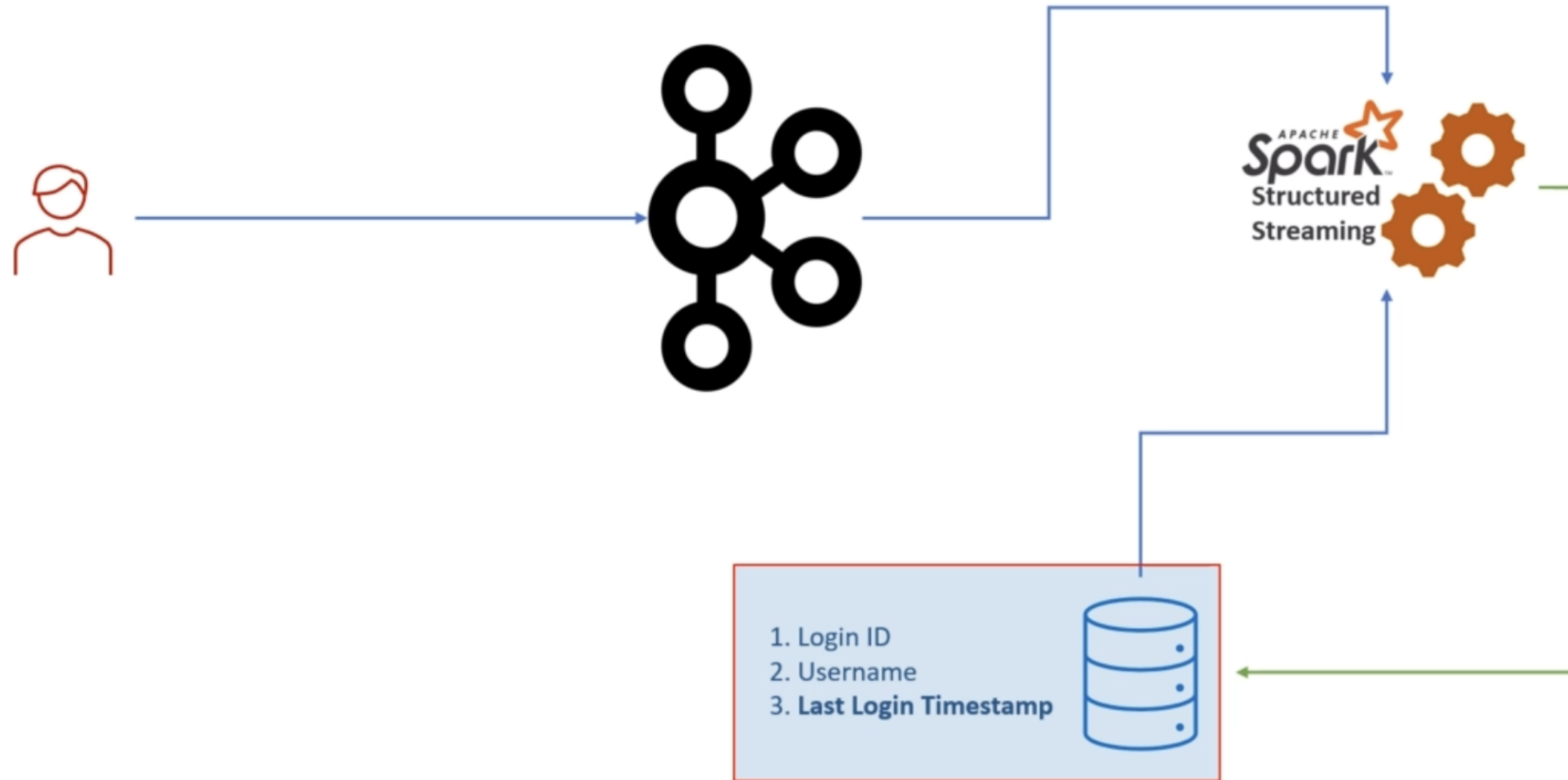
Spark Streaming Joins

1. Streaming Dataframe to Static Dataframe
2. Streaming Dataframe to Streaming Dataframe

Spark Structured Streaming and JOINS



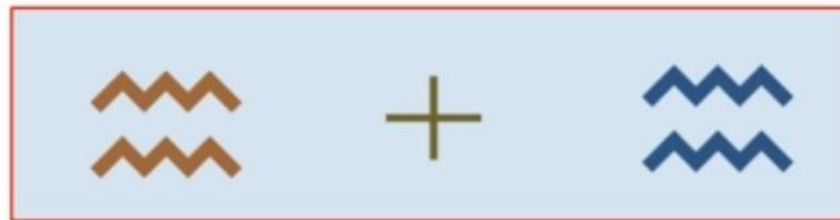
```
{"login_id": "100001", "created_time": "2020-09-09 10:18:00"}
```



Spark Structured Streaming and JOINS



Streaming Joins



Spark Structured Streaming and JOINS



Streaming Joins



.....



Spark Structured Streaming and JOINS



```
{"InventoryID": "100001", "CreatedTime": "2020-09-09 10:00:00", "Campaigner": "ABC Ltd"}
```



```
{"InventoryID": "100001", "CreatedTime": "2020-09-09 10:18:00"}
```



Clicked Impressions

Spark Structured Streaming and JOINS



```
joined_df = impressions_df.join(clicks_df, expr(join_expr), join_type)
output_query = joined_df.writeStream \
    .format("console") \
    .outputMode("append") \
```

|

Spark Structured Streaming and JOINS



```
joined_df = impressions_df.join(clicks_df, expr(join_expr), join_type)
output_query = joined_df.writeStream \
    .format("console") \
    .outputMode("append") \
    .option("checkpointLocation", "chk-point-dir") \
    .trigger(processingTime="1 minute") \
    .start()
```



Outer Joins in Spark Streaming

1. Left Outer – Conditionally Allowed
2. Right Outer – Conditionally Allowed
3. Full Outer – Not Allowed

Spark Structured Streaming and JOINS



Outer Joins in Spark Streaming



- 1. Left Outer
- a. Watermark on the right-side stream
 - b. Maximum time range constraint between the left and the right-side events
2. Right Outer
- a. Watermark on the left-side stream
 - b. Maximum time range constraint between the left and the right-side events